THM | CAMPUS FRIEDBERG | IEM Informationstechnik-Elektrotechnik-Mechatronik

TECHNISCHE HOCHSCHULE MITTELHESSEN

Detection and Mitigation of Rollback Attacks in Automotive Over-The-Air Updates

Master's Thesis

Submitted by

Tarush Gupta

Matrikel Nummer: 5457922

In partial fulfillment of the requirements for the degree of

Master of Science in Computer Control Communication Engineering

Supervised by

Prof. Dr.-Ing. Rahamatullah Khondoker (THM)

Mr. Mario Hoffmann (Group Lead - Automotive Cyber Resilience Department, ARRK Engineering GmbH, Munich)

Mr. Vatsal Patel (CyberSecurity Engineer, ARRK Munich GmbH, Munich)

Hesse, Germany

October 10, 2024

Detection and Mitigation of Rollback Attacks in Automotive Over-The-Air Updates

by

Tarush Gupta

ABSTRACT

In the modern times, with the integration of smart devices all around has made life easier, but along with bringing comfort to our day to day activities, there has been an increased risk in the privacy and security of the user of such smart devices. Similarly like the other industries, smart features have also become a part of the Automotive Industry transforming from traditional mechanical vehicles to hybrid models integrated with Electronics and Software inside the Vehicular systems, this in turn increases the landscape of problems relating to the cybersecurity of such devices. One critical aspect of modern automotive security is the provision of timely software updates by Original Equipment Manufacturer (OEM)s. These updates are often delivered over-the-air via wireless Wi-Fi connections, enhancing the functionality and quality of vehicle features. However, this process also represents a vulnerability point, as attackers may attempt to disrupt or manipulate the update process to either deny the update from reaching its destination or tamper with its functionality. These attacks can be broadly categorized into two types: Deny attacks, which aim to prevent the update from reaching its intended destination, and Deny Functionality attacks, which seek to interfere with or alter the functionality of the update itself. This thesis is dedicated to the development of innovative cybersecurity algorithms tailored to enhance or supplant existing security measures for updating systems, specifically targeting a distinct type of attack within the Over-The-Air (OTA) update framework. The selected focus revolves around one of the following attack the Rollback attack, aimed at deceiving an Electronic Control Unit (ECU) into installing outdated software containing known vulnerabilities.

Acknowledgment

I would like to express my deepest gratitude to those who have supported and guided me throughout my master's thesis journey.

First and foremost, I am sincerely grateful to Professor Rahamatullah Khondoker at Technische Hochschule Mittelhessen for his invaluable guidance, insightful feedback, and unwavering support. His expertise and encouragement have been instrumental in the successful completion of this thesis.

I extend my heartfelt thanks to ARRK Engineering GmbH Munich for providing me with the opportunity to work as a Master Thesis employee. I am particularly grateful to Mr. Mario Hoffmann and Mr. Vatsal Patel for their mentorship, patience, and the wealth of knowledge they have shared with me. Their practical insights and constructive suggestions have significantly enriched my research.

I would also like to thank my colleagues and friends for their continuous encouragement and assistance. Last but not least, my profound gratitude goes to my family for their unconditional support and encouragement throughout my academic endeavours.

Contents

Abstract **2**

Acknowledgments **3**

List of Figures **9**

List of Tables **11**

1 Background information and Introduction **13**
1.1 Secure OTA Update . . . 15
1.1.1 Principal In-Vehicle Modules in OTA Update Execution . . . 15
1.1.2 Cloud and Communication in OTA Update Execution . . . 19
1.1.3 Backend Infrastructure . . . 21
1.2 Understanding Rollback Attacks . . . 22
1.2.1 Rollback Attack Execution . . . 23
1.2.2 Consequences on the System due to Rollback attack . . . 23
1.2.3 Importance of mitigation . . . 24
1.3 Methodology . . . 24
1.4 Work Flow . . . 25

2 Literature Review **27**
2.1 Overview of Automotive Cybersecurity . . . 27
2.2 Existing solutions for Secure OTA Updates . . . 27
2.3 Analysis of Cybersecurity Protocols in OTA Update Automotive Systems . . . 29
2.4 Case Studies and Incident Reports: Version Rollback Attacks . . . 30

3 Research and Analysis **33**
3.1 Phase 1: Automotive Architecture, OTA Updates, Prototype Development . . . 34
3.1.1 Research Design . . . 34
3.1.2 Analysis . . . 34
3.1.3 Designing of Prototype . . . 34
3.2 Phase 2: OTA Update Implementation Reference Architectures . . . 35
3.2.1 Research Design . . . 35
3.2.2 Analysis . . . 36
3.3 Phase 3: Implementation of cyber-security and anti-rollback mechanisms . . . 39
3.3.1 Research Design . . . 39

3.3.2 Analysis . . . 40

4 System Architecture **51**
4.1 Overview of Proposed System prototype . . . 51
4.2 Detailed Description of System Components used in the prototype . . . 51
4.2.1 In-vehicle Modules . . . 52
4.2.2 Server and Cloud Infrastructure . . . 54
4.2.3 Backend System . . . 55
4.3 Communication protocols . . . 55
4.3.1 CAN Bus . . . 55
4.3.2 Wi-Fi . . . 55
4.4 OTA Feature and Security . . . 56

5 Implementation of the Prototype **59**
5.1 Development Environment . . . 59
5.2 Security Deployment . . . 63
5.2.1 NGINX Deployment . . . 63
5.2.2 Intrusion Detection Algorithms . . . 64

6 TARA Evaluation based on ISO21434 **67**
6.1 Evaluation Criteria . . . 67
6.1.1 Asset Identification . . . 67
6.1.2 Threat Scenario Identification . . . 69
6.1.3 Impact Rating . . . 72
6.1.4 Attack Path Analysis . . . 77
6.1.5 Attack Feasibility Rating . . . 82
6.1.6 Risk Value Determination . . . 82
6.1.7 Risk Treatment Decision . . . 83
6.2 Security Goals . . . 83
6.3 Security Concept . . . 83

7 Proposal of Security Algorithms **89**
7.1 Security Algorithms . . . 89
7.1.1 Algorithm 1: Feature Validation & Update Verification . . . 89
7.1.2 Algorithm 2: Serialising Update script lines/functions . . . 96
7.1.3 Algorithm 3: OTP Based Authentication with Rolling OTP Authentication . . . 102

8 Research Contributions **105**

9 Conclusion and Summary **107**
9.1 Conclusion of the Research . . . 107
9.2 Summary . . . 108

10 Future Work **109**

A Code listing **111**

B Abbreviations **121**

References **125**

List of Figures

1.1 Conventional OTA Update . . . 14
1.2 OTA Update Architecture . . . 16
1.3 Vehicle ECU functionality . . . 16
1.4 Role of cloud in OTA update Architecture . . . 20
1.5 Role of Backend in OTA update Architecture . . . 22
1.6 Workflow for Master Thesis . . . 26

3.1 Phases of the Methodology . . . 33
3.2 RAUC Architecture . . . 37
3.3 SWUpdate Architecture . . . 39

4.1 A/B Type Partition Architecture . . . 53
4.2 CAN-HAT Mounted on Raspberry Pi . . . 54

5.1 ECM Simulation Pannel . . . 60
5.2 Prototype of Automotive OTA update . . . 60
5.3 TCU Simulation Pannel . . . 61
5.4 SWUpdate make menuconfig . . . 62
5.5 Prototype Architecture . . . 65

6.1 ECM and TCU Possible Attack Paths . . . 78
6.2 Wi-fi Possible Attack Paths . . . 79
6.3 CAN bus Possible Attack Paths . . . 80

7.1 Algorithm 1: Data Flow Diagram . . . 91
7.2 SWUpdate .swu content . . . 92
7.3 Algorithm 1: Architecture . . . 93
7.4 Algorithm 2 Data Flow Diagram . . . 98
7.5 Authentication Work Flow . . . 103
7.6 Authentication Sequence Diagram . . . 104

List of Tables

2.1 Comparison of Cyber Security Techniques . . . 29
2.2 Recently Implemented or Discovered Version Rollback Attacks . . . 30

3.1 Overview of existing researches on Automotive OTA updates and its cyber-security properties . . . 42
3.2 Device Simulation Options and Considerations . . . 44
3.3 Backend Simulation Options and Considerations . . . 45
3.4 Summary of Software Update Architectures . . . 46
3.5 Reference Architectures Used in Automotive Industry . . . 47
3.6 Comparison of Uptane, RAUC, and SWUpdate architectures . . . 48
3.7 Comparison of NGINX, Apache, and Built-in Authentication . . . 48
3.8 OTA Optimization . . . 49
3.9 IDS used in Automotive Industry . . . 50
3.10 Comparison between Snort and Suricata . . . 50

6.1 Threat Scenarios for Different Assets 1 . . . 73
6.2 Threat Scenarios for Different Assets 2 . . . 74
6.3 Threat Scenarios for Public Verification Keys and Backend Infrastructure . . . 75
6.6 Risk Assessment Matrix . . . 82
6.4 Attack Paths and Feasibility Ratings 1 . . . 85
6.5 Attack Paths and Feasibility Ratings 2 . . . 86
6.7 Risk Assessment Table with Multiple Threat Scenarios . . . 87
6.8 Risk Assessment Table with Risk Treatment Decisions . . . 88

7.1 OTA Update Processes and Diagnostic Approaches of Various Automakers . . . 97
7.2 Automotive OEMs Using Static Code Analysis Tools . . . 100
7.3 Automotive Software Development Tools . . . 101
7.4 Comparison of Automotive Security Mechanisms with OTP and Rolling Code Systems . . . 104

Chapter 1

Background information and Introduction

In the realm of modern vehicles and infrastructure, we can only speculate about the innovative features introduced by each new technology. But how are these features integrated into these advancements? The answer lies in the fusion of electronics and computer science with traditional mechanical vehicles. By embedding small electronic chips and systems into cars, we have significantly enhanced their performance, comfort, control, and debugging capabilities. OEMs and other manufacturers can now easily identify and address specific issues within the vehicle's systems, facilitating repairs without disrupting the entire vehicle architecture. This chapter aims at providing some background information in the context of Automotive ECUs, Vehicle Architectures, Communication protocols, OTA Updates, Cybersecurity in Automotive Vehicles and Version Rollback attacks.

In modern vehicle, the technology is focused on less complexity and higher efficiency, keeping in mind the Cybersecurity of the system. A modern Smart Vehicle is equipped with different ECU which are responsible for different functions of the vehicle.

Each feature or system supported by the vehicle is implemented through the coordinated operation of multiple ECU, each managing different critical components of the vehicle. For instance, a seemingly simple functionality like adjusting the volume involves one ECU, specifically the Infotainment Control Unit (ICU). However, adding more features to this system requires additional ECUs. For example, if the volume can also be controlled via an OEM provided mobile app, the Telematics Control Unit (TCU) or Connectivity Control Unit (CCU) would also be involved.

These ECUs, do not only play an important role in managing the internal communication and functioning of the Automotive vehicle but also are responsible for external communication of networks, cellular data, remote connections etc. According to a report by Precedence Research [1] Modern Automobiles specifically cars are equipped with 70-100 ECUs in premium vehicles. Since the Automotive industry has moved from 100% mechanical based systems to an integration of electronics, programming and mechanical systems, working in sync to give a better user friendly and environment aware vehicle which reduces human effort and increases comfort in several ways possible, but with these benefits, the Automotive structure becomes vulnerable to cyber-threats. With the integration of ECUs and other electronic components, that control the main working systems, control of vehicle functionality

and, in some cases, the complete automotive vehicle can be gained by a third party through a series of cyber attacks for personal gain (e.g., theft of vehicles, theft of private information, ransom, causing deliberate accidents, etc.). To prevent this from happening, several cyber-security protocols like encryption, hashing etc. are put in place according to the respective need of each component in the vehicular working system. However, overtime vulnerabilities tend to be found in such systems by several experts (Black hats/White hats/Gray hats), either for personal gain or for the safety of people and the benefit of the OEMs. In any case the systems need to be kept secure by the OEMs from newly discovered vulnerabilities that can be exploited by such attackers, ensuring not just the safety of people and customers but also the maintenance of the OEM's image. In earlier times, these problems could be addressed by a timely trip to the nearby OEM service/repair centers where the Update on the OEM server could only be accessed by the authorized dealership or the authorized repair as shown in Figure 1.1: Conventional OTA Update.

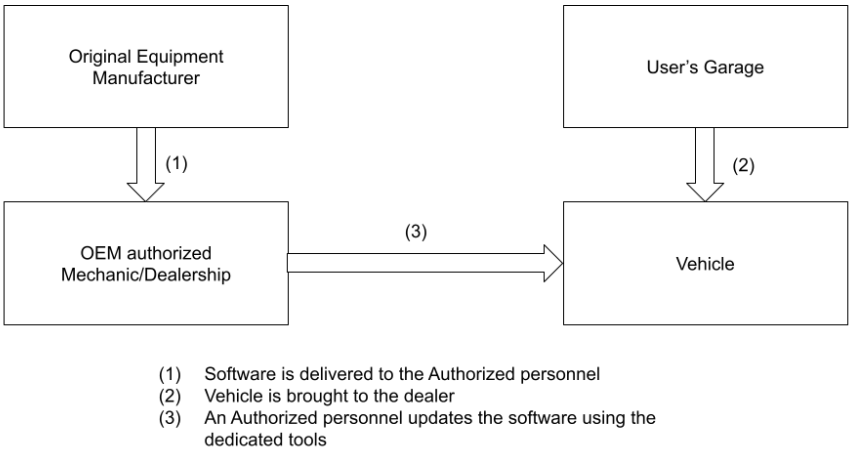


Figure 1.1: Conventional OTA Update

Traditionally, a visit to a service center was required for vehicle software updates, where updates were manually installed by technicians via physical connections. This process was not only time-consuming and inconvenient for vehicle owners but also resulted in delays in the implementation of critical security patches and feature enhancements. With the advent of OTA updates, updates can now deliver updates wirelessly by manufacturers directly to vehicles, without the need for physical intervention. This seamless approach ensures directly to vehicles, without the need for physical intervention. This seamless approach ensures that updates are deployed more efficiently, keeping vehicles up-to-date with the latest advancements and security measures. OTA updates are considered a crucial component of modern

vehicle maintenance and enhancement, allowing updates, security patches, and new features to be deployed by OEMs without the need for physical intervention.

1.1 Secure OTA Update

While OTA updates can enhance the entire system and shield it from attackers exploiting newly discovered vulnerabilities, it is also crucial to consider the implementation of cybersecurity measures within the OTA update architecture. If system or data packages become corrupted during the transmission or reception of OTA updates, the entire system can be compromised, as the corrupted data packets can affect several target systems in an automotive vehicle. To prevent this from happening, cybersecurity protocols are implemented in the OTA update architecture. Several types of attacks can compromise a system via an attack on the OTA transmission of update files. The main attacks include:

1. Deny Attacks: These attacks disrupt the communication channels or the update process, preventing the update file from reaching its intended destination. This can leave the system outdated and more vulnerable to attacks.

2. Deny Functionality Attacks: These attacks interfere with the update file, altering its code to introduce malicious functionality or disable key features, effectively denying the update's intended functionality.

3. Rollback Attacks: Rollback attacks deceive the vehicle's ECUs into installing an older version of the software that contains known vulnerabilities

This thesis specifically focuses on detecting and mitigating rollback attacks during OTA updates. Rollback attacks can undermine the entire update process by reverting the software to a less secure state. Tackling this issue is essential for maintaining the long-term security and reliability of vehicle systems.

As represented in the 1.2 several components are involved in the complete process of the OTA update architecture. To secure the OTA update transmission, the components involved in this structure of OTA must first be secured. These components can be divided into the following categories.

1. In-Vehicle Components

2. Cloud and Communication

3. Backend Infrastructure

1.1.1 Principal In-Vehicle Modules in OTA Update Execution

In the system proposed in this Master Thesis, the focus is on OTA updates and their cybersecurity. Consequently, the primary concern is with a select few ECUs that are integral to achieving successful OTA updates. Below is a basic overview of these different ECUs:

Image source Figure 1.3: ARRK Engineering GmbH, Munich

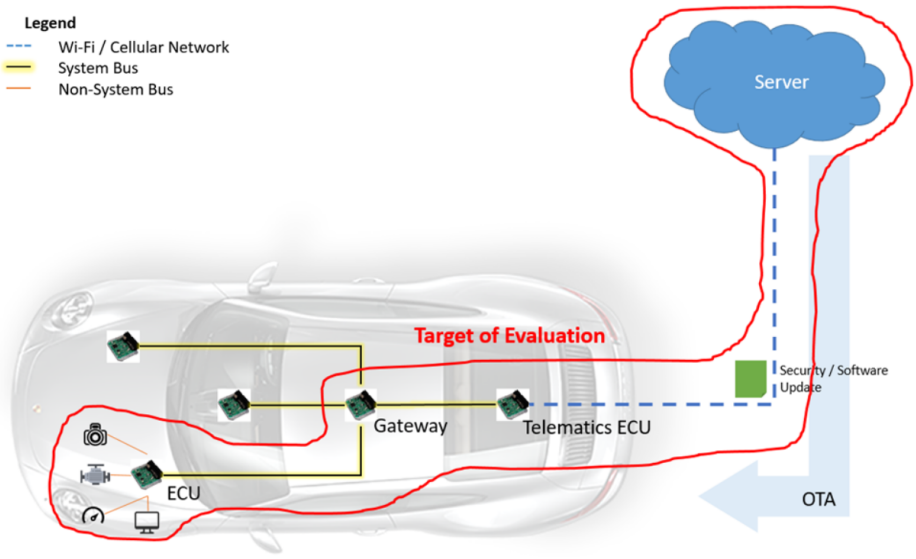


Figure 1.2: OTA Update Architecture

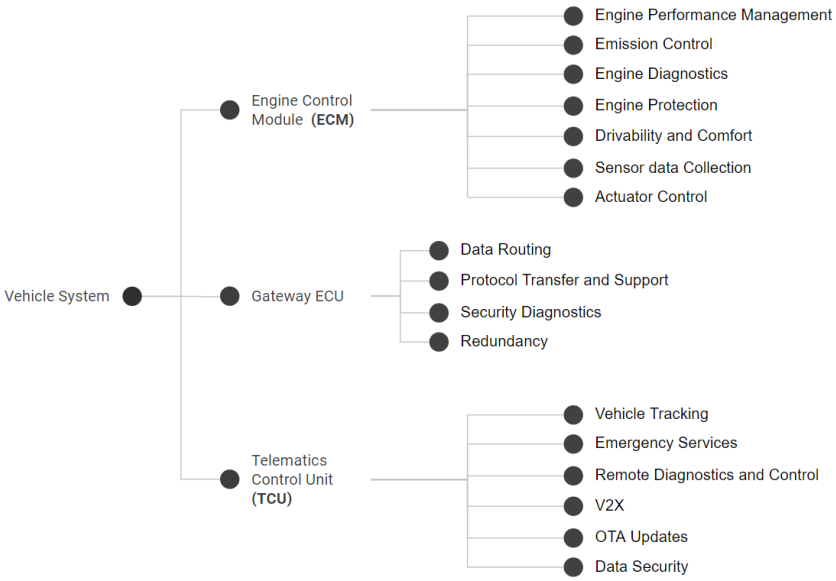


Figure 1.3: Vehicle ECU functionality

The following is a basic overview of the Different ECUs:

1. Engine Control Unit (Module) (ECM): This ECU is responsible for the different functions and features, related to the engine of the vehicle:
   - Engine Performance Management:
     - It is responsible for controlling the timing and the amount of fuel injected in the engine to optimize combustion and efficiency.

  - It also is responsible for the spark timing to improve, the ignition of the air-fuel mixture
  - It also adjusts the air-fuel mixture ratio for efficient combustion
- Emission Control:
  - It manages Exhaust Gas Recirculation (EGR) to reduce nitrogen oxide emissions and reintroduces a portion of exhaust gas back into the engine for improved efficiency.
  - It monitors the performance of the catalytic converter and adjusts engine parameters to ensure optimal operation.
- Engine Diagnostics:
  - The ECM stores Diagnostic Trouble Codes (DTC)s and uses On-Board Diagnostics (OBD) systems to help mechanics identify and troubleshoot engine problems, highlighting the need for secure updates to avoid diagnostic data manipulation.
  - It continuously monitors data from various sensors (e.g., oxygen sensor, mass airflow sensor, throttle position sensor) to detect any issues or abnormalities.
- Engine Protection:
  - The ECM can adjust engine operations to prevent overheating by controlling the cooling system and engine load.
  - It detects engine knocking and can adjust the ignition timing to prevent engine damage.
- Driveability and Comfort:
  - The ECM maintains a stable idle speed by adjusting the throttle and fuel injection.
  - It helps manage the cruise control system by maintaining the set speed.

  The ECM interconnected with various sensors, actuators, and other control modules in the vehicle.
  - Sensors: The ECM is interconnected with several sensors that provide information regarding the vehicle parameters (primarily engine parameters). The integrated sensors include:
    (a) Oxygen Sensors (O2 Sensors)
    (b) Mass Air Flow Sensor (MAF)
    (c) Throttle Position Sensor (TPS)
    (d) Engine Coolant Temperature Sensor
    (e) Crankshaft Position Sensor
    (f) Camshaft Position Sensor
  - Actuators: The ECM is also connected with several actuators, that help the engine functionality based on the sensor data received, so that the engine runs smoothly and efficiently. The following are the connected Actuators:

(a) Fuel Injectors: Controlled by the ECM to deliver the precise amount of fuel to the engine cylinders.

(b) Ignition Coils: Triggered by the ECM to provide the spark needed for combustion.

(c) EGR Valve: Managed by the ECM to control the flow of exhaust gases back into the engine.

(d) Idle Air Control Valve: Adjusts the engine's idle speed based on the ECM's instructions.

2. TCU: This ECU is an integral part of the system in any modern automotive vehicle. As the name suggests, the TCU is primarily responsible for telecommunications within a vehicle, facilitating communication between the vehicle and external networks such as Wi-Fi or cellular networks. One of the critical features the TCU offers is support for OTA updates. OTA updates allow for the remote updating of vehicle software and firmware, ensuring that vehicles can receive the latest features, improvements, and security patches without the need for a physical connection or a visit to a service center. This capability not only enhances vehicle performance and user experience but also plays a vital role in maintaining the cybersecurity and overall reliability of modern automotive systems, underscoring the importance of the TCU in the automobile industry.

   (a) Vehicle Tracking: The TCU uses the Global Positioning System (GPS) module to determine and monitor the Vehicle position and tracking

   (b) Emergency Services: In case of emergency eg. an accident or a crash the TCU can notify the emergency services.

   (c) Remote Diagnostics: The TCU is responsible for reading the vehicle data and Diagnostics and sending it to the OEM servers for maintains and update purposes.

   (d) Infotainment Systems: The TCU provides internet connectivity for steaming and other services for the infotainment systems in a vehicle.

   (e) Vehicle to Everything (V2X): It also supports V2X communication, encompassing both Vehicle to Vehicle (V2V) and V2I interactions, thereby enhancing traffic safety and efficiency.

   (f) Remote control: In modern vehicles users can also control several features of the vehicle remotely which is also enabled by the TCU.

   (g) OTA Updates: Since the TCU is the only module that connects to the external networks, it is also responsible for pulling updates for the servers of the OEMs to keep the vehicles secure, up to date and enable it with new features and programs.

   The TCU provides the following features:

   (a) Data Security: Ensure that the communications and updates are secured and protected from unauthorized access.

(b) User-Friendly User Interface (UI): It offers users a friendly and inclusive user interface, for users.

(c) Power Management: It optimizes battery consumption over different systems in the vehicles to use the battery efficiently.

(d) Integration of all Vehicular systems: It is connected to various vehicular systems via different protocols like CAN, LIN, FlexRay etc. to provide comprehensive telemetries services.

3. Gateway ECU: The Gateway ECU is a fairly important component, it is also known as the central gateway module which acts as a communication hub that communicates between different ECUs. The primary function of the Gateway ECU is to manage data traffic across different systems and protocol busses within the vehicle envrionment. The Gateway ECU has the following features:

   (a) Data Routing: It routes messages and data packets between several ECUs making sure that the data is transmitted to the right ECUs. It is capable of handling high data rates and large volumes of data, essential for modern vehicles.

   (b) Protocols support and transfer: It supports different protocols and translates data between different communication protocols like CAN, LIN.

   (c) Security and Diagnostics: It provides cybersecurity features by enabling/disabling the access to vehicle networks and protecting against unauthorized access and cyber attacks. It also acts as a central access point for diagnostic tools to communicate with the various ECUs.

   (d) Redundancy: It also includes redundancy features, which ensures system reliability and continuous operation in case of failure.

Keeping the emphasis on the Secured OTA Updates, the above two mentioned ECUs namely, the TCU, the ECM and the Gateway ECU are the main ECUs that are needed for a successful OTA Update. While other ECUs such as ICU, the Body Control Module (BCM), the Powertrain Control Module (PCM) etc. are also affected by OTA Updates but comparatively they play a less important role in the OTA-process of updates.

1.1.2 Cloud and Communication in OTA Update Execution

After the in detailed discussion about the In-Vehicle Modules involved in OTA Updates, we now delve into the external environment with which the vehicle communicates. The Cloud platform is an important intermediary service, that enables the update service, by storing and distributing updates to the vehicles securely and remotely. The cloud is usually hosted on an OEM server and is controlled by a backend device/infrastructure controlled by an OEM manager. The following are the features of this cloud platform:

- It acts as the central repository for Update packages, software packages, configuration files etc.
- They are a reliable and efficient means of update

For the vehicle to communicate with the cloud platform, the TCU is enabled with different features.

- **Cellular**: They can use mobile networks to update, giving it a broad coverage.
- **Wi-Fi**: They can connect to any Wi-Fi networks available.
- **Satellite Connection**: Some of them also can connect to these networks in areas with no other wireless connectivity.

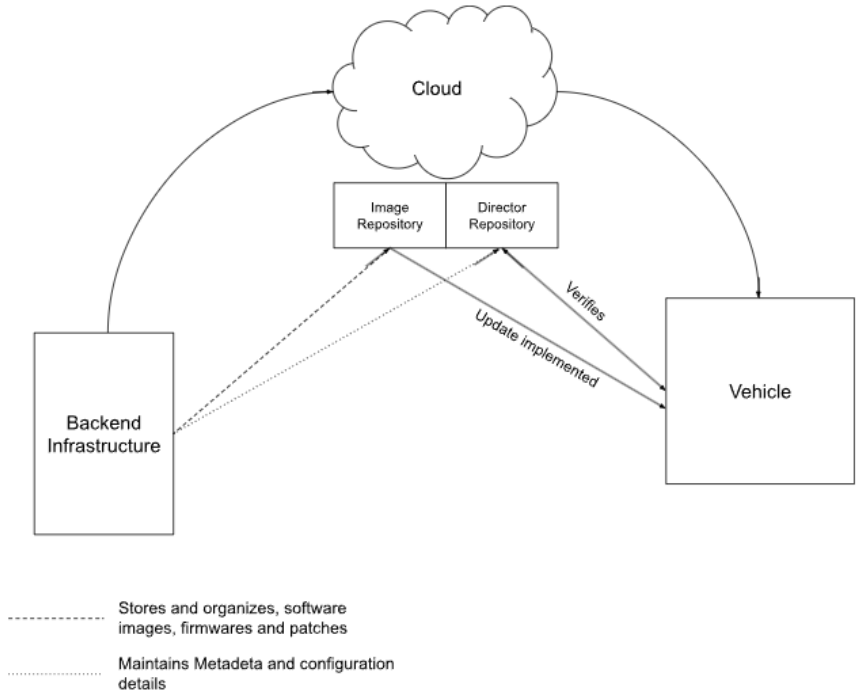


Figure 1.4: Role of cloud in OTA update Architecture

As shown in the figure, a cloud platform usually consists of Image Repository and Director Repository.

- **Image Repository**: The image repository in the cloud platform serves as a centralized storage and management solution for software and firmware updates. It stores various update packages, including firmware images, software images, incremental packages and patches. The repository not only stores these images but also maintains metadata such as version numbers, release notes, and compatibility requirements, which are essential for the Update Management Service to determine the appropriate updates for each vehicle. The repository also employs robust encryption methods to protect data from unauthorized access or tampering. The Image Repository is closely integrated with other backend services, including the Directory Repository and Deployment Orchestration systems, to facilitate seamless update distribution and management to ensure that updates are not only stored securely but are also readily accessible for deployment, enabling a smooth and efficient OTA update process.

- **Director Repository**: The Director Repository in the cloud is a very important component in the infrastructure for OTA update. It organizes and maintains a database of update information, including software versions, firmware patches and other configurational details. This repository tracks various attributes of each update, such as its version number, release notes, dependencies, and the specific vehicle models or configurations it is intended to support. There are 2 other important components that are hosted within the Director Repository:

    1. The Update management Service: It is responsible for the lifecycle of updates. It schedules releases, monitors update statuses, and coordinates with the Image Repository to ensure that the right software versions are available for deployment. This service updates the repository with real-time information on which updates have been applied successfully, which are pending, and any issues encountered during the process.
    2. Deployment Orchestration: It leverages the metadata stored in the Directory Repository to efficiently plan and execute the distribution of updates. It manages the timing and sequencing of update rollouts, ensuring that updates are delivered systematically and according to pre-defined schedules or criteria. This orchestration is crucial for minimizing disruptions and ensuring a smooth update process across a large fleet of vehicles.

1.1.3 Backend Infrastructure

Backend servers play a crucial role in OTA updates for the automotive industry, ensuring that vehicles receive timely and secure software upgrades. These servers manage the distribution of updates by storing, encrypting, and transmitting the software packages to connected vehicles. They authenticate the update requests, verify the integrity of the software, and ensure that the correct version is delivered to each vehicle based on its specific configuration and needs. Backend servers also monitor the update process, providing real-time feedback and analytics to the automotive manufacturer about the status and success rates of the updates. This robust infrastructure is essential for maintaining the safety, performance, and user experience of modern vehicles, allowing manufacturers to roll out new features, security patches, and performance improvements efficiently and reliably. Backend servers manage various aspects of the OTA process:

- Scheduling: Determines the optimal time to deploy updates to minimize disruption.
- Version Control: Keeps track of different software versions and ensures compatibility.
- Data Analytics: Collects and analyzes data on update success rates and any issues encountered.

The importance of backend servers in OTA updates cannot be overstated:

- Efficiency: Enables manufacturers to roll out updates quickly and to a large fleet of vehicles.

- Cost-Effective: Reduces the need for visits to service centers.
- Customer Satisfaction: Keeps vehicles up-to-date with the latest features and improvements.

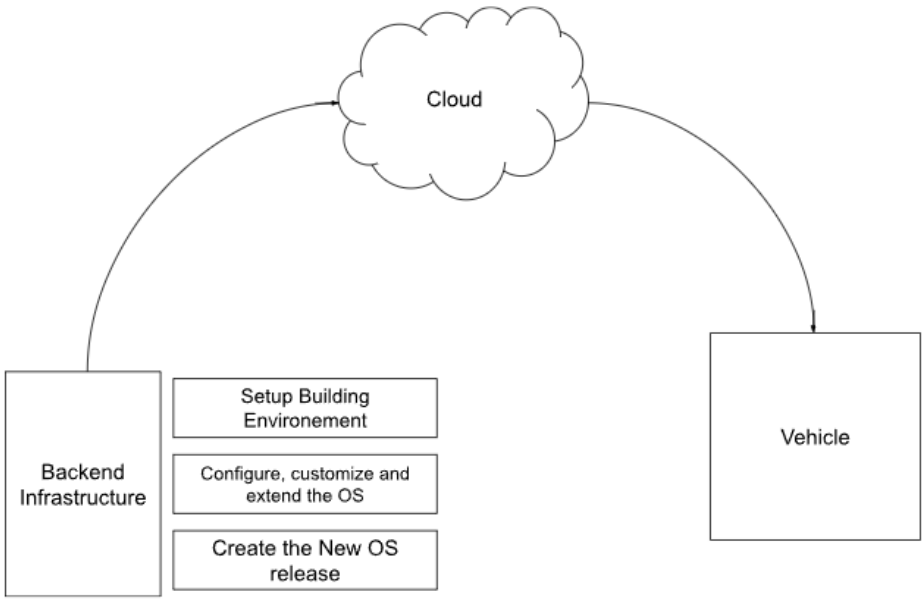


Figure 1.5: Role of Backend in OTA update Architecture

1.2 Understanding Rollback Attacks

If a system or data packages are corrupted during the transmission or reception of the OTA updates, the entire system can be compromised, as the data packets can affect several target systems within an automotive vehicle. To prevent this from happening, cybersecurity protocols are also implemented in the OTA update architecture. Several different types of attacks can compromise the automotive system. Some of the main attacks are:

- Deny Attacks: Deny attacks, commonly referred to as Denial-of-Service (DoS) attacks, are designed to prevent legitimate users from accessing a service or system. These attacks can target the update server, the communication channel, or the device itself, effectively blocking the device from receiving updates.
- Deny Functionality Attacks: Deny functionality attacks focus on disrupting specific functionalities of a device or system without necessarily taking it offline completely. An attacker might allow the update process to initiate but corrupt the update, leading to partial installation, functionality degradation, or making the device unresponsive.
- Rollback Attacks: A version rollback attack involves forcing a device to revert to an older, less secure version of its firmware or software. However, the Version Rollback attack isn't damaging to the system directly, but it can leave the system prone to

known vulnerabilities and can lead to high level system compromise, deletion of new features, or, incompatibility of hardware with older versions, causing breakdowns in the vehicle

1.2.1 Rollback Attack Execution

Rollback attacks, also known as downgrade attacks, are a form of cyber attack where an attacker forces a device to revert to a previous version of its firmware or software, which is typically less secure.

1. The Attacker first gathers information about the target device and its update process. This includes, information about update architectures, verification techniques and installation procedure.

2. The attackers can position themselves between the device and the update servers by exploiting network/Wi-Fi vulnerabilities, DNS spoofing or ARP poisoning.

3. The attacker monitors the communication between the device and the update server to understand the update protocol and capture update files.

4. The attacker can then manipulate the update response by modifying the metadata like manifest files or version control files that the device uses to determine the version of the update.

5. The attacker can also replay an old update file captured during a previous legitimate update process, deceiving the device to accept and install it.

6. If the device's update mechanism does't securely check version numbers or lacks cryptographic verification, the attacker can simply provide an older version as if it were new

7. The attacker may modify the version number in the old firmware to make it appear newer than the currently installed version, thus bypassing basic version checks.

8. The attacker might steal or forge the signing certificate used by the update server to sign firmware updates. If the device does not use secure methods to verify the certificate's authenticity, it may accept the malicious update.

1.2.2 Consequences on the System due to Rollback attack

A version rollback attack involves downgrading a system to an older, less secure version to exploit known vulnerabilities. While the attack itself doesn't cause much damage to the system, it can increase the attack surface of any device and can have severe consequences.

1. Exploitation of Known Vulnerabilities: Older versions of software often contain security vulnerabilities that have been patched in later updates. Rolling back to an earlier version exposes the system to these vulnerabilities. Leading to Data Breaches, System Compromise and Service Disruption.

2. Loss of Security Features: Modern software updates often include new security features designed to protect against emerging threats. A rollback can remove these protections.

3. Data Integrity and Authenticity Issues: Rollback attacks can undermine the integrity and authenticity of data leading to Data corruption and/or Deterioration of security..

4. Malware and Ransomware Infections: Older software versions are more susceptible to malware infections due to outdated security mechanisms.

5. Decreased Performance and Stability: Older software versions might not be optimized for current hardware or workloads, leading to performance issues.

6. Regulatory and Compliance Issues: Organizations are often required to comply with regulatory standards that mandate the use of up-to-date security measures.

7. Financial Loss: The direct and indirect costs associated with a version rollback attack can be substantial eg. Incident Response Costs, Operational Downtime, Reputational Damage etc.

1.2.3 Importance of mitigation

Due to the aforementioned consequences, the risk of version rollback attacks in the OTA update must be mitigated. The importance of mitigation can be highlighted for the following reasons:

1. Damage can be minimized by applying mitigation strategies and protocols, which prevent extensive data loss, system downtime, and financial losses.

2. Mitigation methods such as encryption and authentication ensure that sensitive data remains safe from tampering or unauthorized access, in compliance with regulations like GDPR, HIPAA, and PCI DSS.

3. The system can be kept running efficiently without unnecessary disruptions caused by the presence of an attacker or malware. Automated mitigation techniques secure and protect the system while regular tasks are performed.

4. The reputation of the company can be improved by strong mitigation and cybersecurity techniques, helping to grow customer trust.

1.3 Methodology

The methodology for this master's thesis encompasses a comprehensive approach integrating both theoretical analysis and practical experimentation. Initially, a thorough literature review was conducted to establish a foundational understanding of the current advancements and challenges in automotive OTA updates and the associated cybersecurity issues in this process.

The research and analysis have been divided into three phases to maintain a streamlined and straightforward methodology:

1. Automotive Architecture, OTA Updates, Prototype Development: Current automotive architectures are evaluated, and prototypes for OTA updates are developed in this phase.

2. OTA Update Implementation Reference Architectures: Reference architectures are defined based on the analysis conducted in the previous phase in this phase.

3. Implementation of cybersecurity and Anti-Rollback Mechanisms: The defined system architectures are implemented in this phase, focusing on cybersecurity and mechanisms to prevent rollback attacks.

Following the analysis of the research conducted in each phase, a system architecture is defined based on concrete evidence and comparisons. This includes specifying components, system characteristics, and protocols to be used in the prototype. The implementation phase addresses real-time errors, and solutions for anti-rollback mechanisms are proposed. Finally, the proposed algorithms are evaluated against various attack paths, demonstrating their effectiveness and potential utility.

Goals for the thesis:

- Analysis of State-of-the-art solutions along with the problems/issues respectively
- A working prototype
- A Novel solution for prevention of Rollback attacks in order to replace existing methodologies or significant improvement in existing methodologies.
- Evaluation and Validation of prototype.
- Future Possible contribution possibilities based on the Master Thesis

1.4 Work Flow

Figure 1.6 presents the work flow of the Master Thesis: Detection and Mitigation of Rollback Attacks in Automotive Over-The-Air Updates. First and foremost a background study is done on the topics of automotive architectures, OTA updates, security in OTA updates, in-vehicle architecture, cloud and , backend infrastructure and rollback attacks parallely. On the basis of this understanding, a literature review is done on industry standards, cybersecurity protocols and recent version rollback attacks. Considering the recent trends using the literature review, a research and analysis is done in 3 phases. The first phase focuses on current state of the art OTA architectures and security and creating a basis for the development of a prototype for Automotive OTA updates on the basis of phase 1, phase 2 defines the use industry-level OTA update architectures. As a result of the definition of the prototype, in phase 3 cybersecurity techniques are defined. From the analysis and definitions of the above, the prototype is then developed with system design concept, architecture, principles and features. The developed prototype is then implemented on hardware and software, and Threat Analysis and Risk Assessment (TARA) is performed. With the reference to security goals as an outcome of TARA, algorithms have been proposed.

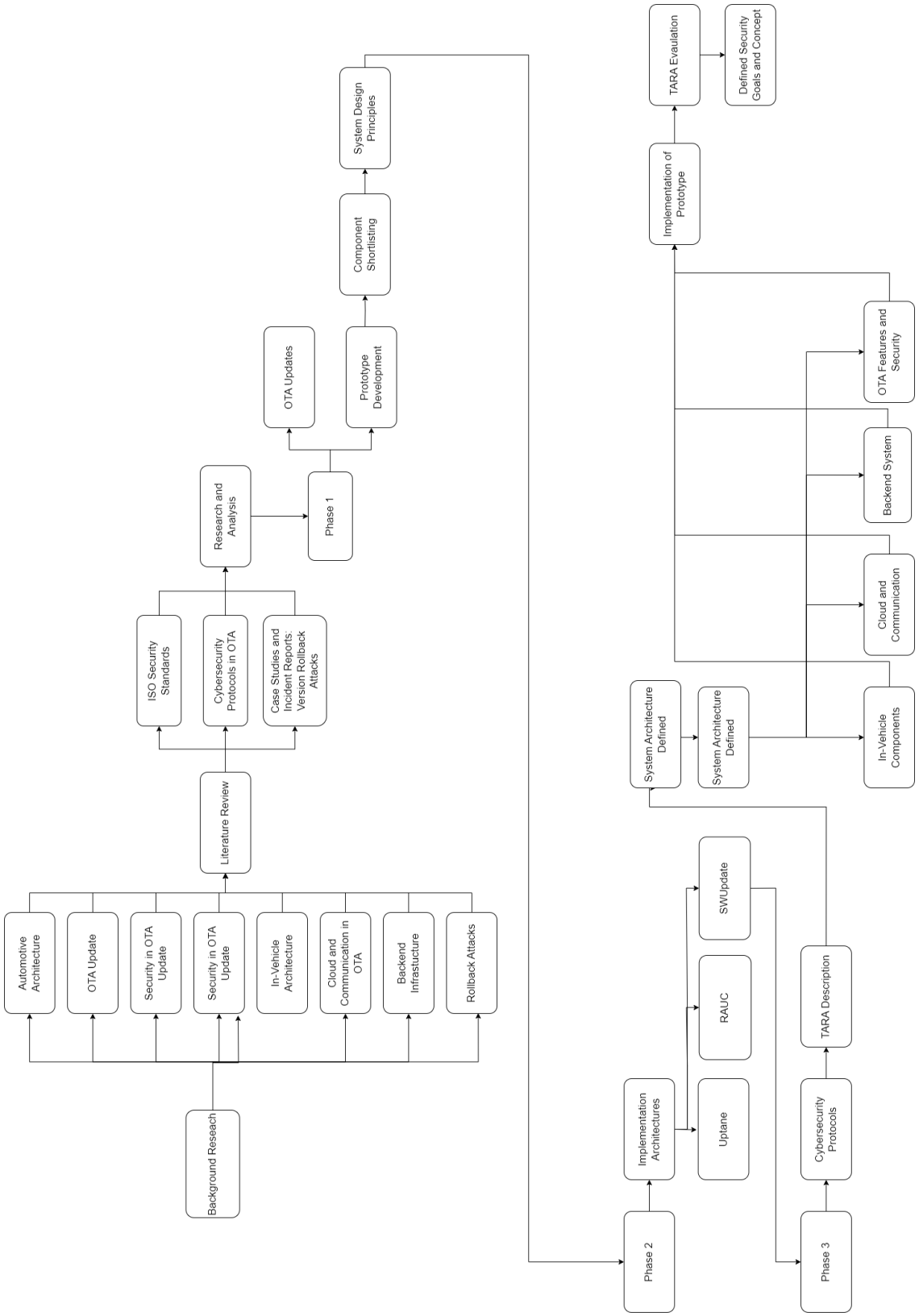


Figure 1.6: Workflow for Master Thesis

Chapter 2

Literature Review

This chapter presents an overview of automotive cybersecurity, with a focus on securing OTA updates, which are considered vital for modern vehicles. Key cybersecurity protocols, such as encryption and Public Key Infrastructure (PKI) are examined, and their roles in protecting OTA processes are assessed. Real-world vulnerabilities, including rollback attacks, are highlighted, setting the stage for the exploration of advanced security solutions.

2.1 Overview of Automotive Cybersecurity

Automotive cybersecurity is concerned with the protection of vehicular electronic systems, communication networks, and data from cyber threats, which include malicious attacks and unauthorized access. Modern vehicles, which may contain 40-80 ECUs (and up to 150 in luxury models) [2] [3] [4], are increasingly interconnected, enhancing functionality but also creating vulnerabilities. A single compromised component can jeopardize the entire system, posing significant risks to user safety, security, and privacy.

From 2016 to 2019, automotive cybersecurity incidents were reported to have surged by 605%, with a notable spike in 2019 alone [5]. Upstream.auto [6] documented 633 publicly reported cyber incidents in the automotive sector since 2010, with 207 occurring in 2020. These statistics underscore the urgent need for robust cybersecurity measures.

A critical challenge in this field is the defence against rollback attacks in OTA updates, which are regarded as a key method for delivering software updates and patches. Rollback attacks are characterized by the reversion of a system to an older, vulnerable software version by attackers. Various security challenges in Automotive OTA Update Architectures are highlighted in the research, including unauthorized access, data integrity issues, authentication problems, confidentiality concerns, tampering, spoofing, and Man-in-the-Middle attacks. Solutions that have been proposed include the use of hash functions, digital signatures, cryptography techniques, secure bootloaders, and Intrusion Detection Systems (IDS).

2.2 Existing solutions for Secure OTA Updates

The challenges of ensuring safety and security during OTA updates are discussed in some chapters [7] which are critical as vehicles become increasingly software-dependent. Assurance

case templates that comply with ISO 26262 for functional safety and SAE J3061 for cybersecurity have been proposed. In automotive cybersecurity, several standards are regarded as crucial for user safety and security. These regulatory compliance standards are utilized to assess and mitigate risks in vehicle design, making adherence essential for OEMs.

1. ISO 26262 [8]: Focuses on functional safety, ensuring that electronic and electrical systems operate reliably and safely, reducing the risk of failures leading to accidents.

2. SAE J3061 [9]: Provides guidelines to help manufacturers incorporate cybersecurity into automotive systems from the outset

3. ISO 21434 [10]: Establishes a framework for managing cybersecurity risks throughout the vehicle's lifecycle, protecting against cyber threats.

To complement these standards, certain cybersecurity protocols and algorithms, mostly corresponding to ISO 21434, are applied to ensure comprehensive protection.

1. Encryption: Encryption is regarded as vital for securing OTA updates in connected vehicles, ensuring the confidentiality, integrity, and authenticity of software packages. Key techniques are utilized, including symmetric key encryption for secure connections, hash functions for integrity verification, and blockchain for decentralized, tamper-resistant update management. These methods are employed to protect against unauthorized access, tampering, and man-in-the-middle attacks, ensuring the safety of vehicle software and its occupants [11].

2. Hashing: Hashing is recognized as a key cryptographic technique used in OTA software updates for connected vehicles to ensure the integrity and authenticity of software. A secure OTA firmware update protocol utilizing a hash chain to verify each data fragment's integrity has been proposed in research [12] adding an extra security layer by encrypting the hash chain fragments with a pre-shared key. Another study [13] presents AiroDiag, which employs hashing and AES encryption to secure OTA updates, involving entities such as the OEM and the car.

3. PKI: PKI is characterized as a framework that ensures authenticity and security by utilizing various cybersecurity techniques [14].

    (a) Key Creation: Keys, both private and public, are generated by the PKI. The private keys are kept confidential and used to sign updates, while the public key is distributed for signature verification.

    (b) Digital Signing: When an OTA update is received by a vehicle, it is cryptographically signed by the OEM with a private key, and the signature is verified by the vehicle using the corresponding public key. This process confirms the legitimacy of the update and protects against cyber threats such as man-in-the-middle attacks. Digital signatures are considered essential for maintaining safety in vehicles and complying with automotive security standards [15].

    (c) Signature Verification: Once the OTA update is transmitted to the vehicle, public keys are used by the TCU to verify the update.

(d) Security and Installation: Security during the communication process is ensured by PKIs, which are also responsible for the installation and its verification.

4. Transport Layer Security (TLS): TLS is identified as a cryptographic protocol designed to provide secure communication over a network. It is ensured that the data transmitted between the vehicle and the OTA update server remains confidential, authenticated, and protected from tampering. First, a secure network is established using a network handshake, followed by the exchange of keys, which include those used for encrypting and decrypting data. The confidentiality, integrity, and authentication of the update are ensured through this process.

5. Hardware Security Modules (HSM): Physical and digital protection for cryptographic keys is provided by HSMs, ensuring that sensitive data and operations are securely managed and stored.

2.3 Analysis of Cybersecurity Protocols in OTA Update Automotive Systems

As automotive systems become increasingly reliant on software updates delivered via OTA mechanisms, understanding and mitigating cybersecurity protocols in these systems is considered crucial. Existing literature on cybersecurity protocols specific to OTA updates in automotive environments is reviewed in this section, exploring the nature of these protocols, their target components, and the protection against different types of attacks.

	Encryption	Hashing	Public Key Infrastructure	Transport Layer Security	Hardware Security Modules
Cyber Security					
Confidentiality	Y	N	Y	through encryption	encryption
Integrity	N	Y	Y	# and MAC	secure key mgmt
Availability	Y	N	Y	N	Y
Protection for					
ECU	Y	Y	Use trusted CAs	Network protocol	Storing Keys
TCU	Y	Y	Use trusted CAs	N	Storing keys
CAN Bus	N	N	N	N	N
Keys for ECU and TCU	N	N	N	N	Storing
Wi-Fi - TCU (Client)	Y	Y	Y	Y	N
Wi-Fi Protocol HTTP, HTTPs, MQTT etc.	Y	Y	Y	Y	N
Cloud Server	Y	Y	Y	Y	Y
Backend OEM Client	Y	Y	Y	Y	Y
Protection Against					
Spoofing	Y	Y	using CA	using CA	Y
Tampering	Y	Y	Signatures	MAC and #	Y
Repudiation	N	with Digital Signatures	N	N	Y
Information Disclosure	Y	N	Key pairs	Y	Y
Denial of Service	N	N	N	N	N
Elevation of Privilege	Y	N	Authentication Mechanism	Y	Y

Table 2.1: Comparison of Cyber Security Techniques

Table 2.1: Comparison of Cyber Security Techniques, presents a comparison of different cybersecurity protocols used in modern systems, such as encryption and hashing, along with their cybersecurity properties, the attacks against which they provide protection, and the components in which they are implemented within a vehicle architecture.

2.4 Case Studies and Incident Reports: Version Rollback Attacks

Attack	Discovered	Mechanism	Impact
FREAK (Factoring RSA Export Keys) [16]	2015	Forces a connection to use weaker "export-grade" cryptography (RSA with 512-bit keys).	Allows decryption of intercepted communications and possible impersonation of a secure server.
Downgrade Attacks on HTTPS (e.g., SSL Strip) [17]	2009	Tools like SSL Strip downgrade HTTPS connections to HTTP, removing encryption.	Enables interception and modification of transmitted data.
BEAST (Browser Exploit Against SSL/TLS) [18]	2011	Targets vulnerabilities in SSL/TLS handling of block ciphers using a chosen plaintext attack.	Decrypts portions of encrypted traffic by exploiting predictable initialization vectors in older TLS versions.
Sweet32 [19]	2016	Targets older ciphers like 3DES using the birthday paradox to decrypt data from large captured traffic.	Allows recovery of plaintext from a TLS session by observing many encrypted messages.
KRACK [20]	2017	Exploits a flaw in the WPA2 protocol used for securing Wi-Fi networks.	Enables an attacker within range to read information previously assumed encrypted.
Spectre [21] and Meltdown [22]	2018	Exploits vulnerabilities in processors to read memory contents from other processes.	Exposes sensitive data like passwords and encryption keys.
BlueKeep [23]	2019	A critical vulnerability in Microsoft's RDP allowing for remote code execution.	Could spread malware across networks, similar to WannaCry.
Ripple20 [24]	2020	Multiple vulnerabilities in the Treck TCP/IP stack affecting millions of IoT devices.	Allows remote code execution, data theft, and device takeover across sectors.

Table 2.2: Recently Implemented or Discovered Version Rollback Attacks

In the context of rollback attacks, various rollback mechanisms can be implemented depending on the type of security possessed by a system, the specific goals of the attackers, or the simplicity of the attack, which may depend on different vulnerabilities.

Table 2.2: Recently Implemented or Discovered Version Rollback Attacks summarizes recent version rollback attacks, detailing their mechanisms and impacts. It highlights notable threats like FREAK and KRACK, emphasizing vulnerabilities in outdated cryptographic methods and system protocols. This overview underscores the need for robust security measures in automotive systems, particularly for OTA updates, to mitigate cyber risks and enhance overall vehicle cyber-security.

In the upcoming chapter, a detailed exploration of the comparative analysis of OTA updates, automotive architectures, components for the prototype, implementation architecture, and more will be conducted.

Chapter 3

Research and Analysis

This chapter presents background research on various topics that contribute to the overall goal of detecting and mitigating version rollback attacks. These topics are explored through well-documented studies that provide the foundation for the development and proposal of a prototype. The research and analysis are structured into three phases, each with a distinct research design and data collection approach. The findings from this research guide the implementation of relevant techniques in subsequent sections.

The research phases, as outlined in Figure 3.1: Phases of the Methodology, each follow their own methodologies for data collection and analysis. The results from these phases lead to conclusive insights, which in turn inform the implementation strategies discussed in the later sections of the chapter.

Data Collection Methods: All data presented in this chapter have been collected from sources such as Google Scholar, ResearchGate, IEEExplore, ACM Conferences, and Science Direct. The information related to Table 3.2: Device Simulation Options and Considerations (appended at the end of chapter 3), which outlines the various components for the development of the prototype, has also been sourced from the cited research studies. Additionally, other sources, including OEM websites, have been utilized, such as those of Infineon Technologies, Continental, Audi, and BMW.

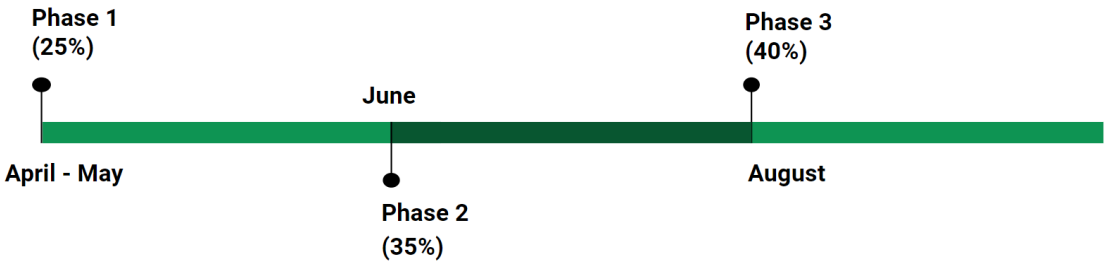


Figure 3.1: Phases of the Methodology

3.1 Phase 1: Automotive Architecture, OTA Updates, Prototype Development

This initial phase involves basic research on automotive cybersecurity, vehicle architectures, and OTA updates. The main focus of this phase was to gather information on different types of cybersecurity protocols, components involved in OTA update architectures, and the development of a vehicle environment prototype.

3.1.1 Research Design

An in-depth examination of automotive cybersecurity, vehicle architectures, and OTA update mechanisms is conducted in this section. The research undertaken to identify and shortlist controllers for use as ECM and TCU is summarized in Tables 3.2: Device Simulation Options and Considerations and 3.3: Backend Simulation Options and Considerations (appended at the end of chapter 3). The various components considered, along with the final selections, are detailed in the tables. This environment is intended to serve as a prototype for testing OTA updates and cybersecurity features.

3.1.2 Analysis

Based on the data that has been collected and understood, the following methods are considered to be the best possible approaches for the development of the prototype:

1. A working vehicle environment with OTA update functionality can be simulated using the TCU and ECM that communicate via the CAN bus. Since this is a prototype, the Gateway ECU is not required due to the low volume of in-vehicle communication packages, which eliminates the need for data routing. For the limited scope of the prototype, only the CAN bus is utilized, making the use of buses like LIN or FlexRay and protocol transfer and support unnecessary.

2. General platforms are utilized for prototyping, as programming tailored controllers is more challenging. For the prototyping process, general platform like Raspberry Pis are preferred, enabling later customization to meet the needs of various service providers and other OEMs.

3. The prototype is comprised of one ECM that produces mock sensor data like fuel injection, revolutions per minute (RPM) etc. and one TCU that is connected to the ECM via a CAN bus. Mock data such as latitude, longitude, speed, etc., is generated by the TCU. OTA update functionality is implemented in the TCU, with Wi-Fi being utilized for this purpose.

3.1.3 Designing of Prototype

Based on the above research and analysis, the following conclusions have been reached regarding the appropriate system for the prototype:

1. A Raspberry Pi 3B+ is designated to be used as the ECU, and another Raspberry Pi 3B+ is designated to be used as the TCU.
2. A CAN HAT Shield is to be mounted on the Raspberry Pi to enable the CAN protocol.
3. To obtain mock sensor values, sensors can either be connected to the ECU and TCU boards or simulated values can be used.
4. Wi-Fi capabilities are to be enabled in the TCU to facilitate OTA update functionality.

System Design Principles

1. The ECU and TCU are to be designed to act like a real vehicle environment.
2. Values such as revolutions per minute (RPM), fuel injection, fuel percentage, volume control (for the Infotainment ECU), and a refuel button will be generated by the ECU.
3. The TCU will be equipped with Wi-Fi to connect to the server for OTA update functionality.
4. The system is to be connected across a single CAN bus.
5. Cryptography and cybersecurity measures will be supported by the TCU and ECU.
6. Since none of the mentioned available VM were deemed suitable, a backend Linux machine was set up at the company office, accessible via remote connection.

3.2 Phase 2: OTA Update Implementation Reference Architectures

In this phase, the focus shifts to the examination of OTA Update Implementation Reference Architectures, emphasizing the evaluation of their effectiveness for the prototype system. Multiple architectures are analyzed, with three—Uptane, RAUC and SWUpdate—taken into consideration based on system requirements. A detailed comparison of these architectures is then conducted to determine their suitability for enhancing the security and efficiency of the OTA update process in the context of the thesis objectives.

3.2.1 Research Design

The research design for this section focuses on OTA Implementation Techniques and Architectures, specifically analyzing three selected OTA Update Implementation Reference Architectures. The objective is to evaluate their features, characteristics, and the flow of update files to determine their suitability for the prototype system. The reference architectures considered are Uptane, Robust Auto-Update Controller (RAUC), and SWUpdate.

3.2.2 Analysis

Based on the data from Table 3.4: Summary of Software Update Architectures and 3.5: Reference Architectures Used in the Automotive Industry (appended at the end of chapter 3), key points were derived from the three mentioned OTA Update Implementation Reference Architectures that are appropriate for this application. A detailed description, along with the features, characteristics, and flow of update files for each of the three selected Reference Architectures, is provided in the following section.

Uptane

Uptane is a security standard specifically designed for automotive OTA updates. Strong protections against cyber-attacks targeting the software update process are provided, ensuring that updates to a vehicle's ECUs are authenticated and trusted. Uptane is characterized as an open, flexible system that can be adapted to different automotive architectures and scales, making it widely applicable in the automotive industry to prevent vulnerabilities during software updates.

1. The software update process is separated into multiple roles (Director repository, Image repository), ensuring no single point of failure in the update chain.
2. Risks from rollback, replay, and mix-and-match attacks are mitigated by using metadata and version numbers to validate updates.
3. All software and metadata updates are signed with cryptographic keys, ensuring authenticity and integrity during transmission and installation.
4. Even if one repository is compromised, Uptane ensures that malicious updates cannot be installed on the vehicle.
5. Adaptability to various vehicle architectures is supported, facilitating both partial and full ECU updates, and it works alongside existing security mechanisms.
6. Alignment with emerging standards and regulations in automotive cybersecurity, such as those from the UNECE, is maintained.
7. Mechanisms for safely recovering from compromised or failed updates are included, reducing the risk of bricked ECUs.

According to Update - Best Practices [35], Uptane is not characterized as a framework or an architecture but rather as a standard that aims to provide best practices for Reference Architecture Implementation in Automotive OTA architectures. Therefore, a framework or a defined workflow is not provided by Uptane; instead, standard practices are outlined that should be applied to the system to ensure security.

RAUC

RAUC is characterized as a lightweight and flexible framework designed for implementing secure and reliable software updates on embedded Linux devices. It is intended to handle complex OTA updates for devices, firmware, bootloaders, or other software components embedded in the device. The following features are included:

1. Robustness: RAUC is designed as a redundant system, ensuring that recovery from interrupted updates or errors can be achieved.

2. Security: Cryptographic signing of update bundles is supported, ensuring that the authenticity and integrity of updates are maintained through verification and SHA hashing.

3. Flexibility and Modularity: RAUC is highly configurable and can be tailored to the application, with integration support for different build systems being provided.

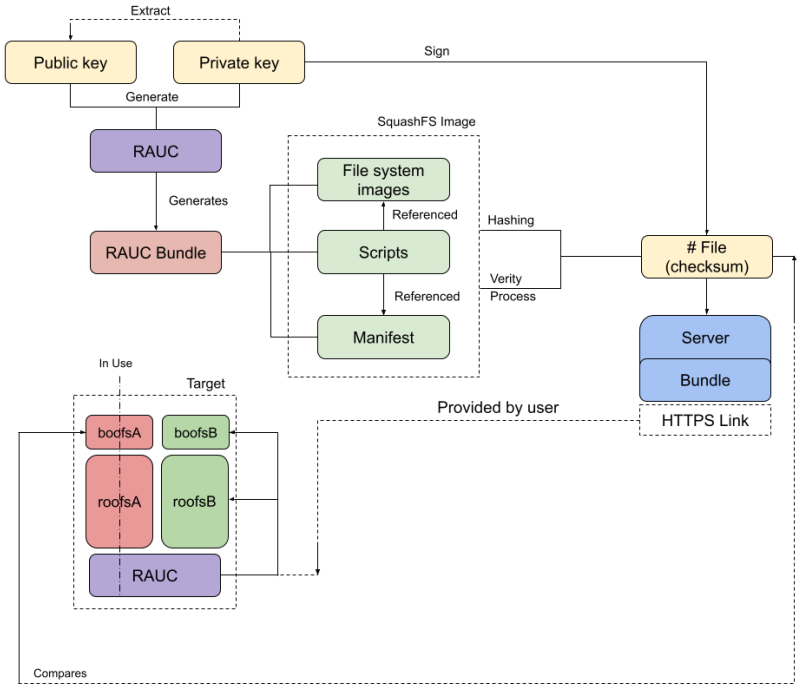


Figure 3.2: RAUC Architecture

The workflow of the OTA update is explained in Figure 3.2: RAUC Architecture. The bundle is created by the RAUC application that includes file system images, a manifest (explaining the different configurations of the update process), and any scripts referenced by the other two. The compilation of these files is converted to a SquashFS Image, hashed, and verified alongside the process. The hash file with the checksum is signed using a private key, uploaded to the server, and the update can be downloaded to the target device using the HTTPS link, which has two partitions. Before the update is downloaded to the target

device, the checksum is compared to the existing partitions by the RAUC software layer. If the checksum is the same or of a lower version, the update process is stopped. If the checksum of a higher version is recognized by the system, the update is downloaded and installed in the unmounted partition.

SWUpdate

SWUpdate is characterized as a robust and flexible software update framework designed primarily for embedded systems. An easy solution for updating the firmware, software, or configuration files of embedded devices in a reliable manner is provided by SWUpdate. It has the following features:

1. Multi-Format Support: Various formats such as images, packages, scripts, etc., are supported by SWUpdate.
2. Multiple Update Strategies: Multiple update strategies, including Single Image Update, Dual Bank Update, and Modular Update, are supported by SWUpdate.
3. Compatibility with Bootloaders: Compatibility with different bootloaders, such as U-Boot, is provided by SWUpdate.
4. Security Features: Security features such as authentication, encryption, and digital signatures are offered by SWUpdate.
5. Customization: SWUpdate is highly customizable, allowing for the customization of update logic, custom handlers, or integration with other components using APIs.
6. Redundancy: Redundant features through A/B type update options are offered, along with fail-safe mechanisms. Logs and reports on the update process are also maintained.

The workings of the SWUpdate OTA update flow are explained in Figure 3.3: SWUpdate Architecture. Initially, a package with the extension .swu is created using the host, which can be done manually or automated via mkfs.swu. This .swu package must contain the sw-description file, which includes all the information regarding the update files, their locations, and the update conditions. Additionally, it must include the firmware update files, any application executables, and configurations required for the OTA update. Once the .swu package is prepared, cybersecurity techniques may or may not be applied before the update is pushed to the HTTP/HTTPS server. The update can then be downloaded from the server by the target device, and the integration of the update with the system is performed by the swupdate software layer, which then reboots the system.

Based on the information regarding the three different Reference Architectures, it can be concluded that SWUpdate is suitable for the application in terms of flexibility and support. As specified in the goal of the thesis, the existing OTA Architecture will be optimized and made more secure. In this context, SWUpdate will be implemented in the ECU, TCU, and a host Linux machine, which will be capable of setting up cloud services and other cybersecurity measures as prerequisites for SWUpdate.

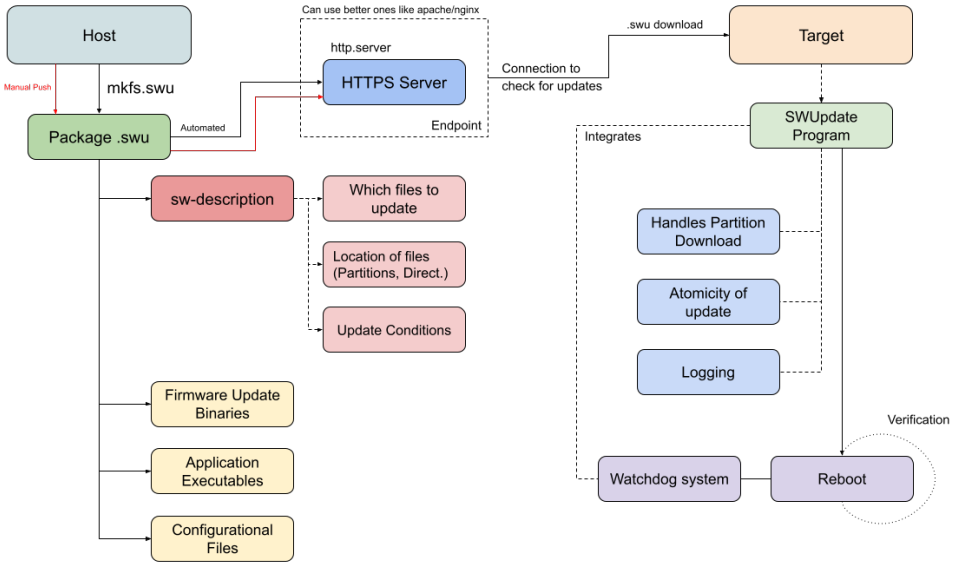


Figure 3.3: SWUpdate Architecture

3.3 Phase 3: Implementation of cyber-security and anti-rollback mechanisms

After the first two phases, the final phase is focused on the importance of cybersecurity in the system, Version Rollback attacks, and the design of an optimal OTA Update Architecture. Since the goal of the thesis is to design an Architecture with an optimal OTA update structure, Table 3.8: OTA Optimization (appended at the end of chapter 3) presents various research studies that aim to enhance the OTA Update Architecture based on several different factors.

3.3.1 Research Design

As mentioned above, the research is done on the different topics in this phase 3.

First and foremost, a secure web server must be chosen, as shown in Table 3.7: Comparison of NGINX, Apache, and Built-in Authentication (appended at the end of chapter 3). Since SWUpdate does not include any authentication or protection mechanisms for the host server, it can be accessed by any device within the network. Therefore, one of the available options is selected based on the conclusions drawn from Table 3.7, and this will be used to handle HTTP/HTTPS requests, reverse proxying, and load balancing.

As recommended in the SWUpdate documentation, Hawkbit is suggested as the update management platform for managing and delivering software updates to connected devices.

Hawkbit's primary role is to manage the lifecycle of updates and ensure the smooth delivery of updates to client devices, such as embedded systems or automotive ECUs. Although Hawkbit is designed to handle updates independently, it can be deployed behind a web server like NGINX to enhance scalability, security, and performance. In this configuration, NGINX functions as a reverse proxy, with incoming requests forwarded to the Hawkbit server while tasks such as SSL termination, caching, and load balancing are managed. Through this integration, the security of the update delivery system is reinforced, and its performance is optimized, as NGINX effectively manages web traffic.

Now, as per our application, NGINX is deemed appropriate for use due to its ease of use, scalability, and low resource consumption, making it suitable for deployment in our prototype model. It is also utilized at the industry level by OEMs like Audi [36], where it is categorized as future-proof.

As part of the detection of Version Rollback attacks, an intrusion detection algorithm or system is needed that can recognize any unauthorized or irregular activity. The Version Rollback attack may or may not be mitigated but if they are detected timely and blocked, they will not do much damage to a system, but one of the main problems in the Automotive systems is the late detection of such an attack, which can do extensive damages on the system, user and the OEM as well. There are several different Automotive IDS used in the industry sown and compared in Table 3.9: IDS used in Automotive Industry (appended at the end of chapter 3).

3.3.2 Analysis

Prototype

From the above research on the various aspects of system development in Automotive OTA security, the following analysis has been derived.

1. To monitor and protect OTA update packages targeted at a system, the deployment of a secure server is necessary to safeguard server communication and packages. According to Table 3.7: Comparison of NGINX, Apache, and Built-in Authentication, NGINX has been selected as the main web server for implementation. NGINX should be utilized as a reverse proxy, with Hawkbit implemented on the NGINX server as the update management system.

2. To detect Version Rollback Attacks, an IDS is needed. Due to the limitations of the prototype's hardware, the implementation of an industry-level IDS is not possible; however, based on the features listed in Table 3.9: IDS used in the Automotive Industry, Suricata should be considered an ideal choice. After a detailed analysis of both systems presented in Table 3.10: Comparison between Snort and Suricata, the selection has been made (appended at the end of chapter 3).

3. Basic cryptographic techniques are provided by SWUpdate/RAUC, which can be utilized and implemented to enhance the system's readiness in comparison with industry standards. Signing, encryption, and hashing are offered as base features by these frameworks.

Along with the implementation of the cybersecurity protocols, the prototype needs to be analyzed in reference to ISO 21434. A Threat Analysis and Risk Assessment should be performed to identify possible vulnerabilities in the system architecture, followed by the provision of new solutions in comparison to the state-of-the-art solutions currently used in the industry.

TARA: Threat Analysis and Risk Assessment

Methods to determine the extent to which a road user can be impacted by a threat scenario are described in Clause 15 of ISO 21434. These methods and their work products are collectively known as TARA and are performed from the viewpoint of affected road users. The methods defined in this clause are generic modules that can be systematically invoked from any point in the lifecycle of an item or component.

1. Asset Identification
2. Threat Analysis and Risk Assessment
3. Impact Rating
4. Attack Path Analysis
5. Attack feasibility rating
6. Risk value determination
7. Risk treatment decision

Analysis of the Prototype

Based on Clause 15 of ISO 21434, an analysis of the defined prototype should be conducted to identify vulnerabilities, followed by a comparison to state-of-the-art techniques. As a result, significant improvements and/or a new algorithm for the cybersecurity of the OTA update architecture would be proposed in the paper.

Table 3.1: Overview of existing researches on Automotive OTA updates and its cyber-security properties

Research Automotive CyberSec								
Paper		AutoSec [25]	Data Security in Vehicle Wireless Firmware Update Process [26]	Lightweight FPGA-based IDS-ECU Architecture [27]	QRNG ECU for automotive security [28]	Systematic threat and assessment security testing of automotive OTA updates [29]	A Hierarchical Security Framework for Defending Against Sophisticated Attacks on Wireless Sensor Networks in Smart Cities [30]	Security design of OTA upgrade for intelligent connected vehicle [14]
Simulation								
Prototype	Controller ECU		AURIX TC275-TRB	Deep-learning processing unit (DPU) IP block from Xilinx	RaspberryPi 3B+	RaspberryPi 3B+		
	Controller TCU		RaspberryPi 3B+		RaspberryPi 3B+	RaspberryPi 3B+		
	Controller Gateway							
Cyber- Security	Technology Used	AutoSec Innovation	Cryptography and Image Steganography (Authentication) Gray Level Co-Occurrence Matrix (GLCM) (Authorization)	Intrusion detection, machine learning models, and FPGA-based architecture	Quantum Random Number Generator, high-quality random numbers as the foundation for encrypted communication systems true random number generators (TRNG) based on quantum technology (QRNG).	Uptane Algorithm with 3 main parts. Image Repository (offline signed). Directory Repository (Signed online), Time server	It employs a two-level detection system, The UCON approach is proposed for control usage management, while Chance Discovery uses an adaptive algorithm to identify unknown attacks. Key Graph Detection targets low-level attacks, and Data Crystallization is used to address and mitigate high-level threats. It used Software-Defined Networking (SDN) and Network Function Virtualization (NFV) to enhance security and safety.	Digital Signature - Hashing - Message Authentication
Threat Assessment		N/A	N/A	N/A	N/A	N/A	The security is based on Sensor Networks controlled by WiFi in Smart cities.	N/A
Attacks		N/A	Different attacks to retrieve the data and hinder the firmware update process	Denial-of-Service, Fuzzing, and Spoofing	Denial of Service (DoS) attacks	Tampering of updates, Mix and Match attack, rollback attack, Endless data attack, DOS, Information disclosure	Dynamic ongoing attacks relating to changing MAC addresses	Repudiation, tampering, information disclosure
CAN Bus		Transmission Authorization (TACAN)		Intrusion Detection System with CAN bus	N/A			
OTA Updates	Type of Update	N/A	Firmware OVer-the-air (FOTA)	N/A	FOTA	FOTA and Software OVer-the-air (SOTA)		
	Cellular							
	WiFi					Wireless Sensor Networks		
	Bluetooth							
OTA Security		N/A		N/A	N/A			
Properties		Message Integrity, User Authentication	Data Integrity and Authentication	Increased efficiency and computing power in budgeted power	N/A	Authentication, integrity, authorization and confidentiality	Unified attack detection strategy and software-defined networking and network function virtualization.	N/A

Research Automotive CyberSec CONTINUED...						
Paper		A Model-Based Security Testing Approach for Automotive Over-The-Air Updates [31]	Research on Cybersecurity Technology and Test Method Of OTA for Intelligent Connected Vehicles [32]	FCSS: Fog-Computing-based Content-Aware Filtering for Security Services in Information-Centric Social Networks [33]	Secure FOTA for Internet of things (IOT) [34]	**Properties**
Simulation			Vehicle Dark Room	NS-3 Based Named Data Networking Simulator		N/A
Prototype	Controller ECU	Raspberry Pi 3B+	N/A			N/A
	Controller TCU	Raspberry Pi 3B+	N/A			N/A
	Controller Gateway		N/A			N/A
Cybersecurity	Technology Used	Attack path execution to tamper FOTA	Authentication, Data Security, Man-Machine Interaction, Access Control, Communication Security, Code Security, Upgrade Security	Classification and filtering, Performance evaluation resources are shifted to the network edge to reduce latency, enhance end-to-end communications, and improve security service filtering	PUSH and PULL OTA methods. FOSE (FoTA Object Signing and Encryption) enhances security by encoding the firmware payload with JOSE. FOSE also supports protocol-independent metadata, hash verification for updates, and resuming interrupted transfers.	N/A
Threat Assessment		Intelligence Gathering, Threat Modelling, Case Test Generation, Automatic Case Test Generation, Case Test Execution	ADTool, a threat modeling and analysis tool, to identify various potential threats to the system from the perspective of an attacker	Intelligence Gathering, Threat Modelling, Risk Evaluation, Penetration Testing (Car Design), Penetration Testing (Simulated Strike), Penetration Testing (Vulnerability Analysis), Penetration Testing (Preliminary Report), Systematic Correction, Reset	End-to-end security: Confidentiality, Integrity, and Availability (CIA) for the user.	N/A
Attacks		Eavesdropping, replay attacks, denial of update installation, rollback attacks, arbitrary software attacks, and compromising Uptane repositories hosting firmware image files and associated metadata	Various attacks targeting Authentication, Data Security, Man-Machine Interaction, Access Control, Communication Security, Code Security, Upgrade Security	Information Disclosure, **DOS!**, Tampering	Elevation of Privilege, Information Disclosure, **DOS!**, Tampering	N/A
CAN Bus					N/A	N/A
OTA Updates	Type of Update	FOTA	FOTA		FOTA	N/A
	Cellular					N/A
	WiFi					N/A
	Bluetooth					N/A
OTA Security		N/A	N/A	FCSS: Fog-Computing-based Content-Aware Filtering for Security Services in Information-Centric Social Networks	FOSE (FoTA Object Signing and Encryption)	N/A
Properties		Focusing on the Model-Based Security Testing (MBST) approach. The Uptane framework and the use of Communication Sequential Processes	N/A	N/A	N/A	N/A

Table 3.2: Device Simulation Options and Considerations

Device (to-be-simulated)	Controller Option	Reason for Use	Reason for not using
ECU	Raspberry Pi (3B+)	It is a general development board, not an overkill or under-featured device. It supports most functions of a real-life ECU, and the functions that are not available in the device can be integrated by attaching different peripherals, or simulating them in a Python-based environment.	
	32-bit AURIX™ TriCore™ Microcontroller series		A dedicated series of ECU boards by Infineon Technologies consists of specialized boards. The primary reasons for not using these boards are as follows: First, the system would be considered overkill, as the focus is not on the functioning of the ECU but on the attacks targeting the ECU during OTA updates. Instead of emulating real-life ECUs, the environment can be simulated on an OS-based system, such as the Raspberry Pi. Additionally, with this particular dedicated board, the integration of sensors would be necessary rather than merely simulating them on the board.
	Engine Control Unit MS 6.1 EVO		Also like the previous board, this is a dedicated unit and it has many communication protocols along with several cores, motorsport and high-performance applications. These may not provide the level of accessibility and customization needed for cybersecurity research and development. It also does not have any cybersecurity integrations.
	Haltech ECUs		Like the above, these are overkill and focused more on the working of the ECU and away from the scope of our development and the cybersecurity of our scope.
	Link Engine Management ECUs		Like the above, these are overkill and focused more on the working of the ECU and away from the scope of our development and the cybersecurity of our scope.
TCU	The Raspberry Pi (3B+) can also be used to simulate the TCU, as this general development board is capable of fulfilling the requirements of the project scope. Since it is not a dedicated board, it can function as the ECU and be programmed accordingly to act as a TCU. Additionally, its functionality and limited hardware can be extended through the integration of hardware and software modifications.		
	Quectel Telematics Control Unit (TCU)		All of these are dedicated boards, and for no other reason than cost compared to Raspberry Pi, they weren't used. Also, like the ECU, it would be an overkill for our scope.
	BOSCH telematics control units		This is a dedicated board for TCU based communications, it surpasses the requirements by a huge number of features, but using this in a thesis scope would be an overkill and it might not even be compatible with components with less processing power.

Table 3.3: Backend Simulation Options and Considerations

Device (to-be-simulated)	**Controller Option**	**Reason for Use**	**Reason for not using**
Virtual Machines	Windows Machine		Since it is my main PC and I like to tinker with the machines I work on, I do not want to damage my data for the sake of tinkering.
	Ubuntu 64 Bit (Specialized for ROS - Kinetic)		This Ubuntu machine is tailored with libraries and packages which are needed to run ROS on the system.
	DEVSAC VM by Cisco Netacad		Hands-on experience in a controlled, virtualized environment. DEVSAC VM helps students and professionals develop and hone their skills in various areas of network security, automation, and configuration.
	Security Work Station		Cisco Networking Academy (NetAcad) incorporates the use of a security workstation to enhance its cybersecurity training programs. A security workstation is typically a virtual or physical machine configured with various security tools and software to provide students with hands-on experience in cybersecurity practices.
	CyberOps Workstation by Cisco		Cisco Networking Academy (NetAcad) uses the CyberOps Workstation to provide a comprehensive and immersive learning experience for students pursuing careers in cybersecurity operations. The CyberOps Workstation is a specialized environment equipped with tools and software that simulate real-world cybersecurity operations tasks.
	Security Onion Workstation		Security Onion is an open-source Linux distribution for intrusion detection, network security monitoring, and log management. It's widely used for both learning and operational purposes in cybersecurity.
Ubuntu Linux	Ubuntu 20.04 / Dual Booted	Provides access to a native Linux environment, making it ideal for running Linux-specific applications including acting like a backend system, integration with open source architectures and the Linux system would work at full potential using the hardware in the PC.	

Type of Architecture	Classification	Explanation	Advantages	Disadvantages
Push Based	Direct Push	Server sends update directly to device for immediate installation or scheduling	Immediate updates, good for urgent fixes	High server load, may disturb user experience
	Push Notification	Server sends a notification; device downloads and installs when convenient	Less disruptive	May delay updates as action is required from the device
Pull-Based	Scheduled Pulls	Device checks for updates at regular intervals	Controlled server load, can schedule during idle times	Delayed deployment, less immediate
	On-Demand Pulls	Device checks for updates based on specific triggers	Updates at convenient times	Requires management of triggers
Hybrid Type	Push Notification with Pull	Server notifies device, which pulls update when ready	Immediate awareness, controlled download timing	Implementation complexity
	Conditional Push	Updates pushed only under certain conditions	Reduces disruption, efficient	May delay updates if conditions are infrequent
Delta Update Architecture	Differential Update	Transmits only differences between current and new versions	Low bandwidth, faster updates	High complexity in front and back-end
Dual Bank (A/B) Update	-	Allows rollback, no user disruption, secure with digital signatures	Safe, reliable	Requires more storage, complex management
Staged Update Architecture	Phased Rollouts	Updates in incremental stages	High monitoring, better management	Longer process, high network dependency
Secure Update Architecture	Secure Phases	Adds cybersecurity layers like encryption, authentication	More secure	Increased complexity, performance overhead
Modular Update Architecture	Independent Module Updates	Modules or systems updated independently	Efficient, flexible, reliable	Complex management, security concerns
Over-the-Cloud and Local Gateway Hybrid Architecture	-	Cloud-based system with local gateways managing updates	Centralized control, offline capability	-

Table 3.4: Summary of Software Update Architectures

Update Systems	Explanation	Characteristics	Compatibility (Communication protocols)	Supported Protocols (Cybersecurity)	Pros/Cons
Mender.io	It is an OTA update software used for Linux-based systems with embedded hardware.	Mender Authentication: Uses JWT (JSON Web Tokens) for encoding and decoding. Usable with Raspberry Pi	Compatible with Linux and embedded systems using HTTPS. It also uses CI/CD (Continuous Integration and Continuous Deployment) Compatible with IOT devices and platforms	HTTPS Communication, MQTT, TLS Authentication, webhooks	**Pros**: Robust, secure, efficient management, supports scalability and remote updates **Cons**: Complex setup, higher system requirements, limited to Mender servers
Uptane	Security-focused OTA update framework, particularly used for automotive ECUs.	Consists of three components: Image repository (signed offline keys), Director repository (online signed keys), and Time Server.	Uses Docker Cloud, HTTPS, Virtual Private Networks (VPN), MQTT, SSH, Docker-API, RESTAPI, FTP/SFTP	Cryptographic Signatures, Role Separation, Metadata Verification, Versioning, and Expiry	**Pros**: Highly secure, specifically designed for the automotive industry **Cons**: Not widely adopted by OEMs, complex setup
RAUC (rauc.io)	Lightweight CLI update client; it creates, verifies, and authenticates update bundles in the target device.	Inspect, create, modify update bundles; versatile with multiple storage protocols (ext4, vfat, UBIFS, etc.)	HTTPS (Delta-Type update), D-Bus interface	Cryptographic signing and verification	**Pros**: Minimal design, lightweight, less RAM and storage usage, versatile storage support **Cons**: Limited feature set, primarily for simple embedded systems
SWUpdate (sbabic.github.io)	Framework for embedded devices that adds custom protocols and handlers to an application for OTA updates.	Uses single image delivery system and cpio format for updates. Compatible with FPGA.	Linux framework, HTTP, HTTPS, backend Hawkbit server	Supports hardware-software compatibility checks, image extraction, custom handler installation	**Pros**: Flexible framework, compatible with various devices, handles multiple update types **Cons**: Configuration complexity, lacks built-in security features
Resin.io (now balena)	Platform for deploying and managing fleets of IoT devices. Uses a Push-type OTA architecture.	Balenacloud for device management, BalenaOS for OS, and BalenaEngine for container runtime.	Uses Docker Cloud, HTTPS, VPN, MQTT, SSH, Docker-API	Authentication and Authorization, TLS/SSL Encryption	**Pros**: Easy to manage fleets of devices, strong container support, user-friendly interface **Cons**: Limited to Balena ecosystem (cloud lock-in)
UpdateHub (Updatehub.io)	User-friendly app for software and OS updates with rollback as a feature.	App-controlled, rollback capabilities	Simple software installation	-	**Pros**: Simple, user-friendly, supports rollback features **Cons**: Lacks extensive configurability and scalability for larger systems
Artifact Updater	NodeJS-based application used for automating the update process on embedded systems.	Lightweight script-based client	-	-	**Pros**: Lightweight, easy to integrate with NodeJS environments **Cons**: Minimal security features, limited functionality compared to other systems
IOT.js	Lightweight JavaScript framework designed for resource-constrained IoT devices.	Utilizes JerryScript, a lightweight JavaScript engine, for constrained devices.	MQTT, HTTP, HTTPS, WebSocket	Hashing, Key Management, Secure TLS, Data Integrity, and Authentication	**Pros**: Lightweight and suitable for constrained IoT devices, JavaScript-based for ease of use **Cons**: Limited feature set, primarily for microcontroller-based systems

Table 3.5: Reference Architectures Used in Automotive Industry

Architecture	Details
Uptane	OS: Linux, FreeRTOS, Zephyr, Mbed OS, RIOT OS, Contiki-NG, Ubuntu, Debian etc. Hardware: ARM Cortexv7 and v8 x86 architecture Storage: Needs enough Storage and RAM. Stable power supply, Backend infrastructure, API for device management Bootloader: Primary and Secondary Security: Hardware Security Modules, Trusted Platform Module (TPM), Secure elements, AES, RSA, Code Signing and Verification, Secure Boot, Chain of Trust, Integrity Checks Tools: Uptane Libraries, TUF (Uptane Framework) Programming: C/C++, Python, Shell Scripting, YoctoProject Power Consumption: High (not a priority)
RAUC	OS: Linux, FreeRTOS, Zephyr, Mbed OS, RIOT OS, Contiki-NG, Ubuntu, Debian etc. Hardware: ARM Cortexv7 and v8 x86/x64 architecture, MIPS, PowerPC, RISC-V Storage: Lightweight, needs enough for A/B type updates and storage Bootloader: U-boot and barebox Security: Hardware Security Modules, Trusted Platform Module (TPM), Secure elements, AES, RSA, Code Signing and Verification, Secure Boot, Chain of Trust, Integrity Checks Tools: Linux Kernel, Update Client, GLib or SSL/TLS Programming: C, Shell Scripting Power Consumption: High (not a priority)
SWUpdate	OS: Linux (Ubuntu, Debian, Yocto, Buildroot, etc.) Hardware: ARM (Cortex-A, Cortex-M), x86/x64 architecture, MIPS, PowerPC, RISC-V Storage: Lightweight, enough for A/B type updates (dual-rootfs system), fault tolerance Bootloader: U-Boot supported, compatible with bare-metal bootloaders Security: Hardware Security Modules (HSM), Trusted Platform Modules (TPM), AES, RSA, Code Signing and Verification, Secure Boot, Chain of Trust, Integrity Checks Tools: Linux Kernel, Update Client (SWUpdate), GLib, SSL/TLS Programming: C, Shell Scripting Power Consumption: High (not a priority)

Table 3.6: Comparison of Uptane, RAUC, and SWUpdate architectures

Feature/Aspect	NGINX	Apache	Built-in Authentication
Performance	High, low resource usage	Good, but higher resource usage	Depends on implementation, likely limited
Ease of Use	Simple config for basic needs	More complex, but highly configurable	Easiest if well-implemented, but limited
Authentication Flexibility	Basic authentication, third-party	Supports various auth mechanisms	Likely limited to basic authentication
Module Support	Limited, mostly built-in features	Extensive third-party module support	Minimal or none
Security	High, simple and secure	High, extensive options	Depends on implementation, lower in general
Scalability	Excellent for high traffic	Good, but may struggle under extreme load	Limited scalability
SSL/TLS Support	Easy to configure	Easy to configure	Likely limited or manual configuration
Resource Usage	Low	Higher than NGINX	Depends, usually low

Table 3.7: Comparison of NGINX, Apache, and Built-in Authentication

Papers	Description, Features and Findings	Optimization Parameters	Implementation Techniques
OTA Updates in Embedded Microcontroller Applications: Design Trade-Offs and Lessons Learned	• Design trade-offs in OTA Updates. • Focus on rapid software lifecycles for microcontrollers. • Efficient hardware use: Direct Memory Access (DMA) and Low power modes. • OTA split into Second Stage Bootloader (SSBL), OTA Client, HAL, libraries.	• Reduces memory lifecycles. • Focus on footprint and cycle optimization.	• SSBL for boot management. • AES-128 and SHA256 encryption. • Hash chain automation.
On Optimal Scheduling of OTA Software Updates for Smart Vehicles Leveraging Fog Computing	• Proposes optimal scheduling for OTA. • Introduces methods SP1 (reduce waiting) and SP2 (maximize transmission). • SP1: Reduces waiting time. • SP2: Maximizes transmission time.	• Faster response with fog computing. • Maximizes waiting/-transmission efficiency.	• 2 algorithms: Random and Greedy. • Local search optimizes performance.
Faster OTA Updates in Smart Vehicles using Fog Computing	• OTA updates from fog to vehicles. • Reduces handovers and update time. • 37% improvement over cloud data centers. • Impact on software size, mobility, and transmission.	• Data transfer rate. • Fog node range.	• Fog Computing framework to avoid dealership visits. • Mobility management reduces handovers.
Automotive OTA Upgrade Scheme Based on Optimal Difference Algorithm	• Minimizes data transfer using delta updates. • Efficient bandwidth usage. • Faster updates in small data packets. • Energy efficient.	• Data size and compression. • Bandwidth and time efficiency.	• Difference algorithm for updates. • Fail-safe scheduling and application.

Table 3.8: OTA Optimization

Table 3.9: IDS used in Automotive Industry

IDS	Developed by	Algorithm	OEMs that Use It	Features
ESCRYPT	Bosch	Intrusion Detection and Prevention System (IDPS)	Multiple Automotive OEMs	End-to-end encryption for OTA updates, monitors for tampering, anomaly detection, secure delivery and installation of updates.
Argus	Karamba Security	In-vehicle network monitoring	Several OEMs	Monitors OTA update traffic for anomalies, prevents MITM attacks, detects unauthorized access to update channels.
C2A Security	C2A Security	Behavioral and signature-based detection	Various OEMs	Ensures OTA package authenticity and encryption, monitors update integrity, protects against tampering and unauthorized updates.
Kaspersky Automotive Adaptive Security	Kaspersky	Threat detection with encryption and authentication	OEM partnerships	Monitors OTA communication channels, encryption and integrity checks, prevents unauthorized access and tampering of updates.
NXP Secure Gateway	NXP	Traffic filtering, anomaly detection	Tier 1 suppliers and OEMs	Filters and monitors traffic, ensures only authorized OTA updates pass through, detects and blocks abnormal traffic.
Snort (Prototyping)	Cisco (Open Source)	Signature-based IDS	Used in testing/prototyping environments	Network monitoring, detects unauthorized traffic during OTA updates, suitable for Raspberry Pi environments.
Suricata (Prototyping)	Open Information Security Foundation	Signature-based and anomaly detection IDS	Used in testing/prototyping environments	Monitors OTA traffic, detects anomalies, supports Raspberry Pi setups for prototyping and testing OTA security.

Feature	Snort	Suricata
Architecture	Single-threaded (Snort 3: multi)	Multi-threaded from the start
Performance	Lower throughput in high traffic	Handles higher throughput, better scaling
Protocol Support	Good	Excellent, with advanced protocol support
Detection Method	Signature-based	Signature-based + anomaly detection
Logging	Flexible, requires configuration	Native JSON, multi-format, full packet capture
Community	Large, backed by Cisco	Active, backed by OISF
Best For	Small to medium-sized networks	Large networks, high traffic, full protocol analysis

Table 3.10: Comparison between Snort and Suricata

Chapter 4

System Architecture

The following chapter presents the System Architecture of the prototype designed to simulate an Automotive OTA Update.

4.1 Overview of Proposed System prototype

As mentioned in Phase 1, Chapter 3, the finalized components for the prototype were identified as follows:

1. Raspberry Pi 3B+ is utilized to simulate the ECU
2. Raspberry Pi 3B+ is utilized to simulate the TCU
3. A CAN bus connection is established between the devices using the Waveshare WH-CAN HAT Shield enabled in the Raspberry Pis.
4. Directories named **Vehicle/fdata** are created in both the ECU and TCU, containing all the files necessary for implementing vehicle functions, such as fuel meter values and volume control.
5. A graphical interface is implemented using Tkinter, a standard Python library for creating Graphical User Interface (GUI)s, which simulates a panel to display mock sensor values and provides interactive buttons and sliders for changing values, such as volume.
6. The Raspberry Pi functioning as the TCU is configured with the OTA update reference architecture—SWUpdate—to enable remote OTA updates.

4.2 Detailed Description of System Components used in the prototype

In the following sections, all the devices and protocols that comprise the prototype are described, along with details about the functioning of the system.

4.2.1 In-vehicle Modules

The in-vehicle architecture in a Vehicle, as shown in Figure 1.2: OTA Update Architecture, consist of the following Components:

1. The Raspberry Pi 3B+ is an ARM-based processor equipped with multiple I/O interfaces and support for various peripherals. General Purpose Input/Output (GPIO) pins are provided for interfacing with external hardware, such as sensors and actuators, making it suitable for electronics prototyping and IOT applications. The board is compatible with a range of operating systems, primarily Linux-based distributions like Raspberry Pi OS, and offers connectivity options, including HDMI, USB, Ethernet, and Wi-Fi. In this project, the microcontroller is utilized as both the ECU and the TCU, with the following features of the device being employed:

   (a) GUI mode is accessed on the Raspberry Pi using the TightVNC application. The TightVNC application is installed on the Debian Operating system (OS), and the corresponding application is installed on Windows. Once the application is running on the Raspberry Pi, the VNC server, which hosts the GUI mode of the Raspberry Pi, can be accessed by connecting to the IP address or hostname of the device.

   Alternatively, the Raspberry Pi 7TD 7" TFT LCD-Touch Display can be employed to access the GUI directly from the Raspberry Pi instead of using TightVNC.

   (b) Serial Peripheral Interface (SPI): It is a synchronous communication protocol based on a master-slave architecture, where communication is controlled and initiated by the master device, with one or more slave devices responding accordingly. Although SPI is not directly utilized, it is required for enabling CAN communication.

   (c) Tkinkter: Tkinter, the standard GUI library in Python, provides a powerful and easy-to-use toolkit for creating windows, dialogs, buttons, labels, text boxes, and other typical graphical user interface elements. A GUI panel is created using this library to generate random values for various parameters, along with offering user-controlled manual options.

   (d) A/B type Dual Bank: A/B type Dual Bank refers to a structure within the memory of the Raspberry Pi that enables software redundancy presented in Figure 4.1: A/B Type Partition Architecture. The memory is divided into two separate and independent storage areas, each containing software and/or firmware. When the device is operating on the software in partition A and an update is detected, the update can be downloaded and installed in partition B without disrupting the system. Upon reboot, the bootloader is configured to boot the device from partition B. To achieve this, a compatible bootloader must be selected that aligns with Automotive OTA Update Architectures. The following bootloaders are compatible with the Raspberry Pi 3B+:

      i. (Default) RaspberryPi Firmware bootloader: The default Raspberry Pi firmware bootloader does not support dual or multi-boot capabilities.

ii. U-Boot: This traditional bootloader is used for multiple applications, with key features including dual/multi-boot capabilities based on user input and/or configuration files, as well as compatibility with various open-source architectures.

iii. GNU GRand Unified Bootloader (GNU GRUB): Similar to U-Boot, this bootloader is highly configurable and flexible, making it suitable for industrial-level applications. However, it is not directly compatible with the Raspberry Pi and requires additional configuration to function, which increases complexity and the likelihood of failure, rendering it unsuitable for the system.

iv. New Out Of Box Software (NOOBS): This beginner-level bootloader supports dual-boot but lacks flexibility due to its limited customization and configurability. Its enhanced version, PINN, offers additional features but remains less flexible and supportive than U-Boot for industrial-level applications. The lack of flexibility in NOOBS is primarily attributed to its focus on simplicity and user-friendliness, which restricts advanced configuration options and customization capabilities.

Hence, U-Boot is chosen as the bootloader.

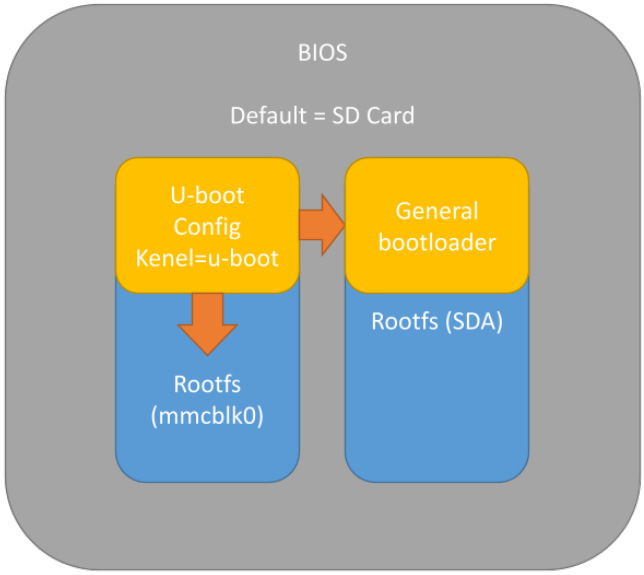


Figure 4.1: A/B Type Partition Architecture

(e) Wi-Fi: Wireless connectivity is provided by Wi-Fi on the Raspberry Pi, enabling access to networks without the need for physical Ethernet cables. This feature is utilized for network access, allowing remote connections to the Raspberry Pi via Secure Shell (SSH).

2. Waveshare 2-CH CAN HAT: The Waveshare 2-CH CAN HAT is designed as an add-on board for the Raspberry Pi, enabling communication over the CAN bus, which

is a robust and widely used communication protocol in automotive and industrial applications. The HAT is built around the MCP2515 CAN controller, a popular and reliable chip from Microchip that manages the specifics of the CAN protocol. This configuration allows communication over the CAN network via SPI on the Raspberry Pi. The above setup is shown in Figure 4.2: CAN-HAT Mounted on Raspberry Pi

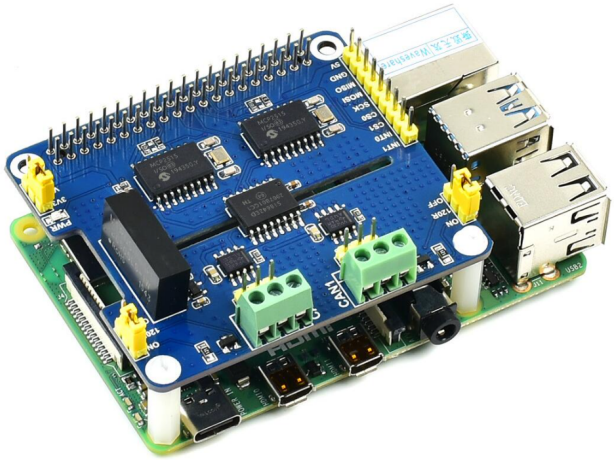


Figure 4.2: CAN-HAT Mounted on Raspberry Pi

Image source: https://www.waveshare.com/wiki/2-CH_CAN_HAT

4.2.2 Server and Cloud Infrastructure

The server-cloud infrastructure is considered an integral part of the system, responsible for storing the update files. It remains synchronized with the backend devices and the vehicle, verifies the vehicle devices and update files, and subsequently allows for the download, as shown in Figure 1.4: Role of cloud in OTA update Architecture. In this case, a specific cloud storage is not utilized, as it would not be particularly useful due to the minimal amount of data and update files required. In the prototype, only a secure server integrated with SWUpdate is set up, which is under direct control of a backend device, allowing for updates to be scheduled or manually pushed. The Nginx server is integral to our system architecture for managing the uploading and downloading of updates. It hosts the Hawkbit Update Manager, facilitating seamless communication between the backend system and target devices. Key functionalities of the Nginx server include:

1. Secure Hosting: Nginx supports HTTPS to ensure encrypted communications during the update process.

2. Hawkbit Integration: By hosting the Hawkbit Update Manager, the server can effectively manage and schedule OTA updates, track update statuses, and handle rollbacks.

3. Update Management: The backend system generates update packages that are uploaded to the Nginx server for retrieval by target devices, allowing for both scheduled and manual push updates.

4. Data Synchronization: Nginx verifies the authenticity of update files and devices, maintaining a secure update environment.

4.2.3 Backend System

The backend system is considered the most important component in an OTA update process, as it is responsible for creating the update packages, securing them, and then uploading them to a server, from which the updates are subsequently delivered to the target device. The SWUpdate software is installed on this backend to facilitate the creation of the update .swu packages. Additionally, the Hawkbit server is hosted on this backend, enabling effective management and scheduling of OTA updates. The Nginx server acts as a reverse proxy, ensuring secure communication between the backend and the target devices while handling the upload and download of updates efficiently. This architecture allows for a streamlined and secure OTA update process.

4.3 Communication protocols

The entire system is based on two different sets of communication protocols: one is the in-vehicle communication bus, and the other is the vehicle-to-external communication. In this case, CAN is used as the in-vehicle communication bus, while Wi-Fi is utilized for vehicle-to-external communication.

4.3.1 CAN Bus

The most commonly used protocol in automotive vehicles for communicating with different peripherals and controllers using a simple connection is the CAN bus, which connects all devices as nodes.

The Waveshare CAN HAT Shield (Transceiver) is used to enable the CAN bus in the Raspberry Pi through three connections: Common ground, CAN High, and CAN Low.

Currently, the following tasks are performed using the CAN bus:

1. All sensor and actuator data is moved/uploaded on a single line of communication
2. Files are sent as data packets.
3. Files specific to the target node's requirements are received, while irrelevant data is considered a dump by the target node.

4.3.2 Wi-Fi

In our case, Wi-Fi is used for multiple applications, with the primary use in an automotive vehicle being the following:

1. Communication with the cloud is facilitated.
2. V2X communication is enabled.

3. Updates are downloaded.

4. Network configurations with the cloud are checked.

In our case, Wi-Fi is utilized to establish an SSH connection with the Raspberry Pi. This setup allows secure remote access to the device, enabling management and control of the Raspberry Pi from a distance. The SSH protocol ensures that all data transmitted over the network is encrypted, protecting against eavesdropping and unauthorized access. Through this connection, commands can be executed, configurations modified, and real-time monitoring of system performance conducted, enhancing the overall management and functionality of the prototype system. This capability is considered particularly important for debugging, software updates, and adjusting settings without the need for direct physical access to the device.

4.4 OTA Feature and Security

Based on the analysis in Section 3.2: Phase 2 - OTA Update Implementation Reference Architectures, it is concluded that the OTA update architecture can be implemented using the SWUpdate reference architecture. The following features of SWUpdate are relevant to the thesis topic:

1. The primary purpose of SWUpdate is to manage software updates in embedded Systems.

2. It focuses on safe and reliable firmware and software delivery to devices.

3. SWUpdate has integrated features which can be used to improve security:

   (a) Cryptographic verification is employed to prevent tampering of updates and unauthorized access.

   (b) Secure boot and integrity checks are supported, ensuring that only trusted software is executed. Integrity checks validate the authenticity and integrity of the update images before installation.

   (c) Encrypted communication is facilitated, with support for encrypted channels (e.g., HTTPS) to securely deliver updates, ensuring data protection during the update process.

   (d) Access control and authentication measures are integrated, restricting access to the update service and allowing only authorized users to trigger or perform updates.

   (e) Signed update packages are utilized, ensuring that only valid and authorized updates are installed on the device.

4. A secure server can be integrated with SWUpdate, in this case, the Nginx web server, which is integrated with the Hawkbit update manager.

5. Despite the integration of these cybersecurity properties, the possibility of multiple attacks aiming to bypass these protocols and properties exists. To counter these threats, requirements for low power consumption, intrusion detection, and secure algorithms to prevent attacks in the first place are established.

6. Based on the research and understanding of the comparative analysis of the OTA architectures and several case studies of version rollback attacks, a TARA based evaluation of ISO 21434 is performed. As a result, the security goals and security concepts needed to mitigate the risks are defined.

7. Drawing on the security goals and security concepts, new algorithms are then proposed that can help mitigate risks.

Chapter 5

Implementation of the Prototype

This chapter outlines the development environment and implementation steps for the prototype designed to facilitate OTA updates. It details the setup process for the TCU, which is based on a Raspberry Pi, including the partitioning of memory and the installation of the U-Boot bootloader. The steps are structured to ensure that the Raspberry Pi can boot from an external USB and configure necessary overlays for functionality, such as GUI and CAN modules. Additionally, the chapter discusses the integration of SWUpdate as the reference architecture for the OTA update process, including the installation of the latest source code and the configuration of security protocols. The implementation also covers setting up a secure server using Nginx, along with intrusion detection software on the Raspberry Pi, enhancing the overall security posture of the system. Through these comprehensive steps, a robust framework for secure and efficient OTA updates is established.

5.1 Development Environment

The development environment here consists of:

1. ECM (Raspberry Pi): The ECM should be running as mentioned in Chapter 4. It simulates mock sensor data as shown in the Figure 5.1: ECM Simulation Pannel using tkinter with the script mentioned in Appendix A. The ECM is connected to the TCU using a CAN bus as shown in Figure 5.2: Prototype of Automotive OTA update.

2. TCU (Raspberry Pi): The TCU should also be running as described in Chapter 4. The TCU controller should also simulate TCU functions like latitude, and others as shown in Figure 5.3: TCU Simulation Pannel with the script mentioned in Appendix A. Once the TCU is running, the current MicroSD memory in the device is used as Partition A, while a partition B is set up on an external USB as depicted in Figure 5.2: Prototype of Automotive OTA update. The successful completion of this process requires the following steps:

   (a) The Raspberry Pi is programmed using the original memory to enable booting from the USB. The commands for this step are provided in the attached Appendix A. If the output matches the expected output mentioned in Appendix A, the device is successfully programmed to boot from the USB.

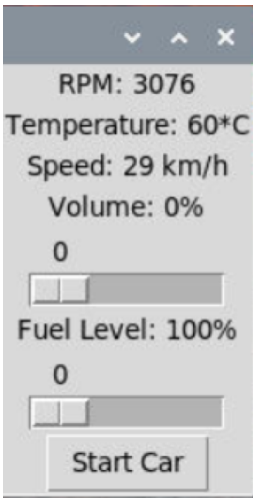


Figure 5.1: ECM Simulation Pannel

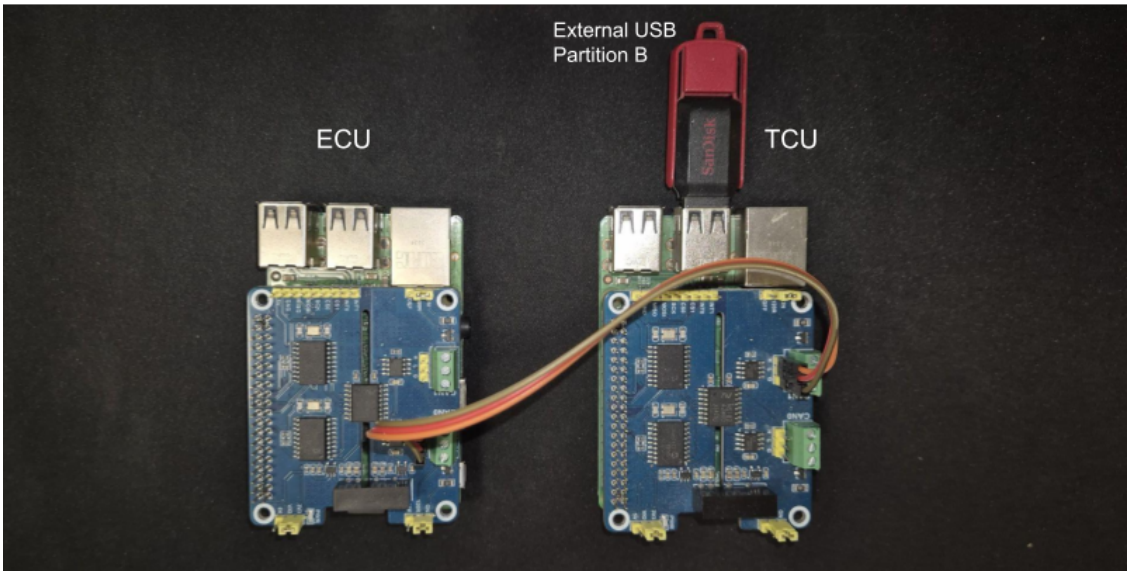


Figure 5.2: Prototype of Automotive OTA update

(b) The U-boot bootloader is installed in the main memory, as discussed in section 4.2.1d: In-vehicle Modules.

(c) The data from the U-boot documentation is utilized for setting up U-boot. After multiple approaches, the most suitable and successful method in the current scope is mentioned in the attached Appendix A. Once U-Boot is built, the generated u-boot.bin file is copied to the /boot/firmware directory of the SD card memory

(d) Configuration changes must be made in the /boot/firmware directory of the Raspberry Pi SD memory.

(e) After rebooting, the U-Boot bootloader is launched as the terminal. This U-Boot terminal can be configured to boot the original Linux image, with the relevant commands provided in Appendix A.

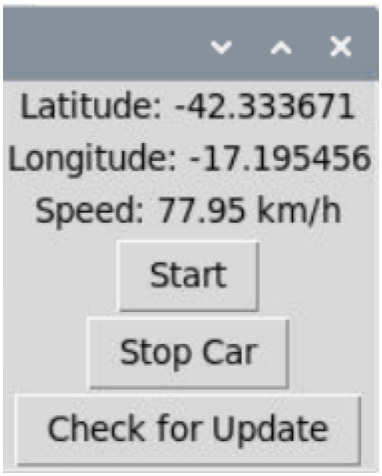


Figure 5.3: TCU Simulation Pannel

(f) The kernel should boot and be accessible by using SSH.

(g) Overlays are required by the Raspberry Pi to load files that are not included by default. For the GUI, an overlay called vc4-kms-v3d is needed, which is typically included in the config.txt file. However, when the config file is loaded with the kernel set to u-boot.bin, the overlays are not loaded. Consequently, after the kernel boots and the Raspberry Pi is accessed via SSH, a script can be created to manually include the overlays. This step is important as the overlays contain the CAN module overlays and GUI overlays.

3. As per the analysis in section 3.2.2: Analysis (Phase 2), SWUpdate is used as the implementation reference architecture for the OTA update architecture. The setup of SWUpdate is performed through the following steps:

   (a) The GitHub source with the latest patches and updates (2024.05) is installed in this case.

   (b) A configuration file is created using the menuconfig, where all possible cybersecurity protocols supported in SWUpdate are enabled. Issues with some libraries are encountered, preventing the functioning of certain features that can be selected through the menuconfig.

   (c) To ensure successful compilation of SWUpdate, certain features need to be excluded. The following features are listed:

      i. Lua and MTD support are enabled in SWUpdate Settings > General Configurations, as they are important for customization and pre/post installation of updates.

      ii. PKCS#11 is excluded due to two errors. The first error, related to the pk11-kit-lib location, is resolved by changing the file location in include/sslapi.h. The second error, involving a clash between OpenSSL and WolfSSL libraries, persists even after excluding either one of the include headers.

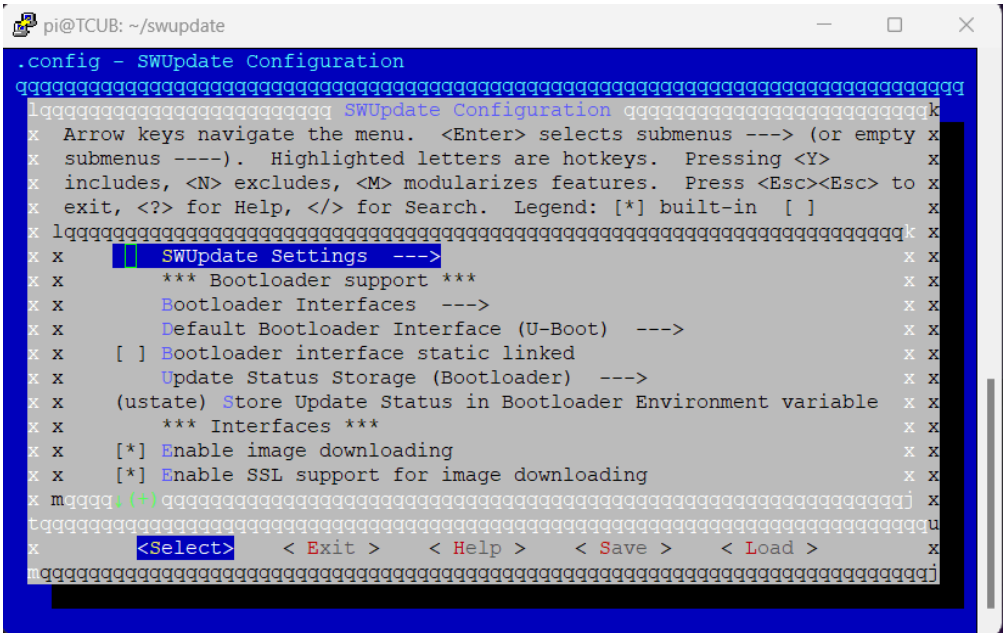


Figure 5.4: SWUpdate make menuconfig

iii. In the Handlers, all types of handlers are accepted except for the Delta update handler and the BTFSFile system. The Delta file option causes errors with undeclared variables in the script, which are too complex to correct, and the BTFSFilesystem requires specific libraries that are not recognized even after installation.

In the Handlers, all types of handlers are accepted except for the Delta update handler and the BTFSFile system. The Delta file option is associated with errors regarding undeclared variables in the script, which are too complex to correct. The BTFSFilesystem requires specific libraries that are not recognized even after installation.

i. In the general configurations settings for SWUpdate in the menuconfig, Lua5.3 and MTD support are included. These two are important as the Lua5.3 package enables customization of the update process, while MTD support is crucial for compatibility with raw flash devices such as NAND, NOR, or other memory devices.

ii. The Default Bootloader must also be configured. For the Raspberry Pi, which is the target, it is set to U-Boot, while the host device uses the built-in bootloader without any further configuration changes.

iii. Other straightforward options that are required include enabling SSL support for image downloading, allowing SHA256 hash for each image, enabling verification of signed images, encrypting the sw-description file, and enabling the WebServer setting.

iv. Lastly, one of the most important customizations is the use of Image Handlers. This feature allows for different file formats for the update image to

be utilized, with the image type defined to link with the respective handler. Once the image type is set, the necessary handler information and fields are populated in the device.

(d) The make command is then followed by sudo make install.

(e) The SWUpdate system is successfully built, and it can be checked using the SWUpdate –version command.

4. Once SWUpdate is set up on the device, a .swu package can be created and sent to the system via OTA, allowing the target to use the TCU internal files to validate, unpackage, and install the update. The .swu package contains a sw-description file that defines the target locations, scripts to be updated, directories to be modified or replaced, and other relevant information regarding the version number, hardware compatibility, and more. The package also includes scripts, update images, delta-type updates, and multiple supporting scripts, such as pre-install and post-install.

5. After enabling the network settings on the host, it can be configured for all devices on the network to access the IP address of the SWUpdate Server.

   (a) The server details can be customized in the /examples/www section.

   (b) In Ubuntu Linux, the /etc/netplan/.yaml file needs to be configured with the enp0s3 IP address, gateway, and DNS.

   (c) The new netplan is then applied.

   (d) Next, firewall rules are allowed using ufw allow 8080/tcp.

   (e) Since this setup is performed on Ubuntu in a Virtual Machine, the network settings must be changed from NAT to Bridged in the VirtualBox settings.

6. Access to http://<IP Address>:8080 is now available to all devices in the network; however, cybersecurity measures are required to protect the system from unauthorized access and uploads.

5.2 Security Deployment

5.2.1 NGINX Deployment

Nginx is considered a robust choice for a custom update server, especially when security against version rollback attacks is taken into account. HTTPS/SSL support is provided by Nginx, which encrypts data in transit, ensuring that update requests and responses remain secure. Access control can also be managed and update versions validated through Nginx, helping to prevent unauthorized or malicious rollbacks. The modular design allows for the integration of security features and custom rules that can detect and mitigate rollback attempts. By leveraging Nginx's performance and security capabilities, effective safeguarding against potential rollback attacks can be achieved while updates are efficiently managed. In addition to its security features, Nginx is considered highly suitable for managing large volumes of update requests from multiple devices simultaneously due to its performance and

scalability. Static files are efficiently served by Nginx, minimizing latency during the update process to ensure that updates are delivered swiftly to the target devices. Nginx is integrated with Hawkbit, which functions as the update manager responsible for the centralized control of OTA updates, including version management and device communication. The configuration of Nginx also allows access attempts to be logged and traffic patterns to be monitored, providing insights into potential security threats. Its reverse proxy functionality enables the seamless integration of additional security layers, such as Web Application Firewalls (WAF), further enhancing the protection of the OTA update process. Through this comprehensive approach, the update management is streamlined, and the security posture of the system is fortified, creating a reliable and secure environment for OTA updates.

5.2.2 Intrusion Detection Algorithms

In the context of OTA updates and automotive security, intrusion detection algorithms play a vital role in identifying and mitigating potential threats to the system. Suricata, is a good option for an IDS. It is an open-source network threat detection engine, is a highly effective tool for intrusion detection and prevention in such environments. It operates by analyzing network traffic and applying predefined rule sets to detect suspicious activity, abnormal behavior, or known attack patterns.

Suricata's integration into the OTA update architecture provides an additional layer of defense by constantly monitoring the incoming and outgoing traffic associated with the update process. It can detect anomalies, such as unauthorized access attempts, potential rollback attacks, or unexpected data transfers that may indicate a security breach. Suricata's rule-based detection system allows for real-time analysis and immediate responses to potential threats, reducing the likelihood of successful exploits.

In this chapter, the implementation of a secure OTA update system was detailed, establishing a prototype using a Raspberry Pi as the ECU and TCU. Key components, including the setup of SWUpdate, the deployment of Nginx for secure server management, and the integration of the Suricata intrusion detection system, were outlined to enhance the overall security posture against potential threats. With these foundational steps completed, the next chapter will focus on evaluating the effectiveness of the proposed security algorithms, evaluating the developed prototype using Clause 15. TARA of ISO21434.

As a summary the above, the Figure 5.5: Prototype Architecture, shows the system architecture of the prototype

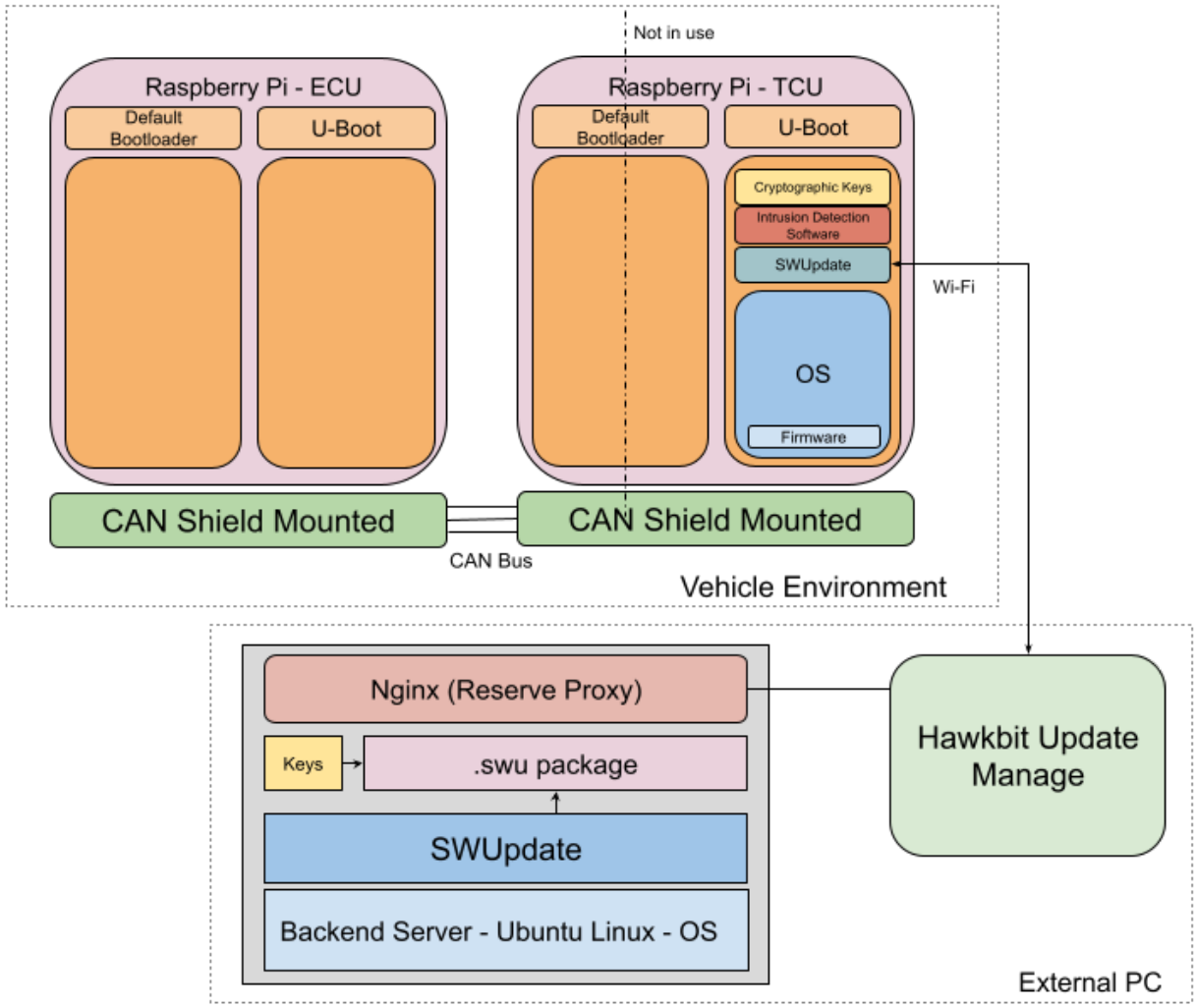


Figure 5.5: Prototype Architecture

Chapter 6

TARA Evaluation based on ISO21434

This chapter provides an evaluation of the developed prototype proposed in Chapter 4 and 5.

6.1 Evaluation Criteria

In this section, ISO 21434 is followed as a reference, specifically Clause 15 on Threat Analysis and Risk Assessment methods, to identify various attack paths and vectors. Accordingly, algorithms for the protection of the OTA update system are proposed.

6.1.1 Asset Identification

In ISO 21434, the term *asset* is used to refer to any item requiring protection from cybersecurity threats. In the context of this thesis, focusing on OTA updates and version rollback attacks, these elements are considered critical assets. The following assets have been identified in the system:

1. **Vehicle ECUs**: The embedded systems in the vehicle that receive the updates must be protected from unauthorized downgrades or rollbacks to maintain security.

   (a) **ECM**:The Engine Control Module can be identified as an asset. It is considered the main functional unit of an automotive vehicle architecture. The ECM serves as the brain of the working vehicle, and its compromise may lead to severe problems for road users.
   Cybersecurity properties: Confidentiality, Integrity and Availability
   Damage Scenario: If the ECM is compromised, consequences may include the loss of control over critical functions, disruption of communication, malicious firmware updates, data theft or manipulation, OTA update hijacking, and the disablement of safety features.

   (b) **TCU**: The Telematics Control Unit can also be identified as an asset under vehicle ECUs. The TCU is responsible for features that require a connection with external networks, including V2X communication, OTA updates, remote diagnostics and monitoring, emergency calls, navigation and GPS, and remote management

and control.
Cybersecurity Properties: Confidentiality, Integrity and Availability
Damage Scenario: If the TCU is compromised, potential consequences include data theft and privacy breaches, remote vehicle control, DoS attacks, malicious OTA updates, man-in-the-middle attacks, fleet management disruption, false eCall activation, supply chain attacks, disruption of V2X communications, and monetary extortion (ransomware).

2. **OTA Communication channels**: The network infrastructure, including Wi-Fi used for data transmission, must be secured to prevent interception or manipulation of update packages.

   (a) **Wi-Fi**: Wi-Fi is utilized by the TCU as a communication channel to interact with the external environment.
   Cybersecurity properties: Confidentiality, Integrity and Availability
   Damage Scenario: If Wi-Fi as a communication channel is compromised, the vehicular systems would be prone to confidentiality and privacy breaches, such as exposure of real-time vehicle sensor data or unauthorized access to proprietary vehicle performance data. Additionally, integrity compromises of OTA update packets and server cloud communications could occur, along with availability disruptions of OTA updates, network disruptions, and a lack of communication. A man-in-the-middle attack could also lead to tampered data and messages from external communication nodes.

   (b) **CAN**: The Controller Area Network is also utilized as a communication channel in the prototype. It is used to transmit and receive sensor data between vehicle nodes. Additionally, it can be employed to send files split into CAN messages, which include a header and footer to recognize the order of the file for reintegration.
   Cybersecurity Properties: Availability
   Damage Scenario: If an attacker is able to tap into the CAN communication channel, a DoS attack may disrupt critical vehicle functions. Messages can be injected to alter vehicle operations, and CAN messages can be saved by the attacker for later replay. Additionally, CAN data may be spoofed and manipulated, firmware OTA update files could be damaged for target nodes on the CAN bus, and sensitive vehicle information may be accessed by the attacker.

3. **Update Verification Mechanisms**: Systems that check the integrity and authenticity of updates on the vehicle may be compromised, making version rollback attacks easier.

   (a) **SWUpdate**: The secure updating of software on embedded devices, including vehicles in automotive systems, is the responsibility of this system.
   Cybersecurity Properties: Confidentiality, Integrity
   Damage Scenario: The compromise of SWupdate may result in tampering of update packages, unauthorized access, DoS attacks, tampering of the manifest

and configuration files, failure to verify updates, disabling of rollback protection mechanisms, and man-in-the-middle attacks.

(b) **Update Server (Nginx with Hawkbit)**: The server responsible for the update can be identified as an asset, including Nginx integrated with Hawkbit. Nginx is used as the reverse proxy and is primarily utilized for its security features and for managing HTTP/HTTPS requests. It is integrated with Hawkbit, which functions as an update manager. OTA updates are orchestrated by Hawkbit, and update versioning is managed, scheduling is performed, and records of which vehicles need specific updates are kept.
Cybersecurity Properties: Confidentiality, Integrity and Availability
Damage Scenario: The compromise of the server may lead to version rollback attacks, man-in-the-middle attacks, unauthorized access, DoS attacks, malware injection, and server configuration and data breaches.

4. **Cryptographic Keys**: The keys used for signing and verifying updates must be protected, as their compromise could allow attackers to forge signatures for older software versions.

   (a) **Private Signing Keys**: Digital signatures are used to sign packages, ensuring that the software update is authentic and originates from a trusted source.
   Cybersecurity Properties: Confidentiality, Integrity
   Damage Scenario: If they are compromised or leaked, unauthorized updates, version rollback attacks, and system takeovers could occur.

   (b) **Public Verification Keys**: The signature on update packages is verified by these keys to confirm that they were signed by the authorized party.
   Cybersecurity Properties: Integrity
   Damage Scenario:The compromise of these keys may lead to the acceptance of malicious updates, denial of updates, and a breakdown of trust.

5. **Backend Infrastructure**:The latest software versions are produced and hosted on the backend device. Protecting this system ensures that only valid and current software versions are distributed to the vehicle by authorized OEM personnel.
   Cybersecurity Properties: Confidentiality, Integrity and Availability.
   Damage Scenarios: If the backend infrastructure is compromised, access to user data as well as OEM data may be gained by the attacker, along with remote control capabilities, the ability to send malicious updates, unauthorized access, and potential damage to OEM data and reputation.

6.1.2 Threat Scenario Identification

In the context of version rollback attacks, the following threat scenarios corresponding to all the identified assets are presented.

1. ECM:

- Spoofing of ECM: A legitimate ECM could be pretended by an attacker to communicate with other vehicle systems, potentially causing malfunctions.
- Tampering: Unauthorized modification of ECM firmware could lead to incorrect engine behavior, affecting performance and safety.
- Repudiation: Logs or system data related to ECM communications may not be properly recorded, making it difficult to track changes.
- Information Disclosure: Sensitive data (e.g., engine performance data) could be exposed if not properly encrypted.
- Denial of Service: The ECM could be overloaded by an attack, causing engine failure or erratic performance.
- Elevation of Privilege: Unauthorized access to the ECM could allow for the escalation of privileges and control over other vehicle systems.

2. TCU:

- Spoofing: The TCU could be impersonated by a malicious actor to send false telematics data (e.g., location, diagnostics) to backend systems.
- Tampering: Unauthorized changes to the TCU software could affect connectivity, leading to compromised communication..
- Repudiation: False telematics data may not be traceable back to the source, enabling manipulation of vehicle tracking and diagnostics.
- Information Disclosure: Sensitive data, such as vehicle diagnostics or GPS data, could be leaked.
- Denial of Service: TCU services could be disrupted by an attack, cutting off vehicle-to-infrastructure communication.
- Elevation of Privilege: Unauthorized access to the TCU may enable access to sensitive systems, such as infotainment or navigation.

3. Wi-Fi

- Spoofing: A fake Wi-Fi network (e.g., Evil Twin) could be set up by an attacker to steal sensitive information from the vehicle's systems.
- Tampering: Manipulation of Wi-Fi connections or firmware could enable interception of communication or injection of malicious data by the attacker.
- Information Disclosure: Unencrypted Wi-Fi communications could expose data such as OTA updates or vehicle telemetry.
- Denial of Service: A DoS attack could prevent the vehicle from connecting to necessary networks for updates or diagnostics.

4. CAN

- Tampering: Malicious modifications to CAN messages could result in faulty commands being sent, affecting critical vehicle systems.

- Information Disclosure: Eavesdropping on CAN bus communication can reveal sensitive control data.
- Denial of Service: The CAN bus could be overloaded with bogus messages, leading to critical commands being delayed or lost.

5. SWUpdate
   (a) Spoofing: The SWUpdate server could be spoofed by a malicious actor to push unauthorized or malicious updates to the vehicle.
   (b) Tampering: Update packages could be manipulated or the update process could be altered, introducing vulnerabilities into the vehicle's software.
   (c) Repudiation: Insufficient logging in the system may make it difficult to trace unauthorized updates.
   (d) Information Disclosure: If update packages are not encrypted, sensitive software data could be exposed to attackers.
   (e) Denial of Service: The update process could be disrupted by an attacker, preventing the vehicle from receiving critical updates.
   (f) Elevation of Privilege: Unauthorized access to SWUpdate could enable an attacker to deploy malicious firmware or gain control of vehicle systems.

6. Update Server
   (a) Spoofing: The legitimate update server could be impersonated by attackers to send unauthorized updates.
   (b) Tampering: Unauthorized changes to the server could lead to the distribution of malicious updates.
   (c) Repudiation: If updates are not properly logged, it may be difficult to prove which version was installed or whether an update occurred.
   (d) Information Disclosure: Sensitive data about the update packages or software versions could be exposed.
   (e) Denial of Service: An attack on the server could disrupt the availability of updates to the vehicle.
   (f) Elevation of Privilege: Unauthorized access to SWUpdate could enable an attacker to deploy malicious firmware or gain control of vehicle systems.

7. Private Signing Keys
   (a) Spoofing: If private signing keys are stolen, impersonation of the OEM could be carried out by attackers to send malicious updates.
   (b) Tampering: With stolen private keys, update packages could be modified and signed by attackers, making them appear legitimate.
   (c) Information Disclosure: If private keys are leaked, sensitive cryptographic material could be exposed to attackers.

(d) Denial of Service: Compromise of private keys may result in legitimate updates being blocked or delayed as a security response.

(e) Elevation of Privilege: High-level control over the update process could be granted to an attacker through stolen private keys, enabling a wide range of attacks.

8. Public Verification Keys

   (a) Spoofing: While public keys are meant to be shared, an attempt could be made by an attacker to replace the legitimate public key with their own to intercept communications.

   (b) Tampering: Modifications to public keys could invalidate the verification of legitimate updates.

   (c) Denial of Service: Replacement of public keys could result in legitimate updates being rejected.

9. Backend Server Infrastructure

   (a) Spoofing: The backend server could be impersonated by attackers to send malicious or unauthorized updates to vehicles.

   (b) Tampering: Unauthorized modifications to the software or firmware stored on the backend could lead to the distribution of compromised updates.

   (c) Repudiation: If logging is inadequate, an attacker could deny involvement in unauthorized actions or changes to the backend infrastructure.

   (d) Information Disclosure: Sensitive data, such as user information or proprietary software, could be exposed if the server lacks proper security controls.

   (e) Denial of Service: he backend server could be overwhelmed by an attacker with requests, rendering it unable to process legitimate update requests from vehicles.

   (f) Elevation of Privilege: Access to the backend server gained by an attacker could allow for the escalation of privileges to execute unauthorized commands, modify software, or access sensitive data.

6.1.3 Impact Rating

In ISO/SAE 21434, impact rating assesses the potential consequences of cyber-attacks on vehicle assets. It evaluates the severity across categories like safety, financial, operational, and privacy impacts, ranging from negligible to severe. This helps identify the risks posed by each threat scenario, guiding the development of mitigation strategies to ensure cybersecurity in automotive systems. Tables 6.1, 6.2, 6.3: Threat Scenarios for Different Assets show the impact rating of multiple identified assets and their respective threat scenarios.
S - Safety Impact F - Financial Impact O - Operation Impact P - Privacy Impact

1. ECM: **Spoofing** can lead to incorrect engine behavior, which is associated with a severe safety impact and a severe operational impact. Repairs and legal liabilities would be required, resulting in a major financial impact, while only negligible privacy impact

Threat Scenario	Impact Rating	ECM	TCU	Wi-Fi	CAN
Spoofing	S	Severe	Major	Moderate	N/A
	F	Major	Moderate	Major	
	O	Severe	Severe	Moderate	
	P	Negligible	Severe	Severe	
Tampering	S	Severe	Major	Severe	Severe
	F	Severe	Severe	Major	Major
	O	Severe	Severe	Severe	Severe
	P	Moderate	Severe	Severe	Major
Repudiation	S	Moderate	Moderate	N/A	N/A
	F	Severe	Severe		
	O	Major	Major		
	P	Moderate	Severe		
Information Disclosure	S	Moderate	Moderate	Moderate	Moderate
	F	Major	Major	Major	Major
	O	Moderate	Moderate	Moderate	Moderate
	P	Major	Severe	Severe	Major
Denial of Service	S	Severe	Severe	Severe	Major
	F	Major	Major	Major	Severe
	O	Severe	Severe	Severe	Major
	P	Negligible	Negligible	Major	Negligible
Elevation of Privilege	S	Severe	Severe	N/A	N/A
	F	Severe	Severe		
	O	Major	Major		
	P	Major	Severe		

Table 6.1: Threat Scenarios for Different Assets 1

would occur, as not much data is exposed. **Tampering** could result in unauthorized modifications of ECM firmware, leading to incorrect engine behavior, thus having a severe safety and operational impact. High costs for repairs and increased legal liability would contribute to a severe financial impact, while tampering would expose only some vehicle data, such as speed and fuel injection, making the privacy impact moderate. **Repudiation**: If logs are not up to date, tracking changes may be hindered, resulting in less immediate impacts. Thus, the safety impact is moderate, but the operational impact is major as it may cause issues. The financial impacts are severe in such cases due to undetected attacks. **Information Disclosure** can lead to exposure of engine performance data, which has generally less critical safety and operational impact. However, loss of proprietary data can lead to major financial and legal problems, while less personal data is affected, resulting in significant information disclosure. **DoS** can lead to system failure or critical engine function failure, with severe safety and operational

Threat Scenario	Impact Rating	SWUpdate	Update Server	Private Signing Keys
Spoofing	S	Severe	Severe	Severe
	F	Major	Major	Severe
	O	Severe	Severe	Severe
	P	Moderate	Moderate	Major
Tampering	S	Severe	Severe	Severe
	F	Major	Major	Severe
	O	Severe	Severe	Severe
	P	Major	Major	Severe
Repudiation	S	Moderate	Moderate	N/A
	F	Major	Major	
	O	Moderate	Moderate	
	P	Major	Major	
Information Disclosure	S	Major	Major	Severe
	F	Severe	Severe	Severe
	O	Moderate	Moderate	Major
	P	Severe	Severe	Severe
Denial of Service	S	Severe	Severe	Major
	F	Major	Major	Severe
	O	Severe	Severe	Negligible
	P	Negligible	Negligible	Negligible
Elevation of Privilege	S	Severe	Severe	Severe
	F	Major	Major	Severe
	O	Major	Major	Severe
	P	Severe	Severe	Severe

Table 6.2: Threat Scenarios for Different Assets 2

impact, resulting in a major financial impact for repairs and negligible privacy impact. **Elevation of Privilege** can result in unauthorized changes to safety features and other engine functions, leading to severe safety and operational impact, which would result in major financial and legal impact, as well as significant privacy impact due to unauthorized access to the system.

2. TCU: **Spoofing** is associated with a major safety and privacy impact, as critical location and status data of the vehicle may be spoofed. Since this data is essential for connectivity and communication, a severe operational impact is experienced, leading to a moderate financial impact for repairs and service disruptions. **Tampering** can cause a major safety impact, as connectivity and communication may be compromised, resulting in a severe privacy impact due to access to location, diagnostics, and other sensitive data. **Repudiation** can lead to improper logs, which may moderately affect user safety indirectly, incorrect tracking, and substantial financial impact, such

Threat Scenario	Impact Rating	Public Verification Keys	Backend Infrastructure
Spoofing	S	Severe	Severe
	F	Major	Major
	O	Major	Severe
	P	Major	Severe
Tampering	S	Severe	Severe
	F	Severe	Severe
	O	Severe	Severe
	P	Severe	Major
Repudiation	S	N/A	Moderate
	F		Major
	O		Moderate
	P		Major
Information Disclosure	S	N/A	Major
	F		Severe
	O		Major
	P		Severe
Denial of Service	S	Major	Severe
	F	Severe	Major
	O	Negligible	Severe
	P	Negligible	Negligible
Elevation of Privilege	S	N/A	Severe
	F		Severe
	O		Severe
	P		Severe

Table 6.3: Threat Scenarios for Public Verification Keys and Backend Infrastructure

as billing and maintenance issues. Additionally, false telematics data can majorly disrupt several operations of a vehicle, resulting in severe privacy implications due to the potential exposure and untraceability of sensitive data like GPS locations and diagnostics. **Information Disclosure** can indirectly impact the safety and operations of the vehicle, especially if used maliciously, causing a major financial impact from exposed location and diagnostics data, with a high privacy impact due to the sensitivity of telematics data. **DoS** can lead to a loss of telematics services, severely affecting vehicle functions, which results in a major financial impact from lost services and potential liability, alongside negligible privacy impact. **Elevation of Privilege** can lead to severe safety risks due to potential unauthorized control over telematics and connected systems, majorly impacting vehicle operations, which results in severe financial impacts due to unauthorized access and potential misuse of sensitive data, alongside severe privacy implications due to potential access to sensitive personal and location

data.

3. Wi-Fi: **Spoofing** has a moderate safety impact as it can mislead the system, but its direct effect on vehicle safety is limited. However, privacy is severely affected due to potential theft of sensitive personal data, leading to major financial consequences from unauthorized access and data breaches. Operationally, it causes moderate disruptions through falsified communications. **Tampering** poses severe safety risks, potentially leading to critical failures in communication systems. It also leads to significant financial, operational, and privacy consequences due to unauthorized alterations to data and access to personal information. **Information Disclosure** can have moderate safety implications by exposing communications that could be exploited, leading to major financial losses from data breaches and severe privacy risks from leaked personal information. **DoS** has severe safety and operational impacts by disrupting connectivity, resulting in major financial losses from service interruptions, though privacy is only minimally affected.

4. CAN: **Tampering** results in severe safety and operational impacts, with the potential to alter critical commands, leading to major financial losses and significant privacy risks. **Information Disclosure** moderately affects safety and operations by exposing sensitive data, which could lead to major financial losses and significant privacy concerns from data breaches. **DoS** causes severe safety and operational disruptions due to the loss of connectivity, leading to major financial losses but minor privacy impacts.

5. SWUpdate: **Spoofing** poses a moderate safety risk, as malicious actors could inject false updates, leading to operational disruptions. Financially, it can result in major losses due to improper functioning or required system fixes. Operational impacts are severe, while privacy risks are moderate if sensitive update information is compromised. **Tampering** presents a severe risk, as altered updates can directly impact vehicle safety, leading to significant financial losses, operational breakdowns, and major privacy concerns from unauthorized access. **Repudiation** has a moderate impact, as the inability to attribute actions correctly can cause operational and financial difficulties, with severe privacy risks due to false update logs. **Information Disclosure** moderately impacts safety and operational aspects, with severe financial consequences from exposed update data, leading to significant privacy risks. **DoS** severely affects safety, financial, and operational areas by interrupting update services, though privacy concerns are minor. **Elevation of Privilege** results in severe safety, operational, and financial risks as unauthorized access can lead to significant control over update processes, posing serious privacy threats.

6. Update Server: **Spoofing** significantly threatens the update process by delivering false updates, potentially causing severe operational and safety issues. Financially, it has major consequences, while privacy concerns are moderate. **Tampering** severely impacts the update server by introducing malicious changes, posing substantial risks to safety, financial stability, and privacy. **Repudiation** moderately affects safety, financial, and operational areas, as untraceable actions lead to potential disruptions, with

significant privacy concerns. **Information Disclosure** has a major impact on financial aspects due to potential data leaks, with moderate operational disruptions and severe privacy issues. **DoS** causes severe operational and financial risks by making update servers unavailable, though privacy is minimally affected. **Elevation of Privilege** presents severe risks across all areas as unauthorized access could allow attackers to gain control of the update server, leading to severe privacy and operational impacts.

7. Private Signing Keys: **Spoofing** results in severe safety, financial, and operational impacts as unauthorized updates may be signed and deployed. Privacy is also at risk, but the main threat lies in unauthorized updates compromising the system. **Tampering** causes severe risks across all areas, as manipulating private keys could allow unauthorized entities to push updates, severely impacting safety, financial stability, operations, and privacy. **Information Disclosure** severely impacts privacy and financial areas due to exposure of signing keys, leading to major operational disruptions. **DoS** affects the ability to perform updates, with severe operational and safety risks, though privacy impacts are minor. **Elevation of Privilege** results in severe risks across all categories, as unauthorized access to private keys could lead to total system compromise.

8. Public Verification Keys: **Spoofing** has major operational impacts, as false updates may appear legitimate if the public keys are compromised. **Tampering** presents severe risks across all categories, as manipulating public keys can lead to the deployment of harmful updates. **DoS** causes major operational disruption, though the privacy impact is minimal. **Elevation of Privilege** can lead to severe consequences, as attackers could misuse the keys to control updates and affect sensitive systems.

9. Backend Infrastructure: **Spoofing** represents severe risks across all categories, as false communication and updates could compromise safety and financial stability. **Tampering** could severely affect the integrity of data and operations, with significant privacy concerns. **Repudiation** has moderate operational and privacy risks, as untraceable actions could disrupt normal functions. **Information Disclosure** poses severe financial, operational, and privacy risks due to leaked sensitive data from backend systems. **DoS** leads to severe operational and safety disruptions by halting key infrastructure services, though privacy concerns are minimal. **Elevation of Privilege** causes severe risks, as unauthorized access to the backend could result in complete control over infrastructure, leading to massive safety, financial, and privacy consequences.

6.1.4 Attack Path Analysis

Attack path analysis is a critical process in cybersecurity risk assessment, particularly within the framework of ISO/SAE 21434. It involves mapping out potential attack vectors that could be exploited by malicious actors to compromise automotive systems. By identifying entry points, techniques, and possible routes of attack, organizations can better understand how threats may evolve, how assets could be targeted, and where vulnerabilities lie. This analysis helps to prioritize risk mitigation strategies by highlighting the most vulnerable pathways to critical assets like ECUs, OTA communication channels, and update servers. In the context of ISO 21434, attack path analysis is used to ensure that security controls are

placed effectively, focusing resources on the most critical attack surfaces to protect against high-impact threats.

1. ECM

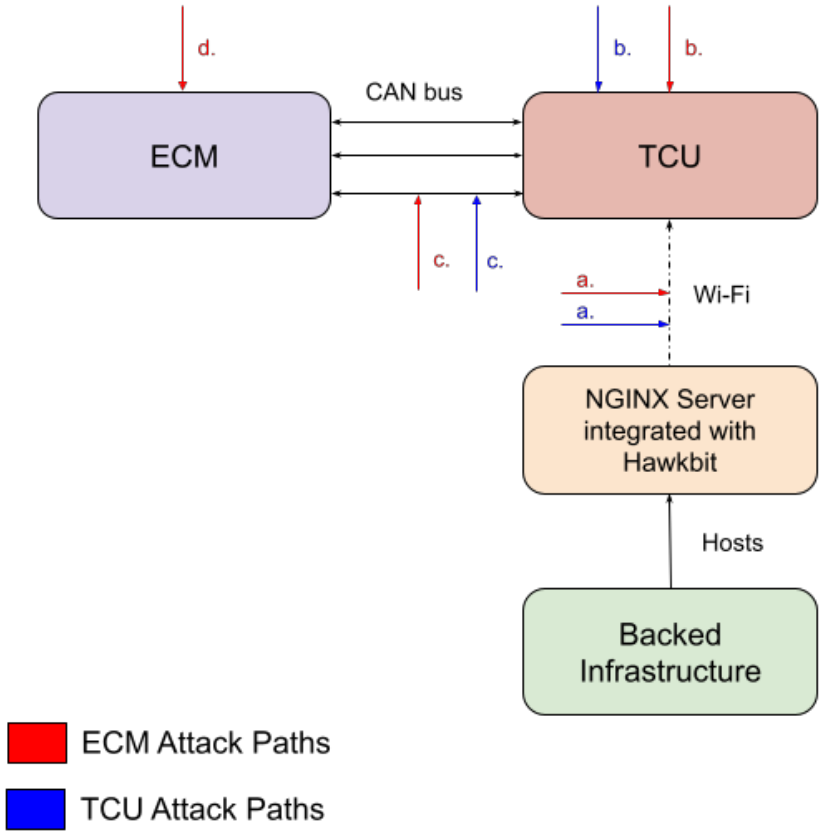


Figure 6.1: ECM and TCU Possible Attack Paths

- **Attack Paths**: As shown in the Figure 6.1: ECM and TCU Possible Attack Paths, the possible attack paths are outlined as follows.
  (a) Access can be gained via a weak Wi-Fi network connection, allowing remote connectivity to the vehicle system.
  (b) Unauthorized access to the TCU can be achieved, either physically or remotely. This may also occur due to the exploitation of vulnerabilities or the downloading of unauthorized programs on the ECU.
  (c) Access to the CAN bus and the CAN packets is available to the attacker.
  (d) Direct access to the ECM can also be obtained by physically manipulating the hardware or by exploiting any features of the ECM. Although this task is challenging, it is not impossible.

2. TCU

   - **Attack Path**: As shown in the Figure 6.1: ECM and TCU Possible Attack Paths, the following possible attack paths are identified.
     (a) Access to the TCU can be gained by the attacker via weak network configuration in the Wi-Fi.
     (b) The TCU can be physically accessed by the attacker or remotely accessed through some third-party features or exploitations of new OEM features.
     (c) Access to the CAN bus and CAN messages can also be achieved by the attacker through physical access or by exploiting vulnerabilities on one of the nodes connected to the CAN bus.

3. Wi-Fi:

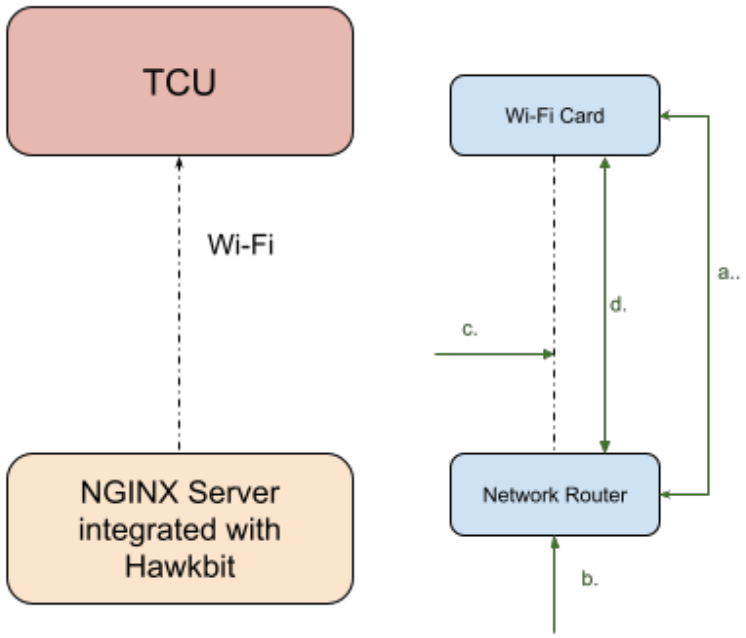


Figure 6.2: Wi-fi Possible Attack Paths

   - **Attack Path**: The attack paths are illustrated in Figure 6.2: Wi-fi Possible Attack Paths.

(a) The connection between the Wi-Fi of the TCU and the network router can be compromised by using a Man-in-the-Middle attack.

(b) The network router can be compromised, either physically or remotely, granting the attacker access to network credentials and various devices present in the network.

(c) Communications between the router and the TCU can be intercepted and/or disrupted; this may be achieved through cybersecurity vulnerabilities in either device or weak connection properties.

(d) A brute force attack, password cracking, packet sniffing, and/or phishing attacks can be performed by the attacker to gain access to the network and network packets.

4. CAN:

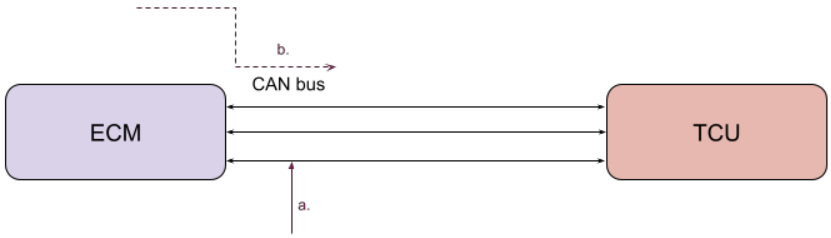


Figure 6.3: CAN bus Possible Attack Paths

- **Attack Path**: Figure 6.3: CAN bus Possible Attack Paths illustrates the different possible attack paths for the CAN bus

  (a) Physical access to the CAN bus can be obtained by the attacker, resulting in CAN sniffing or even tampering and manipulation of CAN packets.

  (b) Access to messages and data packets can be gained by the attacker through a compromised node in the CAN bus.

5. SWUpdate:

- **Attack Path**: The attack paths for the compromise of the SWUpdate software are described below:

  (a) Unauthorized access to the update server can occur due to compromised credentials or the exploitation of vulnerabilities in the system.

  (b) The update delivery system pipeline can be compromised due to weak network communication.

  (c) A backdoor can be injected into the update development pipeline through the exploitation of the development pipeline or source code repository.

6. Update Server:

- **Attack Path**: The attack paths for the compromise of the update server are described below:
  (a) Unauthorized access to the update server (Hawkbit) can occur through the exploitation of weak authentication or authorization mechanisms
  (b) A Man-in-the-Middle attack can be conducted via compromised HTTPS (Nginx) through the exploitation of insecure SSL/TLS configurations or certificate vulnerabilities.
  (c) Network disruptions between the host and server can lead to multiple potential attacks.
  (d) Vulnerabilities in Hawkbit's REST API used for update management and orchestration can be exploited.

7. Private signing key:

   - **Attack Path**:
     (a) Theft of private signing keys can occur through server compromise via social engineering or information disclosure.
     (b) A Man-in-the-Middle attack can be conducted during the key signing process.
     (c) Vulnerabilities in the key management system can be exploited.
     (d) Leakage can occur via insecure backup systems.

8. Public Verification Key:

   - **Attack Path**:
     (a) A Man-in-the-Middle attack can be conducted for public key substitution.
     (b) Weak public key distribution channels can be exploited through the use of insecure distribution methods.
     (c) A failure in the key revocation or expiration management process can be exploited, leading to key revocation issues.
     (d) Vulnerabilities in the key management system can be exploited.
     (e) Keys can be stolen from the PKI through server compromise via social engineering or information disclosure.

9. Backend Infrastructure:

   - **Attack Path**: The following attack paths for the backend infrastructure are identified.
     (a) Server-side vulnerabilities can be exploited.
     (b) Weak authentication or credential threats can be targeted.
     (c) A supply chain attack can occur due to the compromise of third-party services or software integrated into the backend infrastructure.
     (d) Database injection attacks can be executed.

6.1.5 Attack Feasibility Rating

Attack feasibility rating in ISO/SAE 21434 assesses how likely a threat can be successfully exploited. It considers factors like attacker expertise, target accessibility, tools required, and existing security controls. Ratings range from "low" to "high" feasibility, where a high rating means an attack is easy to execute, while a low rating indicates it requires advanced skills or resources. This helps prioritize mitigation efforts, focusing on the most likely and impactful threats to allocate resources effectively. Table 6.4 and 6.5: Attack Paths and Feasibility Ratings (appended at the end of chapter 6), defines the attack feasibility rating for the threat scenarios of the recognized assets.

6.1.6 Risk Value Determination

Risk value determination is a critical process in cybersecurity risk management, especially in the automotive industry, as outlined in standards like ISO/SAE 21434. It involves evaluating potential threats to an asset based on two primary factors: impact and likelihood. Impact refers to the consequences a successful attack might have on areas like safety, operational continuity, financial costs, and privacy. Likelihood assesses how probable it is that a threat can exploit a vulnerability, considering factors such as attack feasibility, threat actor capabilities, and system exposure. These two components are typically combined using a risk matrix, where higher impacts and greater likelihoods result in higher risk values. The risk value (Low, Medium, or High) guides the necessary treatment steps, such as implementing countermeasures to mitigate the identified threats. This methodical approach ensures that risks are managed proportionally to their potential severity and probability.

In Tables 6.1, 6.2, 6.3: Threat Scenarios for Different Assets, and the Attack Feasibility rating (appended at the end of chapter 6), the threat scenarios associated with different assets and their corresponding Attack Feasibility ratings are presented. The Impact Rating and likelihood are derived from these ratings, which together form a Risk Assessment Matrix. This matrix, as detailed in Table 6.6: Risk Assessment Matrix, is used as a reference for evaluating the risk associated with various threat scenarios.

Risk Assessment Matrix				
Impact \Feasibility	**Negligible**	**Moderate**	**Major**	**Severe**
Very Low	Low	Low	Low	Medium
Low	Low	Medium	Medium	Medium
Medium	Low	Medium	High	High
High	Medium	High	High	Critical

Table 6.6: Risk Assessment Matrix

The overall impact rating and Attack Feasibility ratings, which collectively assess the risk for the assets, are presented in Table 6.7: Risk Assessment Table with Multiple Threat Scenarios.

6.1.7 Risk Treatment Decision

In the context of ISO/SAE 21434, risk treatment decisions are essential for ensuring the cybersecurity of automotive systems. Organizations must conduct comprehensive risk assessments to identify potential threats to vehicle safety, functionality, and user privacy. After identifying risks, they should evaluate their severity and likelihood, deciding on appropriate treatment measures such as mitigation, transfer, retention, or avoidance.

The standard also emphasizes the need for continuous monitoring and review of these risk treatment strategies due to the evolving nature of technology and threats in the automotive sector. By proactively addressing risks, organizations can comply with ISO/SAE 21434, enhance their resilience to cyber threats, and maintain trust among stakeholders. This iterative approach protects sensitive data and ensures the integrity of automotive systems. The Risk Treatment Decision is made in context to the Risk Value determined in the above section. Table 6.8: Risk Assessment Table with Risk Treatment Decisions (appended at the end of chapter 6) presents the decisions of the respective threat scenarios. From the Table 6.8, it is evident that the priority for Risk Treatment is high for Spoofing, Tampering and is most cases of Elevation of Privilege in the defined Assets. As a result the following security goals and concepts are defined.

6.2 Security Goals

As defined by the results of the TARA, the following security goals can be defined to keep the system safe and secure.

1. The goal is to mitigate the risks of Man-in-the-middle attacks that can lead to version rollback attacks and other vulnerabilities in the OTA update process that may have been identified as critical during the threat analysis. The TARA highlighted risks associated with the acceptance of outdated, vulnerable software versions and the need for robust validation mechanisms to ensure the integrity and authenticity of updates.

2. The goal is to ensure that incremental OTA updates are applied correctly, particularly when only minor changes are made, without being misidentified or misreported as successful due to unnoticed changes in the update validation process. The primary goal is to secure the integrity and authenticity of every updated line, function or executable by using serial numbers and hash values, preventing tampering and false validation.

3. To prevent unauthorized access and spoofing attacks,the goal is to develop an algorithm that authenticates the user with the server before performing any major task. The server and the user application should be in sync and should be ideally integrated with a global time server and rolling passwords.

6.3 Security Concept

The security concept for the OTA update system should ideally integrate multiple layers of defense to protect against various attack vectors identified during the threat analysis and

TARA. The concept should focus on preventing unauthorized access, tampering, and rollback attacks while ensuring the integrity and authenticity of updates. The design should incorporate strong authentication mechanisms, cryptographic validations, and secure communication protocols.

The following chapter provides a detailed explanation of the security goals as developed algorithms in reference to the security concept and requirement for protection algorithms. addressing each of the three security goals defined earlier.

Assets and Scenarios		Attack Path	Elapsed Time	Specialist Expertise	Knowledge of this Item	Window of Opportunity	Equipment	Rating
Vehicle ECU	ECM	Weak Wi-Fi Network	≤ 1 week	Proficient	Confidential Information	Moderate	Specialized	Medium
		Unauthorized Access	≤ 1 month	Expert	Confidential Information	Difficult	Bespoke	Very Low
		Access to the CAN	≤ 1 month	Proficient	Confidential Information	Difficult	Specialized	Very Low
		Direct Access	≤ 1 month	Multiple Experts	Strictly Confidential Information	Difficult	Multiple Bespoke	Very Low
	TCU	Weak Wi-Fi Network	≤ 1 week	Proficient	Confidential Information	Moderate	Specialized	Medium
		Unauthorized Access	≤ 1 week	Expert	Confidential Information	Difficult	Specialized	Very Low
		Access to the CAN and/or Nodes	≤ 1 month	Expert	Confidential Information	Difficult	Specialized	Very Low
OTA Communication Protocols	Wi-Fi	Man-in-the-Middle	≤ 1 week	Expert	Restricted	Moderate	Specialized	Medium
		Router Communication	≤ 1 week	Expert	Restricted	Moderate	Bespoke	Medium
		Password Cracking/Brute Force	≤ 1 week	Proficient	Confidential Information	Moderate	Bespoke	Low
	CAN	Physical Access	≤ 1 month	Proficient	Restricted	Difficult	Specialized	Low
		Compromised Node	≤ 1 month	Proficient	Confidential Information	Moderate	Bespoke	Very Low
Update Verification Mechanisms	SWUpdate	Unauthorized Access	≤ 1 month	Expert	Confidential Information	Difficult	Bespoke	Very Low
		Compromise of delivery system pipeline	≤ 1 month	Expert	Confidential Information	Moderate	Bespoke	Very Low
		Backdoor Injection	≤ 1 month	Expert	Confidential Information	Moderate	Specialized	Very Low
	Update Server	Unauthorized Access	≤ 1 month	Proficient	Confidential Information	Difficult	Bespoke	Very Low
		Man-in-the-Middle	≤ 1 month	Expert	Confidential Information	Moderate	Specialized	Very Low
		Network Disruption	≤ 1 week	Proficient	Restricted	Moderate	Specialized	Medium
		Hawkbit REST API	≤ 1 month	Expert	Restricted	Difficult	Specialized	Low

Table 6.4: Attack Paths and Feasibility Ratings 1

		Attack Path	Attack Feasibility Rating	Specialist Expertise	Knowledge of this Item	Window of Opportunity	Equipment	Rating
Cryptographic Keys	Private Signing Keys	Theft	≤ 1 month	Expert	Confidential Information	Difficult	Specialized	Very Low
		Man-in-the-Middle	≤ 1 month	Proficient	Confidential Information	Moderate	Specialized	Very Low
		Exploitation of Key Management	≤ 1 month	Multiple Experts	Strictly Confidential Information	Difficult	Bespoke	Very Low
	Public Verification Keys	Man-in-the-Middle	≤ 1 month	Proficient	Confidential Information	Moderate	Specialized	Very Low
		Weak Public Key Distribution Channels	≤ 1 week	Proficient	Restricted	Moderate	Specialized	Medium
		Failure in the Key Revocation or Expiration	≤ 1 month	Expert	Confidential Information	Moderate	Specialized	Very Low
		Exploitation of Key Management	≤ 1 month	Multiple Experts	Strictly Confidential Information	Difficult	Bespoke	Very Low
		Theft	≤ 1 month	Expert	Confidential Information	Difficult	Specialized	Very Low
Backend Infrastructure		Server-Side Vulnerabilities	≤ 1 week	Proficient	Confidential Information	Moderate	Specialized	Low
		Weak Authentication	≤ 1 month	Expert	Confidential Information	Moderate	Bespoke	Very Low
		Supply Chain Attack	≤ 1 week	Proficient	Confidential Information	Moderate	Bespoke	Very Low
		Database Injection	≤ 1 week	Expert	Confidential Information	Moderate	Bespoke	Very Low

Table 6.5: Attack Paths and Feasibility Ratings 2

Assets	Threat Scenario	Impact Rating	Feasibility Rating	Risk Value
ECM	Spoofing	Major	Medium	High
	Tampering	Severe	Very low	Medium
	Repudiation	Major	Very low	Low
	Information Disclosure	Major	Medium	High
	DoS	Major	Very low	Low
	Elevation of privilege	Major	Very Low	Medium
TCU	Spoofing	Major	Medium	High
	Tampering	Severe	Medium	Medium
	Repudiation	Major	Very low	Medium
	Information Disclosure	Major	Medium	High
	DoS	Major	Medium	High
	Elevation of privilege	Severe	Very low	Medium
Wi-Fi	Spoofing	Major	Medium	High
	Tampering	Severe	Low	Medium
	Information Disclosure	Major	Low	Medium
	DoS	Major	Low	Medium
CAN	Tampering	Severe	Low	Medium
	Information Disclosure	Mojor	Low	Medium
	DoS	Major	Low	Medium
SWUpdate	Spoofing	Major	Very Low	Low
	Tampering	Severe	Very Low	Medium
	Repudiation	Moderate	Very Low	Low
	Information Disclosure	Severe	Very Low	Medium
	DoS	Major	Very Low	Low
	Elevation of privilege	Severe	Very Low	Medium
Update Server	Spoofing	Major	Low	Medium
	Tampering	Severe	Medium	High
	Repudiation	Moderate	Very Low	Low
	Information Disclosure	Severe	Medium	High
	DoS	Major	Low	Medium
	Elevation of privilege	Severe	Medium	High
Private Signing Keys	Spoofing	Severe	Low	Medium
	Tampering	Severe	Low	Medium
	Information Disclosure	Severe	Low	Medium
	DoS	Moderate	Low	Medium
	Elevation of privilege	Severe	Low	Medium
Public Verification Keys	Spoofing	Major	Low	Medium
	Tampering	Severe	Medium	Medium
	DoS	Moderate	Medium	Medium
Backend Infrastructure	Spoofing	Severe	Very Low	Medium
	Tampering	Severe	Very Low	Medium
	Repudiation	Moderate	Very Low	Low
	Information Disclosure	Major	Very Low	Low
	DoS	Major	Very Low	Low
	Elevation of privilege	Severe	Very Low	Medium

Table 6.7: Risk Assessment Table with Multiple Threat Scenarios

Assets	Threat Scenario	Risk Value	Risk Treatment Decision
ECM	Spoofing	High	Risk Reduction
	Tampering	Medium	Risk Reduction
	Repudiation	Low	Risk Retention
	Information Disclosure	High	Risk Avoidance
	DoS	Low	Risk Avoidance
	Elevation of privilege	Medium	Risk Avoidance
TCU	Spoofing	High	Risk Reduction
	Tampering	Medium	Risk Reduction
	Repudiation	Medium	Risk Avoidance
	Information Disclosure	High	Risk Avoidance
	DoS	High	Risk Transfer
	Elevation of privilege	Medium	Risk Avoidance
Wi-Fi	Spoofing	High	Risk Reduction
	Tampering	Medium	Risk Reduction
	Information Disclosure	Medium	Risk Transfer
	DoS	Medium	Risk Avoidance
CAN	Tampering	Medium	Risk Reduction
	Information Disclosure	Medium	Risk Retention
	DoS	Medium	Risk Avoidance
SWUpdate	Spoofing	Low	Risk Reduction
	Tampering	Medium	Risk Reduction
	Repudiation	Low	Risk Retention
	Information Disclosure	Medium	Risk Avoidance
	DoS	Low	Risk Retention
	Elevation of privilege	Medium	Risk Reduction
Update Server	Spoofing	Medium	Risk Reduction
	Tampering	High	Risk Reduction
	Repudiation	Low	Risk Retention
	Information Disclosure	High	Risk Transfer
	DoS	Medium	Risk Transfer
	Elevation of privilege	High	Risk Reduction
Private Signing Keys	Spoofing	Medium	Risk Reduction
	Tampering	Medium	Risk Reduction
	Information Disclosure	Medium	Risk Reduction
	DoS	Medium	Risk Avoidance
	Elevation of privilege	Medium	Risk Avoidance
Public Verification Keys	Spoofing	Medium	Risk Reduction
	Tampering	Medium	Risk Reduction
	DoS	Medium	Risk Avoidance
Backend Infrastructure	Spoofing	Medium	Risk Reduction
	Tampering	Medium	Risk Reduction
	Repudiation	Low	Risk Transfer
	Information Disclosure	Low	Risk Reduction
	DoS	Low	Risk Reduction
	Elevation of privilege	Medium	Risk Avoidance

Table 6.8: Risk Assessment Table with Risk Treatment Decisions

Chapter 7

Proposal of Security Algorithms

In the previous chapter, the TARA of the system identified three critical security goals to enhance the overall security posture of the OTA update process. These goals aimed to mitigate the risks associated with man-in-the-middle attacks, ensure the correct application of incremental updates, and enhance authentication and authorization mechanisms to prevent unauthorized access. In this chapter, three corresponding algorithms are proposed, each designed to address one of the defined security goals. These algorithms will provide a structured approach to implementing robust security measures, ensuring the integrity, authenticity, and overall security of the OTA update mechanism in the automotive environment.

7.1 Security Algorithms

Here there would be a recommendation of different (3 security algorithms) that can be used in context of Security goals and concepts derived from Chapter 6: TARA:

1. Feature check algorithm
2. Update Serial Number
3. One-Time-Password (OTP) Based Authentication with Rolling OTP Authentication

7.1.1 Algorithm 1: Feature Validation & Update Verification

Problem Identification: During version rollback attacks, in several different forms, the attacker tries to trick the user or the ECUs in the Automotive Vehicle into accepting older versions or executable, as a result reintroducing the vulnerabilities in the automotive system. This exploitation by this method usually goes undetected by the user and system [37] [38], and when the attack is detected, it is already too late. To prevent this, this algorithm is proposed.

Algorithm Design

- Pre-Update: While making of the configuration files and the image files along with other executable files for the OTA update package, there must also be a file that defines the new features of the update (assuming they are feature addition/improvement updates). This file must have a list of features that the currently deployed version have compared to the previous one and also, a list of features of the new update version.

- Triggering Update: Once an update file is detected, to make sure that the target is secure from any version rollback attacks, after the package is downloaded, before installation, there is a randomized feature test, that makes sure that the current version is the actual version, to check if there have been any damages and exploitation in the system before the update.

  – Once the current version is verified, the newer update version is installed.

- Post-Installation: Random features from the newly iterated features of the update, are run in background. Eg. If there are 5 features updates including Fuel Efficiency optimization in the ECM, Volume sensitivity in the Head Unit, Tire pressure monitoring system optimization etc., out of these 5. randomly 2 or 3 features will be selected by the algorithm and run in the background with any test data/packet. The input and output would be logged in the log files, and once the input/output data has been verified with server expectations, the system will be successfully involved.

Algorithm Requisites

1. **Internal Requirements (within device)**: U-boot Bootloader; A/B type Partition Memory; Basic signing and verification feature; Authentication; Logger system; Manifest files, Config files, Image files and feature list files.

2. **External Integration (Environmental Support)**: SWUpdate software for creation of update package (Update Manager); Validation in external or OEM cloud; OTA integrity and authentication protocols.

Data Flow Diagram

Figure 7.1 defines the data flow of the proposed algorithm. There are 4 stages namely, Pre-update, Post-Update, Processes (in TCU), Proposed Algorithm Implementation. First the update packet is generated, with a config file, Image file, Supporting Scripts and an additional list that is uploaded directly on the server and not the update file itself. Once the .swu update package is recognized as an update on the server, the update file is downloaded and unpacked in a /temp directory for the ECU. After unpacking, our first layer of the cybersecurity layers runs a version test to check if the current version is the last updated version or if there has been a rollback attack that went undetected. This prevents any manipulation in the system further, when a new update is installed. These logs are stored in a log file, then the system proceeds to install the update file. Once the update is installed, another feature check algorithm runs to verify the installation of V2.1, which is also logged in a log file. Before the user can be given access to the system, the log file is compared with

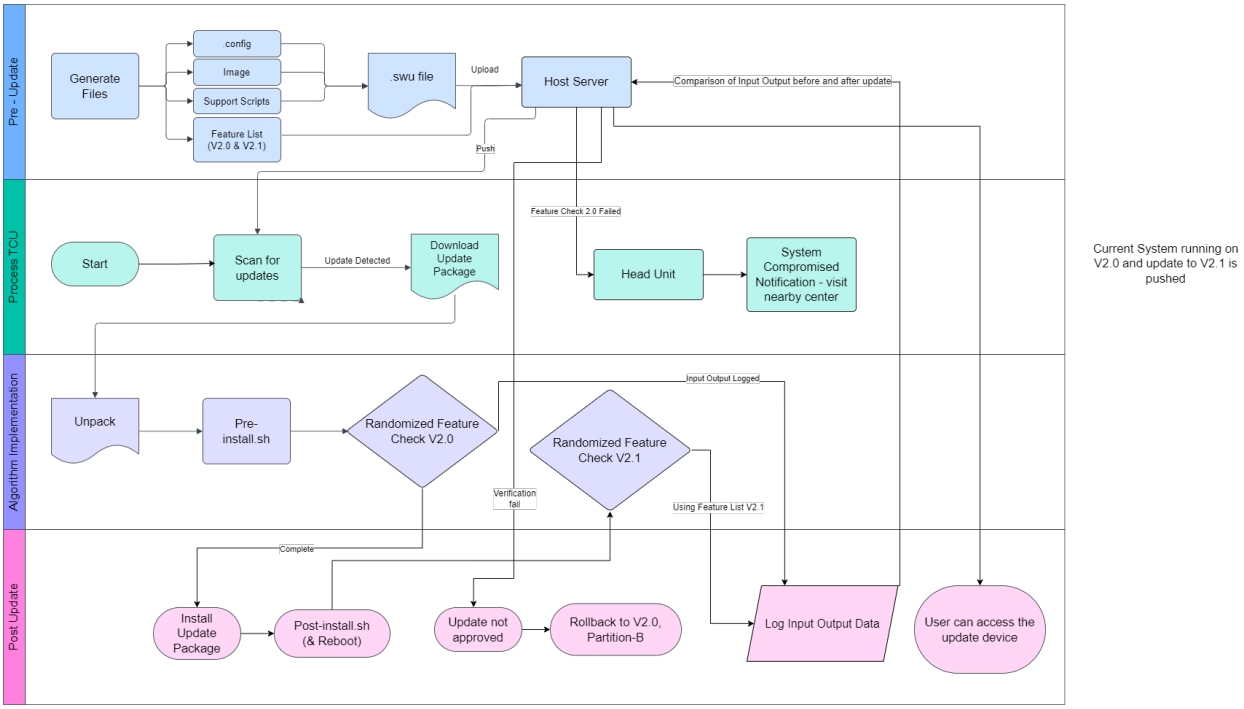


Figure 7.1: Algorithm 1: Data Flow Diagram

the feature list of expected outputs and inputs, on the host server, if the verification checks out, the user is given the access, else, the system rolls-back the system to the second partition.

Algorithm Structure

Update Package Structure: As per our application in the previous section of SWUpdate, the figure 7.2 will be the file structure of the OTA update package.

Following is a basic structure of the update file and update process integrating the algorithm using an example:

System Example

Let's assume that an Automotive vehicle has digital car dashboard unit which has all the informative panels like Speedometer, odometer etc. The OEM wants to add another pannel to define vehicle efficiency. Let E represent the efficiency.
Let F represent the fuel injected (e.g., in liters).
Let t represent the time of travel (e.g., in hours).
Let d represent the distance traveled (e.g., in kilometers).

```
E = f(F,t,d)
```

As per the proposed Algorithm, the Feature Select script should run in such a way, that since the OEM is aware of the precise Efficiency Equation for the display, a Feature Val2.1 file should be created, which maps pre defined simulation like values and their expected outputs. When the log file is sent for verification to the host server, the inputs and outputs are compared against the tested and resulted values and accordingly the update is verified. Using this background information we create the update package. It has the following Structure.

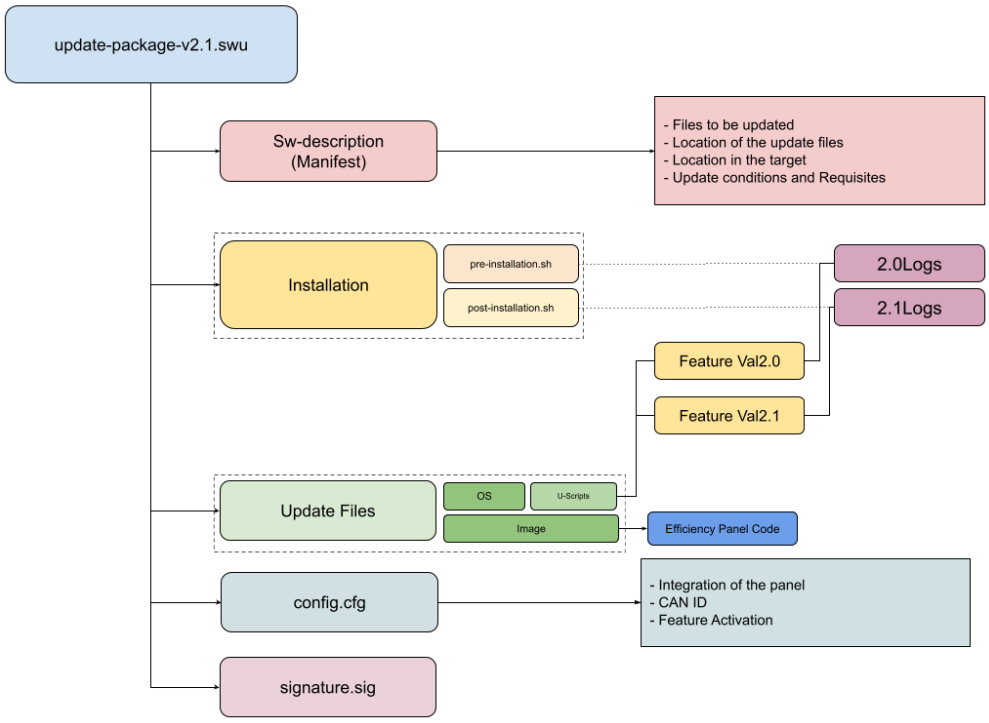


Figure 7.2: SWUpdate .swu content

Assuming the update is ready and prepared in the host as shown in Figure 7.2, the Figure 7.3 represents a graphical representation of the Working Architecture of this Algorithm, the steps are described in detail below:

1. The system keeps scanning for an update on a regular basis & Step 1.1 shows that the cloud pushes an update.

2. The update is downloaded on the device in the Downloads folder and then the package is unpacked in background using SWUpdate application.

3. It unpacks them in a /temp directory after sign verification and hash value for integrity.

4. By default the pre-install.sh is executed which triggers the system, and all pre-install checks are performed

5. The Feature Validation file for Version 2.0 in the current partition which is one of the scripts in .swu>Update Files>U-Scripts>Feature Test 2.0 is executed as a result of pre-install.sh

6. The input and out are then stored in a log file generated as a result of pre-install.sh

7. The image is then installed from the Update files to the other partition B (Dual memory system) & .configs is replaced or appended in the Rootfs (Depending on the application).

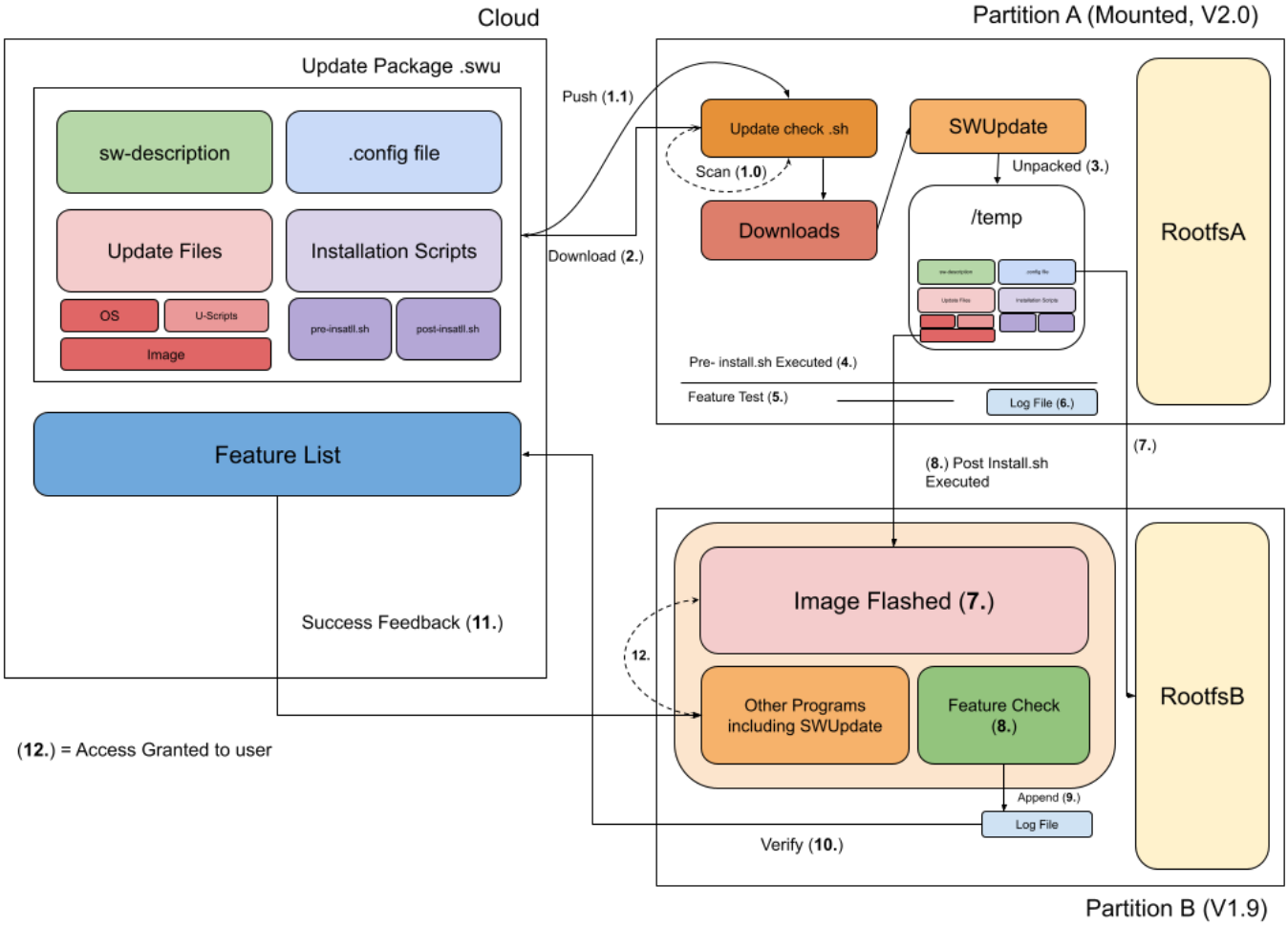


Figure 7.3: Algorithm 1: Architecture

8. The post-install.sh is then executed and the system is rebooted. As per the post-install.sh, it boots and does a Feature Check by executing a feature check script also present at .swu>Update Files>U-Scripts>Feature Test 2.1.

9. The input output data is then logged

10. The log file is sent to the cloud server for validation.

11. Once it verifies the logs with the expected Feature Lists on the cloud it send a update success feedback to the vehicle.

12. The user is allowed to access the new updated partition, in this case, Partition B, if the verifciation fails, it would rollbak to the previous verison in partition A.

Algorithm Pseudocode

```
# Algorithm to perform secure OTA updates with version verification and randomized feature tests

def ota_update_process(current_version, new_version, feature_log_file, server_url):
    # Step 1: Pre-Update - Check current version and feature list
    pre_update_features = get_feature_list(current_version)  # Fetch current feature list
    new_update_features = get_feature_list(new_version)      # Fetch new feature list

    update_package = download_update_package(new_version)    # Download the OTA update package

    if not verify_package_signature(update_package):
        raise Exception("Update package signature verification failed. Aborting update.")

    # Step 2: Trigger Update - Verify current version with randomized feature tests
    if not randomized_feature_test(pre_update_features):
        raise Exception("Current version randomized feature test failed. System may be compromised.")

    # Step 3: Install the new update version
    install_update(update_package)

    # Step 4: Post-Installation - Randomized feature test on new features
    post_install_features = select_random_features(
        new_update_features, count=3)

    for feature in post_install_features:
        test_data = generate_test_data(feature)  # Generate test input
        output_data = run_feature_test(feature, test_data)  # Run test and get output
        log_test_data(feature, test_data, output_data)  # Log the test input/output

    # Step 5: Verify logs with server expectations
    log_file = generate_log_file(post_install_features)
    if not verify_logs_with_server(log_file, server_url):
        raise Exception("Post-installation log verification failed with server. Rolling back update.")

    # Step 6: Mark update as successful
```

```
    mark_update_successful(new_version)
    return "Update successfully installed and verified."

# Helper functions
def get_feature_list(version):
    """Fetch the feature list for the given version."""
    # Read version metadata or configuration file
    return feature_list

def download_update_package(version):
    """Download the OTA update package for the specified version."""
    # Download logic here
    return package

def verify_package_signature(package):
    """Verify the cryptographic signature of the downloaded package."""
    # Signature verification logic
    return True

def randomized_feature_test(features):
    """Run random tests on current features to verify version integrity."""
    # Select random features and test
    return True

def install_update(package):
    """Install the downloaded update package."""
    # Installation logic here
    pass

def select_random_features(feature_list, count=3):
    """Randomly select features from the new update for post-installation testing."""
    import random
    return random.sample(feature_list, count)

def generate_test_data(feature):
    """Generate test data based on the feature."""
    # Create input for the feature test
    return test_data

def run_feature_test(feature, test_data):
    """Run the selected feature with the generated test data and return the output."""
    # Run the feature test
    return output_data
```

```
77
78 def log_test_data(feature, test_data, output_data):
79     """Log the test input and output for server verification."""
80     # Append to a log file or in-memory structure
81     pass
82
83 def generate_log_file(test_features):
84     """Generate a log file from the test results."""
85     # Collect logs from test data
86     return log_file
87
88 def verify_logs_with_server(log_file, server_url):
89     """Send logs to the server and verify against expected output."""
90     # Send logs to server for verification
91     return True
92
93 def mark_update_successful(version):
94     """Mark the update as successful after verification."""
95     # Update version tracker
96     pass
```

State-of-the-art Comparison

Through Table 7.1 it is clear, it is evident that feature validation is utilized by different OEMs at varying scales, depending on the specific application type. Additionally, feature validation is employed in diverse scales across various OEMs. The novelty of this algorithm is as follows:

1. The defined randomized check executable that can select the feature to verify.
2. A Separate log file is generated for the input of the feature test result and output
3. Post installation verification for update and appending the output to this log file.
4. Sending this log file to cloud to verify the update installation

7.1.2 Algorithm 2: Serialising Update script lines/functions

Problem identification: In the case of OTA updates that involve minor fixes or enhancements, such as the update from version V2.0 to V2.1 for the camera module—which only adjusts the ISO (light capturing ability)—the calibration performed by the script-based executable update does not constitute a complete feature. Therefore, an in-depth analysis of the script is necessary. To address this, an algorithm can be developed that assigns serial numbers to individual lines of the update script. For larger update files, serial numbers can also be assigned to specific lines of code and/or functions.

Algorithm Design:

Automaker	Pre-Update Checks	Update Process	Post-Update Diagnostics	Sources
Tesla	Battery level, network connectivity	Background download and installation, integration of new features	System integrity checks, functionality validation	[39], [40]
Volkswagen	Simulation environments, Hardware-in-the-loop (HIL) setups	Validation of Advanced Driver Assistance Systems (ADAS) and Autonomous Vehicle (AV) systems under various conditions	Comprehensive diagnostics to ensure correct functionality	[41], [42]
Toyota	Advanced diagnostic algorithms, anomaly detection	Checks on sensors, actuators, and control units	Comprehensive performance checks before, during, and after updates	[43], [44]
General Motors	Extensive system checks for compatibility	Background update process, VIP and Ultifi platforms	Pre-update and post-update validation tests	[45], [46]
BMW	Advanced diagnostic algorithms	Real-time diagnostics during update, pre-update checks	Post-update validation to verify new features and functions	[47], [48]

Table 7.1: OTA Update Processes and Diagnostic Approaches of Various Automakers

1. Pre-update: When creating the delta update, assign unique serial numbers to all new lines of code. This should be part of the update package that is sent to the device. Also create a manifest file that contains a list of these serial numbers along with their corresponding hash values. This manifest is included in the update package.

2. Update Process: On the target device, apply the delta update by inserting the new lines of code into the existing codebase according to the serial numbers.

3. Post Update: After the update is applied, use the serial numbers and their hash values from the manifest to verify that the new lines of code have been correctly integrated. This involves recalculating the hash values of the updated lines and comparing them with the values in the manifest.

Algorithm Requisites

1. **Internal Requirements (within device)**: U-boot Bootloader; A/B type Partition Memory; Basic signing and verification feature; Authentication; Delta type update; Logger system.

2. **External Integrations (Environmental Support)**: SWUpdate software for creation of update package (Update Manager); Validation in external or OEM cloud; OTA integrity and authentication protocols.

Data Flow Diagram

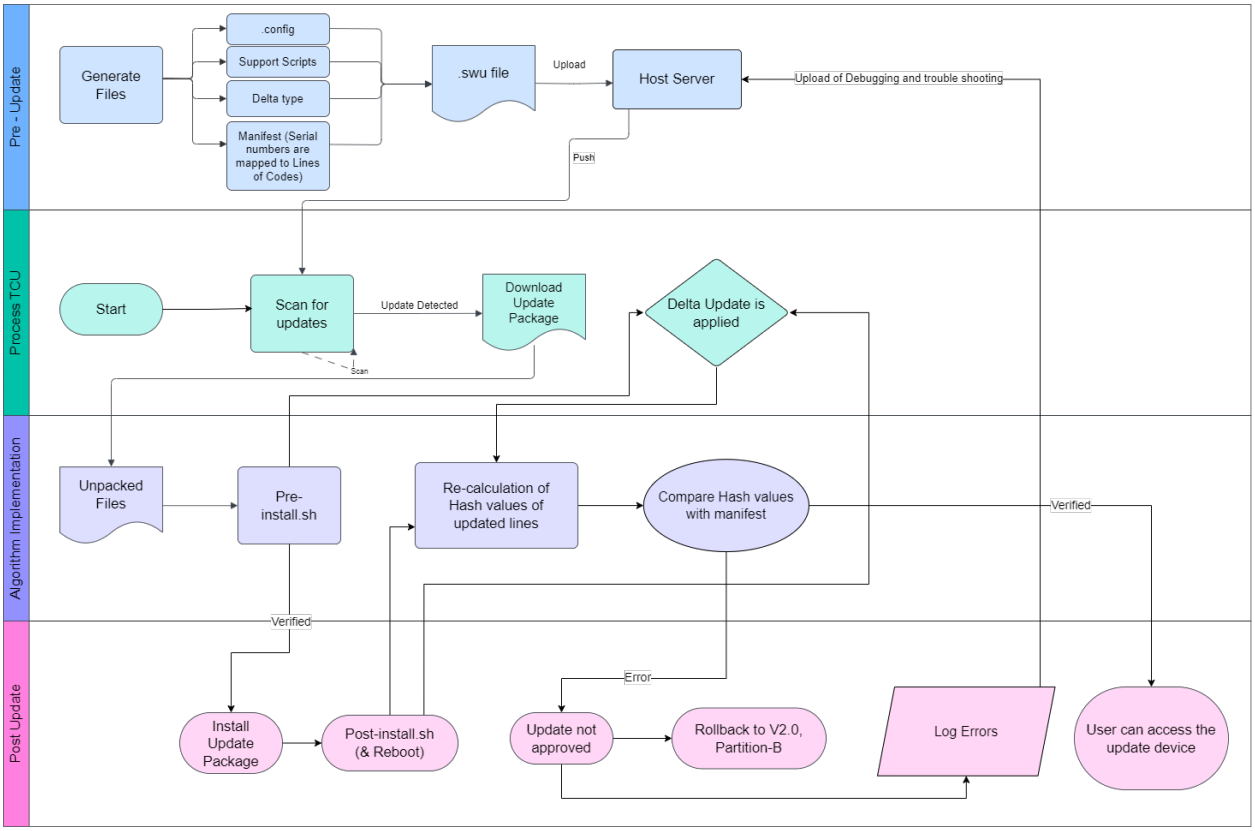


Figure 7.4: Algorithm 2 Data Flow Diagram

Figure 7.4 defines data flow of the proposed Algorithm 2. The stages are same as the previous Algorithm as it takes place in the same part of the OTA update process. In the first phase, the .swu file is generated, with .config file, Manifest file/sw-description, Delta update file and other supporting scripts. The manifest file here contains a mapping of Serial Number: Hash of lines in scripts/functions. Subsequently, it is uploaded on the host server. The system is already started in the vehicle, and it is scanning for the update continuously in the host server. The update is detected and then downloaded, it is then unpacked, and the pre-install.sh is executed and then the update is installed. The system then reboots and now scans the newly updated scripts for the new codes. Once the codes are detected, their hashes are calculated. These hashes are then compared against their serial numbers in the Manifest file, if the verification is successful, then acess is granted to the user.

Algorithm Structure: This algorithm structure is based on incremental updates like Delta types, which aims at verifying script and application level updates.

Update Package Structure: During the building of the update package the script lines should be mapped to a serial number.

```
Serial Number  |  New Code Line
-------------- |  ----------------
001            |  new_line1
002            |  new_line2
003            |  new_line3
...
```

```python
import hashlib

def compute_hash(code_line):
    return hashlib.sha256(code_line.encode()).hexdigest()

def verify_update(applied_code, manifest):
    for serial, code_line in applied_code.items():
        computed_hash = compute_hash(code_line)
        if manifest[serial] != computed_hash:
            return False, serial
    return True, None

# Example applied code after update
applied_code = {
    "001": "new_line1",
    "002": "new_line2",
    "003": "new_line3"
}

# Example manifest file received with update
manifest = {
    "001": "hash_of_new_line1",
    "002": "hash_of_new_line2",
    "003": "hash_of_new_line3"
}

# Perform verification
is_verified, error_serial = verify_update(applied_code, manifest)
if is_verified:
    print("Update verified successfully.")
else:
    print(f"Update verification failed at serial number: {
       error_serial}")
```

The manifest file, should have a section mapping these serial numbers to the hash of each of the script lines.

Once the update is applied and downloaded, the post-installation script, performs a verification using a pre defined script.

State-of-the-art Comparison

OEM	Tool(s) Used	Application	Compliance Standards
Volkswagen Group	Parasoft	Static analysis, unit testing, code coverage tools integrated into CI/CD pipelines	ISO 26262, ISO 21434
Daimler	Parasoft	TÜV SÜD certified static analysis for safety and security during development	ISO 26262, TÜV SÜD ASIL
Ficosa International	Polyspace (MathWorks)	Static analysis integrated with CI practices for ISO 26262 and ISO/SAE 21434 compliance	ISO 26262, ISO/SAE 21434

Table 7.2: Automotive OEMs Using Static Code Analysis Tools

Table 7.3 and Table 7.2 are the comparative anaylsis of different softwares and services that are used for code analysis applications. Table 7.2 does a comparative analysis of the OEMs that use a similar kind of algorithm for scanning code in context of compliance standards like ISO 26262.
Table 7.3 gives a detailed comparison of different categories of model based code analysis softwares that are currently used in the industry and it also compares if these mentioned to be compatible with Automotive ECUs.

- Static Code Analysis: These tools analyze source code without executing it to identify potential errors, coding standard violations, and security vulnerabilities.
- Model-Based Development: This category deals with tools that allow developers to create a system model (usually using MATLAB/Simulink) and then automatically generate code from the model.
- Automated Testing: Automated testing tools execute tests on software to validate its behavior against expected results.

Table 7.3: Automotive Software Development Tools

Category	**Tool Name**	**Description**	**Compliance**	**Automotive ECU Compatible**
Static Code Analysis	Polyspace	Analyzes runtime errors such as division by zero, buffer overflows.	MISRA, ISO 26262	Yes
	QAC	Checks compliance for C/C++ code focusing on MISRA standards.	MISRA	Yes
	PC-lint/FlexeLint	Detects bugs, coding violations, and complexity issues in C/C++ code.	Customizable rules	Yes
	Klocwork	Identifies security vulnerabilities, safety issues, and coding standard compliance.	MISRA, ISO 26262, Security Standards	Yes
Model-Based Development	Simulink Code Inspector	Verifies generated code against model behavior.	MISRA, ISO 26262	Yes
	TargetLink	Generates code from MATLAB/Simulink models with verification and testing capabilities.	Supports integration with testing frameworks	Yes
Automated Testing	VectorCAST	Tests embedded software, including unit, integration, and system testing.	Supports ISO 26262	Yes
	Parasoft C/C++test	Static analysis and testing for C/C++ code with unit testing and code coverage.	MISRA, ISO 26262	Yes
Code Coverage	GCOV	Analyzes code coverage using GCC.	Statement, branch coverage	Configurable
	LDRA Testbed	Provides static and dynamic code analysis with detailed code coverage reporting.	MISRA, ISO 26262, DO-178B/C	Yes
IDE Plugins	IAR, Keil IDE Plugins	Code analysis and checking within automotive development environments.	MISRA, ISO 26262	Yes
Scripting Tools	Pylint	Checks code quality and adherence to standards in Python scripts.	PEP 8 Compliance	No
	ShellCheck	Analyzes shell scripts for syntax and style errors.	Customizable rules	Configurable

- Code Coverage: These tools measure how much of the software's code has been executed during testing, ensuring that critical portions of the software are adequately tested.
- Integrated Development Environment (IDE) plugins provide additional functionalities like static code analysis and compliance checking directly within the coding environment.
- Scripting Tools: These tools analyze scripting languages (like Python or Shell) for syntax errors, style issues, and adherence to coding standards.

From the above tables, it is clear that in the industry, there are external services that provide script code analysis in different model categories, the following is the novelty of the Algorithm 2:

1. Integration of serial numbers mapped to manifest files and configured in similar services like Polyspace, QAC etc. to be verified upon any OTA Update installation.
2. Configuring these 3rd party services not just to verify the integrity, syntax and other features of the script but also to verify encoded serial numbers mapped to the hashed codes.
3. Integration of these services in OTA update process.

7.1.3 Algorithm 3: OTP Based Authentication with Rolling OTP Authentication

Problem Identification: During the OTA Update mechanism in an automotive vehicle, there are chances that due to weak identification mechanisms [49], the attacker can try to spoof OTA updates authentication mechanism, or even get access to local storage of the vehicular system and modify/manipulate the anti-rollback mechnainism and not just that, but in this case it can be quite harmful to all the critical systems of architecture.

Algorithm Design:

1. Pre-Update: The host server and the target, both are linked to a time server to stay in sync. Anytime an update is triggered from the host PC to the server, the server waits for a authentication OTP linked to application/services like Microsoft Authenticator or similar company tailored OTP generation servers. This access can be given to the user during the time of purchase of the car. When the update it triggered, it can ask the user for the OTP using the Head Unit of the vehicle. Also, this OTP can be used if the user wants to access privileged features of the car, this would prevent exploitation by local access to the storage.
2. Triggering Update: When the Update is triggered from the host device onto the server, the Vehicle TCU scans the server for update and asks the user for input on if the user would like to update. If the user chooses to update the device, the Head Unit would

then ask for the OTP in the application responsible for the authentication code. If the authentication matches, the device can download the update.

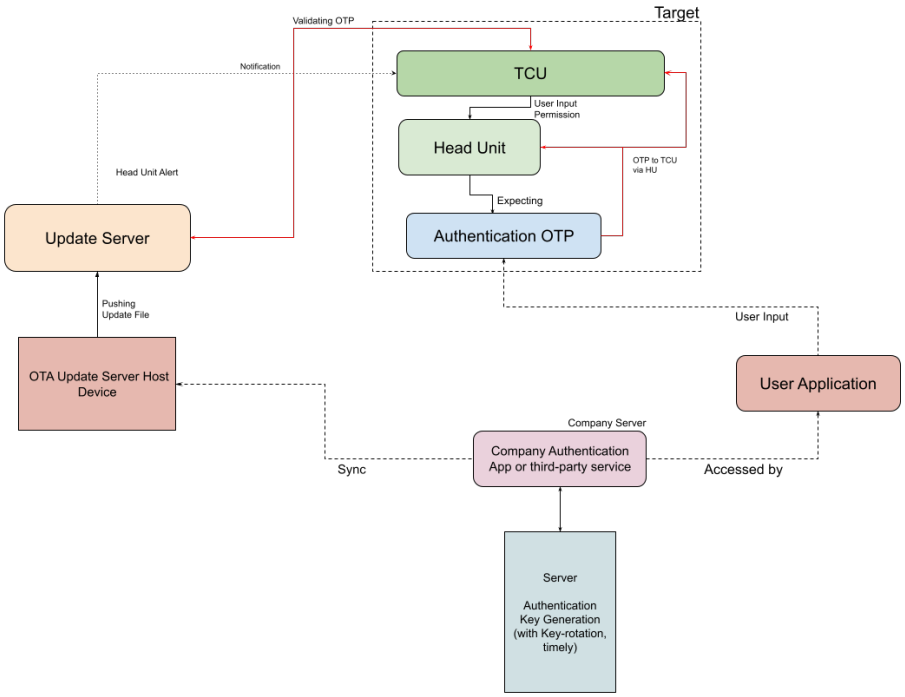


Figure 7.5: Authentication Work Flow

Algorithm Structure

The structure of this algorithm is not a very difficult one, but it makes the system highly secure and resistant to authentication compromising attacks. The authentication proposed in this method is meant to prevent any authentication compromising or spoofing attacks for the communication of the OTA update procedure.

The Figure 7.5 shows the proposed authentication technique. As shown in the figure, the backend update server and the user are given access to an authentication service, can be a company's own development or an authenticator service. Once an update is trigger and available in the server, the Head Unit will ask for the user for permission to carry out the update, if the user prompts yes, he must provide an OTP code, available in the authentication app (access give to the user at the time of purchase and delivery). The system validates the code with the server, if the validation is successful, the update will be carried out.

Algorithm Requisites

1. **Internal Requirements (within device)**: Head Unit; TCU; Authentication; Update Management Software; Logger system.

2. **External Integration (Environment Support)**: SWUpdate software for creation of update package (Update Manager); Global Time Server; Authentication service (Self

Deployed or third party).

Sequence Diagram

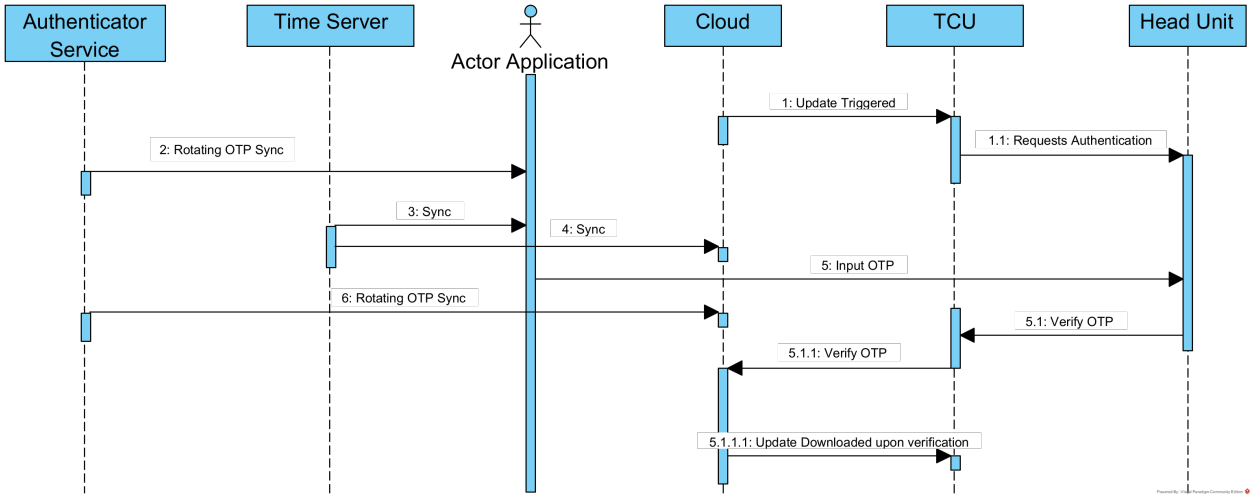


Figure 7.6: Authentication Sequence Diagram

State-of-the-art

The following are the state-of-the-art technologies in context to a similar use case of the proposed Algorithm 3. The Novel features of this Algorithm as compared to state-of-the-art existing solutions are:

1. Use of Rolling OTP Mechanism.
2. The algorithm requires users to enter an OTP via the vehicle's Head Unit each time an OTA update is triggered.
3. Integration with Authentication Services.

OEM/Company	**Technology/System**	**Details**
Bosch	Secure Gateway (SGW)	Uses AutoAuth for secure diagnostics/updates. No explicit mention of rolling OTP.
Remote Keyless Entry	Rolling Code System	Generates new codes per use, preventing replay attacks. Similar to rolling OTP.
Mender	Open Source OTA Platform	Focuses on secure OTA with phased rollouts, custom security. Could integrate OTP.
Uptane	OTA Security Framework	Secures OTA with multi-layer defenses. OTP not mentioned but possible.
Renesas	Compliance with ISO 24089 and UN-ECE WP.29 R156	Follows regulations for secure OTA. Could integrate OTP for additional security.
BMW, Ford, Volkswagen, Volvo	Secure OTA Systems	Extensively use OTA for system updates. Follow similar protocols to Tesla, but exact mechanisms (e.g., rolling OTPs) not publicly detailed.

Table 7.4: Comparison of Automotive Security Mechanisms with OTP and Rolling Code Systems

Chapter 8

Research Contributions

For the fulfilment of the goals mentioned in Section 1.3: Methodology, the following tasks were accomplished

1. Analysis of State-of-the-art and existing OTA update architectures, working design of update mechanism and ECUs used in an automotive industry along with the problems/shortcomings, Chapter 3 does a comparative analysis of the following:
    - Existing Researches based on Automotive OTA update in context of cybersecurity (Table 3.1: Overview of existing researches on Automotive OTA updates and its cyber-security properties).
    - Hardware Analysis of existing prototypes and devices for OTA update simulation. (Table 3.2: Device Simulation Options and Considerations, Table 3.3: Backend Simulation Options and Considerations)
    - OTA Architectures used in the Automotive Industry (Table 3.4: Summary of Software Update Architectures)
    - Reference Implementation Architectures (Table 3.5: In-detail comparison with shortlisted architectures, Table 3.6: Reference Architectures Used in Automotive Industry)
    - Analysis of different secure servers that can be used for secure OTA updates (Table 3.6: Comparison of Uptane, RAUC, and SWUpdate architectures)
    - Optimization techniques for an OTA update architecture (Table 3.8: OTA Optimization)
    - IDS used in Automotive industries and the ones appropriate for prototyping (Table 3.9: IDS used in Automotive Industry, Table 3.10: Comparison between Snort and suricata)
2. A working prototype of the Automotive OTA Architecture:
    (a) Defined components along with reason for use for the prototype
    (b) Integration of industry level OTA Update Architecture: SWUpdate
    (c) A/B Partition for robust update mechanism

(d) Use of U-Boot Bootloader for Dual boot system

(e) Setting up of secure server like NGINX and Hawkbit for update management

(f) Installation of IDS in the ECUs.

3. Evaluation and Verification of the developed prototype: This is done in the chapter 6. The TARA based evaluation provides the vulnerabilities and hence, required security goals and concept (6.1). The TARA methodology in context to ISO21434 included the following:

   - Asset Identification in the prototype.
   - STRIDE based Threat Scenarios for each of the assets.
   - Attack Path Analysis for each of the identified Asset.
   - Attack Feasibility rating for each of the identified attack paths.
   - Risk Value Determination on the basis of impact rating, attack feasibility and risk assessment matrix.
   - Risk Treatment Decision based on risk value.
   - Security Goals and security concept definition on the basis of Risk Value and Risk Treatment determination.

   As the result of Security Goals and defined Security Concept, three Algorithms are proposed.

4. A Novel solution for prevention of Rollback attacks in order to replace existing methodologies or significant improvement in existing methodologies: In chapter (6.1) the algorithms proposed are a result of in-depth research about the functioning of rollback attacks and understanding the OTA update process.

   (a) Feature Validation & Update Verification

   (b) Serialising Update script lines/functions

   (c) OTP Based Authentication with Rolling OTP Authentication

   These solutions are theoretically defined with complete Algorithm Design, Algorithm Architecture, Algorithm Requisites, Data Flow Diagram and in-detail Description with a System Example, Algorithm Pseudocode and a comparison with the state-of-the-art techniques. They can be further implemented on the developed prototype as proposed in Chapter 10: Future Work.

Chapter 9

Conclusion and Summary

The following chapter gives the chapter-wise conclusion and the summary of the document.

9.1 Conclusion of the Research

This thesis addressed the critical challenges in automotive OTA updates with a particular focus on cybersecurity. By conducting an extensive analysis of state-of-the-art solutions and their limitations, an Automotive OTA update prototype is defined and implemented, TARA is performed and cybersecurity vulnerabilities are identified, as a result of which 3 algorithms are proposed.

As a part of the thesis, Chapter 1: Background information and Introduction provides an in-detail description on Automotive vehicles, In-vehicle architectures and components involved, cybersecurity properties, OTA updates, Rollback Attacks, their implementation and consequences, along with the general structure and methodology followed in the thesis.

Via Chapter 2: Literature Review, a comparative analysis of state-of-the-art techniques of cybersecurity is provided in this document. Also certain case studies and incident reports of version rollback attacks that have happened in the last decade.

Chapter 3: Research and Analysis does a comparative analysis of existing OTA Architectures, device simulation options, and secure servers provided a solid foundation for the design and validation of more secure OTA systems and IDS, particularly the comparison between different IDS used on the industry for prototyping. By carefully observing the various pros, cons, capabilities, and comparisons highlighted in this chapter, the insights gained formed the basis for the implementation of the prototype in the subsequent chapter.

As mentioned above, Chapter 4 is defined as the direct outcome of the Chapter 3. Based on the observations in the above chapter, this chapter proposes the system architecture of an automotive OTA Update architecture including the in-vehicle component prototypes, cloud and communication and backend components and devices along with the different features and functionality used from these to create a prototype.

Chapter 5: Implementation, implements the proposed system components and features from chapter 4. This chapter defines the development environment, including the components used for prototype, how they are implemented, what features are used in these components to simulate a real life smart vehicle architecture. Since, this paper uses general microcontrollers

like Raspberry Pis to implement the prototype, this chapter also provides solutions to several errors code that came up during the implementation of different features like SWUpdate to facilitate the OTA architecture.

Once, the architecture is proposed and implemented, Chapter 6 evaluates this implemented prototype based on ISO21434. TARA is performed as per Clause 15. of ISO21434. Assets are identified, threat scenarios are defined, an impact rating is done along with the Attack Path analysis and attack feasibility rating, Risk value determination and Risk treatment decision.

As a result, Chapter 7: Proposal of Security Algorithms, provides 3 different security algorithms keeping in mind the different threat scenarios of the identified assets. They range from Major updates cybersecurity like for feature addition, minor update bug fixes like changing camera calibration and an authentication technique that can be implemented to protect the system overall.

9.2 Summary

This thesis addresses critical challenges in automotive OTA updates, emphasizing cybersecurity enhancements. It identifies significant vulnerabilities in developed Automotive OTA update prototype, OTA infrastructure security, and update process optimization. The research is structured into several chapters, beginning with an overview of automotive vehicles, in-vehicle architectures, cybersecurity, update process and mechanism and the consequences of rollback attacks. A literature review presents a comparative analysis of existing cybersecurity techniques and case studies of past rollback attacks.

The research further evaluates existing OTA architectures, device simulation options, and intrusion detection systems, forming a solid foundation for the developed prototype. A secure OTA update architecture is designed, detailing in-vehicle components, cloud integration, and backend devices essential for implementation. The development of the prototype is discussed, focusing on using general controllers like Raspberry Pis, addressing implementation challenges, and leveraging tools like SWUpdate.

The prototype is evaluated based on ISO 21434 standards through a TARA, which highlights vulnerabilities and establishes necessary security goals. The thesis proposes innovative algorithms to prevent rollback attacks, including feature validation, update verification, and OTP-based authentication with rolling OTPs. Ultimately, the research culminates in a functional prototype that incorporates industry-standard OTA update practices, robust update mechanisms, and secure server implementations. This work contributes valuable advancements in automotive cybersecurity, particularly in enhancing the security of OTA updates and vehicle software management.

Chapter 10

Future Work

There are research topics that come with this project in terms of the broad scope and vastness of the current topic. There can be multiple projects that can be worked upon in different focus areas contributing to one single goal of the cyber-security of OTA updates of Automotive vehicles. The topics are as follows:

1. In-depth application of RAUC and/or SWUpdate to configure these secure open source platforms with more features, cyber-security and improved efficiency.
2. Addition of more features in RAUC and/or SWUpdate and layers in the open-source code of implementation architectures.
3. Use of more hardware peripherals for the system to mimic real sensors and actuators in place of software console for the data generation.
4. As a result of this thesis there are three algorithms proposed to solve the issues of version validation in Automotive vehicle. All the three algorithms can be implemented on the designed Automotive OTA update prototype.

Alternative implementation strategies for the development of the prototype include:

1. Complete Software based simulation of Simulink/MATLAB using AutoSar Libraries. It requires certain licenses and can be developed as more exact model of an automotive architecture by self designing of ECUs, Communication peripherals and protocols etc.
2. Application of industry standards like Uptane for cyber-security.
3. Focus on In-Vehicle protocol protection in other communication services like Ethernet and Flexray.

Appendix A

Code listing

```
#5.1 (1.)
import tkinter as tk
import random
import time
import can
import threading

# Create a CAN bus instance
bus = can.interface.Bus(channel='can1', bustype='socketcan')

# Function to simulate sensor data
def simulate_sensor_data():
    # Simulate engine RPM (random value between 1000 and 6000 RPM)
    rpm = random.randint(1000, 6000)

    # Simulate engine temperature (random value between 20 and 60
       degrees Celsius)
    temperature = random.randint(20, 60)

    # Simulate vehicle speed (random value between 0 and 120 km/h)
    speed = random.randint(0, 120)

    return rpm, temperature, speed

# Function to update sensor labels
def update_sensor_labels():
    rpm_label.config(text="RPM: {}".format(rpm))
    temperature_label.config(text="Temperature: {}*C".format(
       temperature))
    speed_label.config(text="Speed: {} km/h".format(speed))

# Function to update volume level
def update_volume(new_volume):
```

```
    global volume
    volume = int(new_volume)  # Convert to integer
    volume_label.config(text="Volume: {}%".format(volume))
    # Send CAN message for volume control
    volume_msg = can.Message(arbitration_id=0x400, data=[volume])
    bus.send(volume_msg)

# Function to update fuel level
def update_fuel(new_fuel):
    global fuel_level
    fuel_level = int(new_fuel)  # Convert to integer
    fuel_label.config(text="Fuel Level: {}%".format(fuel_level))
    # Send CAN message for fuel level
    fuel_msg = can.Message(arbitration_id=0x500, data=[fuel_level])
    bus.send(fuel_msg)

# Function to update RPM
def update_rpm():
    # Convert RPM to bytes
    rpm_bytes = rpm.to_bytes(2, byteorder='big')  # Assuming a 2-byte
        integer for RPM
    # Send CAN message for RPM
    rpm_msg = can.Message(arbitration_id=0x600, data=rpm_bytes)
    bus.send(rpm_msg)

# Function to update temperature
def update_temperature():
    # Send CAN message for temperature
    temperature_msg = can.Message(arbitration_id=0x700, data=[
       temperature])
    bus.send(temperature_msg)

# Function to update speed
def update_speed():
    # Send CAN message for speed
    speed_msg = can.Message(arbitration_id=0x800, data=[speed])
    bus.send(speed_msg)

# Function to drain fuel
def drain_fuel():
    global fuel_level
    while fuel_level > 0 and car_running:
        fuel_level -= 1
        update_fuel(fuel_level)
        time.sleep(2)

# Main loop to continuously send simulated sensor data
```

```
def send_sensor_data():
    while True:
        # Simulate sensor data
        global rpm, temperature, speed
        rpm, temperature, speed = simulate_sensor_data()
        update_sensor_labels()
        update_rpm()
        update_temperature()
        update_speed()
        time.sleep(0.1)

# Function to start or stop the car
def start_stop_car():
    global car_running
    if car_running:
        start_stop_button.config(text="Start Car")
        car_running = False
        # Stop updating values
        update_value_threads = [update_volume_thread,
           update_fuel_thread, update_rpm_thread, update_temp_thread,
            update_speed_thread]
        for thread in update_value_threads:
            if thread.is_alive():
                thread.join()  # Wait for threads to complete
    else:
        start_stop_button.config(text="Stop Car")
        car_running = True
        threading.Thread(target=drain_fuel).start()
        # Start updating values
        start_update_threads()

# Function to start updating values
def start_update_threads():
    global update_volume_thread, update_fuel_thread,
       update_rpm_thread, update_temp_thread, update_speed_thread
    update_volume_thread = threading.Thread(target=update_volume,
       args=(volume,))
    update_fuel_thread = threading.Thread(target=update_fuel, args=(
       fuel_level,))
    update_rpm_thread = threading.Thread(target=update_rpm)
    update_temp_thread = threading.Thread(target=update_temperature)
    update_speed_thread = threading.Thread(target=update_speed)
    update_volume_thread.start()
    update_fuel_thread.start()
    update_rpm_thread.start()
    update_temp_thread.start()
    update_speed_thread.start()
```

```
119
120 # Initialize the Tkinter window
121 root = tk.Tk()
122 root.title("Car Dashboard Simulation")
123
124 # RPM label
125 rpm_label = tk.Label(root, text="RPM: 0")
126 rpm_label.pack()
127
128 # Temperature label
129 temperature_label = tk.Label(root, text="Temperature: 0*C")
130 temperature_label.pack()
131
132 # Speed label
133 speed_label = tk.Label(root, text="Speed: 0 km/h")
134 speed_label.pack()
135
136 # Volume scale
137 volume = 0
138 volume_label = tk.Label(root, text="Volume: 0%")
139 volume_label.pack()
140 volume_scale = tk.Scale(root, from_=0, to=100, orient=tk.HORIZONTAL,
       command=update_volume)
141 volume_scale.pack()
142
143 # Fuel scale
144 fuel_level = 100
145 fuel_label = tk.Label(root, text="Fuel Level: 100%")
146 fuel_label.pack()
147 fuel_scale = tk.Scale(root, from_=0, to=100, orient=tk.HORIZONTAL,
       command=update_fuel)
148 fuel_scale.pack()
149
150 # Start/Stop button
151 car_running = False
152 start_stop_button = tk.Button(root, text="Start Car", command=
       start_stop_car)
153 start_stop_button.pack()
154
155 # Start sending sensor data
156 sensor_thread = threading.Thread(target=send_sensor_data)
157 sensor_thread.daemon = True  # Ensures thread exits when the main
       program exits
158 sensor_thread.start()
159
160 # Ensure proper shutdown of the CAN bus
161 def on_closing():
```

```
    global car_running
    car_running = False
    sensor_thread.join()  # Wait for the sensor thread to complete
    bus.shutdown()  # Properly shutdown the CAN bus
    root.destroy()

root.protocol("WM_DELETE_WINDOW", on_closing)

# Start the Tkinter event loop
root.mainloop()
```

```
#5.1 (2.)
import tkinter as tk
import random
import can
import subprocess
import os

# Create a CAN bus instance
bus = can.interface.Bus(channel='can1', interface='socketcan')

# Function to simulate sensor data
def simulate_sensor_data():
    # Simulate latitude (random value between -90 and 90 degrees)
    latitude = round(random.uniform(-90, 90), 6)

    # Simulate longitude (random value between -180 and 180 degrees)
    longitude = round(random.uniform(-180, 180), 6)

    # Simulate vehicle speed (random value between 0 and 120 km/h)
    speed = round(random.uniform(0, 120), 2)

    return latitude, longitude, speed

# Function to send CAN messages
def send_can_message(latitude, longitude, speed):
    # Convert latitude, longitude, and speed to integers
    lat_int = int((latitude + 90) * 10000)  # Scale and convert
       latitude to integer
    lon_int = int((longitude + 180) * 10000)  # Scale and convert
       longitude to integer
    speed_int = int(speed * 100)  # Scale and convert speed to
       integer

    # Pack the data into a byte array
    data = bytearray()
    data.extend(lat_int.to_bytes(4, byteorder='big'))  # Convert to 4
        bytes
```

```
34      data.extend(lon_int.to_bytes(4, byteorder='big'))  # Convert to 4
            bytes
35      data.extend(speed_int.to_bytes(2, byteorder='big'))  # Convert to
            2 bytes
36
37      # Create and send the CAN message
38      msg = can.Message(arbitration_id=0x123, data=data[:8],
           is_extended_id=False)
39      bus.send(msg)
40
41  # Function to update sensor labels and send CAN messages
42  def update_sensor_labels():
43      if generating_data:
44          # Simulate sensor data
45          latitude, longitude, speed = simulate_sensor_data()
46          location_name = "Location Name"  # Placeholder location name
47
48          # Update labels
49          latitude_label.config(text="Latitude: {}".format(latitude))
50          longitude_label.config(text="Longitude: {}".format(longitude)
               )
51          speed_label.config(text="Speed: {} km/h".format(speed))
52          location_label.config(text="Location: {}".format(
               location_name))
53
54          # Send CAN message
55          send_can_message(latitude, longitude, speed)
56
57      # Schedule the next update
58      root.after(1000, update_sensor_labels)
59
60  # Function to start or stop data generation
61  def toggle_data_generation():
62      global generating_data
63      generating_data = not generating_data
64      start_stop_button.config(text="Start" if not generating_data else
            "Stop")
65
66  # Function to stop the car
67  def stop_car():
68      global generating_data
69      generating_data = False
70      start_stop_button.config(text="Start")
71      # Optionally, send a CAN message indicating the car is stopped
72      send_can_message(0, 0, 0)
73
74  # Function to check for updates
```

```
def check_for_update():
    script_path = "/home/pi/Desktop/Vehicle5397/OTA_Update/
       Update_Script.py"
    venv_path = "myenv/bin/activate"

    try:
        # Navigate to the Vehicle5397 directory
        os.chdir(os.path.expanduser("~/Desktop/Vehicle5397"))

        # Activate the virtual environment and run the script
        command = f"python '{script_path}'"

        # Use subprocess to execute the command in a new shell
        subprocess.run(command, shell=True, check=True, executable='/
           bin/bash')
    except subprocess.CalledProcessError as e:
        print(f"Error occurred: {e}")

# Create the main window
root = tk.Tk()
root.title("Car Dashboard")

# Sensor labels
latitude_label = tk.Label(root, text="Latitude: ")
latitude_label.pack()
longitude_label = tk.Label(root, text="Longitude: ")
longitude_label.pack()
speed_label = tk.Label(root, text="Speed: ")
speed_label.pack()
location_label = tk.Label(root, text="Location: ")
location_label.pack()

# Start/Stop button
start_stop_button = tk.Button(root, text="Stop", command=
   toggle_data_generation)
start_stop_button.pack()

# Stop Car button
stop_car_button = tk.Button(root, text="Stop Car", command=stop_car)
stop_car_button.pack()

# Check for Update button
check_update_button = tk.Button(root, text="Check for Update",
   command=check_for_update)
check_update_button.pack()

# Initialize data generation state
```

```
generating_data = True

# Start updating sensor labels and sending CAN messages
update_sensor_labels()

# Run the GUI
root.mainloop()
```

```
#5.1 (2.) (a)
echo program_usb_boot_mode=1 | sudo tee -a /boot/config.txt
vcgencmd otp_dump | grep 17:
    #Expected Output 17:3020000a
```

```
#5.1 (2.) (c)
git clone https://source.denx.de/u-boot/u-boot.git
cd u-boot
git checkout v2024.07
sudo apt-get install gcc gcc-aarch64-linux-gnu
sudo apt-get install bc bison build-essential coccinelle \
  device-tree-compiler dfu-util efitools flex gdisk graphviz
     imagemagick \
  liblz4-tool libgnutls28-dev libguestfs-tools libncurses-dev \
  libpython3-dev libsdl2-dev libssl-dev lz4 lzma lzma-alone openssl \
  pkg-config python3 python3-asteval python3-coverage python3-
     filelock \
  python3-pkg-resources python3-pycryptodome python3-pyelftools \
  python3-pytest python3-pytest-xdist python3-sphinxcontrib.apidoc \
  python3-sphinx-rtd-theme python3-subunit python3-testtools \
  python3-virtualenv swig uuid-dev

  make rpi_3_b_plus_defconfig
  DTC=/usr/bin/dtc make
  CROSS_COMPILE=aarch64-linux-gnu- make
```

```
#5.1 (2.) (d)
cd u-boot
sudo cp u-boot.bin /boot/firmware/u-boot.bin
sudo nano /boot/firmware/comfig.txt
    #Add the following lines
    kernel=u-boot.bin
    enable_uart=1
```

```
#5.1 (2.) (e)
mmc dev 0
fatload mmc 0:1 ${kernel_addr_r} kernel8..img
fatload mmc 0:1 ${fdt_addr} bcm2710-rpi-3-b-plus.dtb
setenv bootargs "8250.nr_uarts=1 root=/dev/mmcblk0p2 rootwait console
   =ttyAMA0,115200"
```

```
setenv kernel_comp_addr_r 0x20000000
setenv kernel_comp_size 0x1000000
setenv load_all 'fatload mmc 0:1 ${kernel_addr_r} kernel8.img;
   fatload mmc 0:1 ${fdt_addr} bcm2710-rpi-3-b-plus.dtb' #can be used
    later to run the commands directly
saveenv
booti ${kernel_addr_r} - ${fdt_addr}
```

```
#5.2
sudo apt update
sudo apt install git build-essential libssl-dev libcurl4-openssl-dev
   libjson-c-dev uuid-dev
git clone https://github.com/sbabic/swupdate.git
cd swupdate
make raspi_defconfig
make menuconfig
make
sudo make install
```

Appendix B

Abbreviations

IEM Information Technology-Electrical Engineering-Mechatronics

OTA Over-The-Air

OEM Original Equipment Manufacturer

ECU Electronic Control Unit

ICU Infotainment Control Unit

TCU Telematics Control Unit

CCU Connectivity Control Unit

ECM Engine Control Unit (Module)

MND Mathematik, Naturwissenschaften und Datenverarbeitung

EGR Exhaust Gas Recirculation

OBD On-Board Diagnostics

DTC Diagnostic Trouble Codes

V2X Vehicle to Everything

V2V Vehicle to Vehicle

V2I Vehicle to Infrastructure

GPS Global Positioning System

UI User Interface

ICU Infotainment Control Unit

BCM Body Control Module

PCM Powertrain Control Module

CAN Controller Area Network

LIN Local Interconnect Network

TARA Threat Analysis and Risk Assessment

DoS Denial-of-Service

URI Uniform Resource Identifiers

IDS Intrusion Detection Systems

PKI Public Key Infrastructure

AES Advanced Encryption Standard

TLS Transport Layer Security

HSM Hardware Security Modules

KRACK Key Reinstallation Attacks

FOTA Firmware OVer-the-air

SOTA Software OVer-the-air

IOT Internet of things

RPM Revolutions per minute

GUI Graphical User Interface

GPIO General Purpose Input/Output

SPI Serial Peripheral Interface

SSH Secure Shell

VM Virtual Machine

OS Operating system

RAUC Robust Auto-Update Controller

GNU GRUB GNU GRand Unified Bootloader

NOOBS New Out Of Box Software

VPN Virtual Private Networks

CLI Command Line interface

SSBL Second Stage Bootloader

DMA Direct Memory Access

OTP One-Time-Password

HIL Hardware-in-the-loop

ADAS Advanced Driver Assistance Systems

AV Autonomous Vehicle

API Application programming interface

References

[1] P. Research, *Automotive electronic control unit (ecu) market*, Accessed: 2024-07-05, 2024. URL: https://www.precedenceresearch.com/automotive-electronic-control-unit-market#:~:text=On%20an%20average%2C%2015%2D20,growth%20of%20premium%20luxury%20cars.

[2] eInfochips, *Automotive ecu: An inevitable part of the automobiles*, Accessed: 2024-08-02, 2021. URL: https://www.einfochips.com/blog/automotive-ecu-an-inevitable-part-of-the-automobiles/.

[3] NXP Semiconductors, *The future of vehicles: The ecu-ization of automobiles*, 2020. URL: https://www.nxp.com/docs/en/white-paper/FVNECUA4WP.pdf.

[4] M. C. Moseman, "This car runs on code," *IEEE Spectrum*, 2009. URL: https://spectrum.ieee.org/this-car-runs-on-code.

[5] H. N. Security, *Automotive cybersecurity incidents doubled in 2019, up 605% since 2016*, https://www.helpnetsecurity.com/2020/01/06/automotive-cybersecurity-incidents/, Accessed: 2024-08-02, 2020.

[6] Upstream.auto, *2021 global automotive cybersecurity report*, https://library.cyentia.com/report/report_013871.html, Accessed: 2024-08-02, 2021.

[7] A. Wärn and M. Edström, "Safe and secure automotive over-the-air updates," in *Software Process Improvement and Capability Determination*, B. Rama and Ç. Gencel, Eds., Cham: Springer International Publishing, 2019, pp. 135–146, ISBN: 978-3-319-99130-6. DOI: 10.1007/978-3-319-99130-6_12. URL: https://link.springer.com/chapter/10.1007/978-3-319-99130-6_12.

[8] *Road vehicles – functional safety*, Accessed: YYYY-MM-DD, International Organization for Standardization, 2018. URL: https://www.iso.org/obp/ui/#iso:std:iso:26262:-2:ed-2:v1:en:sec:6.

[9] *Cybersecurity process framework for automotive systems*, Accessed: YYYY-MM-DD, Society of Automotive Engineers, 2016. URL: https://www.sae.org/standards/content/j3061_201601/.

[10] I. O. for Standardization, *ISO/SAE 21434:2021 Road Vehicles – Cybersecurity Engineering*. International Organization for Standardization, 2021. URL: https://www.iso.org/standard/70918.html.

[11] S. Halder, A. Ghosal, and M. Conti, "Secure over-the-air software updates in connected vehicles: A survey," *Computer Networks*, vol. 178, p. 107 343, 2020, ISSN: 1389-1286. DOI: https://doi.org/10.1016/j.comnet.2020.107343. URL: https://www.sciencedirect.com/science/article/pii/S1389128619314963.

[12] D. K. Nilsson and U. E. Larson, "Secure firmware updates over the air in intelligent vehicles," in *ICC Workshops - 2008 IEEE International Conference on Communications Workshops*, 2008, pp. 380–384. DOI: 10.1109/ICCW.2008.78.

[13] K. Mansour, W. Farag, and M. ElHelw, "Airodiag: A sophisticated tool that diagnoses and updates vehicles software over air," in *2012 IEEE International Electric Vehicle Conference*, 2012, pp. 1–7. DOI: 10.1109/IEVC.2012.6183181.

[14] Z. Wu, T. Liu, X. Jia, and C. Sun, "Security design of ota upgrade for intelligent connected vehicle," in *Proceedings of the 2021 1st International Conference on Control and Intelligent Robotics*, ser. ICCIR '21, Guangzhou, China: Association for Computing Machinery, 2021, pp. 736–739, ISBN: 9781450390231. DOI: 10.1145/3473714.3473851. URL: https://doi.org/10.1145/3473714.3473851.

[15] H. Guissouma, C. P. Hohl, F. Lesniak, M. Schindewolf, J. Becker, and E. Sax, "Lifecycle management of automotive safety-critical over the air updates: A systems approach," *IEEE Access*, vol. 10, pp. 57 696–57 717, 2022. DOI: 10.1109/ACCESS.2022.3176879.

[16] H. Bojinov, E. Bursztein, X. Boyen, and D. Boneh, "Kamouflage: Loss-resistant password management," in *2015 IEEE Symposium on Security and Privacy*, IEEE, 2015, pp. 535–550. DOI: 10.1109/SP.2015.39. URL: https://www.ieee-security.org/TC/SP2015/papers-archived/6949a535.pdf.

[17] M. Marlinspike, "New tricks for defeating ssl in practice," in *Black Hat DC*, 2009. URL: https://www.blackhat.com/presentations/bh-dc-09/Marlinspike/BlackHat-DC-09-Marlinspike-Defeating-SSL.pdf.

[18] B. Security, *What is the beast attack?* Beagle Security Blog, Accessed: 2024-08-06, 2024. URL: https://beaglesecurity.com/blog/vulnerability/browser-exploit-against-ssl-tls.html#:~:text=Vulnerability-,What%20is%20the%20BEAST%20attack%3F,encryption%2Fdecryption%20of%20communication%20channels.

[19] K. Bhargavan and G. Leurent, *Sweet32: Birthday attacks on 64-bit block ciphers in tls and openvpn*, https://sweet32.info/SWEET32_CCS16.pdf, 2016.

[20] M. Vanhoef and F. Piessens, "Key reinstallation attacks: Forcing nonce reuse in wpa2," in *Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security*, ACM, 2017, pp. 1313–1328. URL: https://papers.mathyvanhoef.com/ccs2017.pdf.

[21] P. Kocher, D. G. Horn, A. Fogh, M. Gammel, Y. Yarom, and M. Hamburg, "Spectre attacks: Exploiting speculative execution," *Communications of the ACM*, vol. 62, no. 7, pp. 93–101, 2019. URL: https://spectreattack.com/spectre.pdf.

[22] M. Lipp, M. Schwarz, D. Gruss, T. Prescher, W. Haas, S. Mangard, P. Kocher, D. Genkin, Y. Yarom, and M. Hamburg, "Meltdown: Reading kernel memory from user space," *Communications of the ACM*, vol. 62, no. 6, pp. 117–125, 2018. URL: https://meltdownattack.com/meltdown.pdf.

[23] M. S. R. Center, *Cve-2019-0708: Remote desktop services remote code execution vulnerability*, https://msrc.microsoft.com/update-guide/en-US/advisory/CVE-2019-0708, Accessed: 2024-08-06, 2019.

[24] J. R. Lab, *Ripple20: 19 zero-day vulnerabilities amplified by the supply chain*, JSOF Research Lab, 2020. URL: https://www.jsof-tech.com/ripple20/.

[25] T. Limbasiya, A. Ghosal, and M. Conti, "Autosec: Secure automotive data transmission scheme for in-vehicle networks," in *Proceedings of the 23rd International Conference on Distributed Computing and Networking*, ser. ICDCN '22, Delhi, AA, India: Association for Computing Machinery, 2022, pp. 208–216, ISBN: 9781450395601. DOI: 10.1145/3491003.3491024. URL: https://doi.org/10.1145/3491003.3491024.

[26] M. Kathiresh, N. Ramachandran, B. Moses, and A. Ravikumar, "A hybrid approach to enhance data security in wireless vehicle firmware update process," *Wireless Personal Communications*, vol. 125, pp. 1–20, Jul. 2022. DOI: 10.1007/s11277-022-09571-8.

[27] S. Khandelwal and S. Shreejith, "A lightweight fpga-based ids-ecu architecture for automotive can," in *2022 International Conference on Field-Programmable Technology (ICFPT)*, 2022, pp. 1–9. DOI: 10.1109/ICFPT56656.2022.9974508.

[28] H. N. Nguyen, S. Tavakoli, S. A. Shaikh, and O. Maynard, "Developing a qrng ecu for automotive security: Experience of testing in the real-world," in *2019 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2019, pp. 61–68. DOI: 10.1109/ICSTW.2019.00033.

[29] S. Mahmood, H. N. Nguyen, and S. A. Shaikh, "Systematic threat assessment and security testing of automotive over-the-air (ota) updates," *Vehicular Communications*, vol. 35, p. 100 468, 2022, ISSN: 2214-2096. DOI: https://doi.org/10.1016/j.vehcom.2022.100468. URL: https://www.sciencedirect.com/science/article/pii/S2214209622000158.

[30] J. Wu, K. Ota, M. Dong, and C. Li, "A hierarchical security framework for defending against sophisticated attacks on wireless sensor networks in smart cities," *IEEE Access*, vol. 4, pp. 416–424, 2016. DOI: 10.1109/ACCESS.2016.2517321.

[31] S. Mahmood, A. Fouillade, H. N. Nguyen, and S. A. Shaikh, "A model-based security testing approach for automotive over-the-air updates," in *2020 IEEE International Conference on Software Testing, Verification and Validation Workshops (ICSTW)*, 2020, pp. 6–13. DOI: 10.1109/ICSTW50294.2020.00019.

[32] H. Kexun, W. Changyuan, H. Yanyan, and F. Xiyu, "Research on cyber security technology and test method of ota for intelligent connected vehicle," Jun. 2020, pp. 194–198. DOI: 10.1109/ICBAIE49996.2020.00048.

[33] J. Wu, M. Dong, K. Ota, J. Li, and Z. Guan, "Fcss: Fog-computing-based content-aware filtering for security services in information-centric social networks," *IEEE Transactions on Emerging Topics in Computing*, vol. 7, no. 4, pp. 553–564, 2019. DOI: 10.1109/TETC.2017.2747158.

[34] K. Doddapaneni, R. Lakkundi, S. Rao, S. G. Kulkarni, and B. Bhat, "Secure fota object for iot," in *2017 IEEE 42nd Conference on Local Computer Networks Workshops (LCN Workshops)*, 2017, pp. 154–159. DOI: 10.1109/LCN.Workshops.2017.78.

[35] Uptane. "Best practices for deployment." (2024), URL: https://uptane.org/docs/latest/deployment/best-practices.

[36] F. Networks. "Audi future-proofs tech vision and app innovation with nginx." Accessed: 2024-09-04. (2021), URL: https://www.f5.com/case-studies/audi-future-proofs-tech-vision-and-app-innovation-with-nginx.

[37] L. Abrams. "Windows update downgrade attack unpatches fully updated systems," BleepingComputer. (2023), URL: https://www.bleepingcomputer.com/news/microsoft/windows-update-downgrade-attack-unpatches-fully-updated-systems/ (visited on 09/19/2024).

[38] S. Angel and J. Bonneau. "Guest blog post: Rollback attack," Mozilla. (2020), URL: https://blog.mozilla.org/attack-and-defense/2020/10/12/guest-blog-post-rollback-attack/ (visited on 09/19/2024).

[39] K. Singh. "Tesla's battery health test - see your battery health in app or in service mode." (2024), URL: https://www.notateslaapp.com/news/2049/teslas-battery-health-test-see-your-battery-health-in-app-or-in-service-mode.

[40] I. Tesla. "Connectivity." (2024), URL: https://www.tesla.com/support/connectivity.

[41] A. Intuition, *Hil simulation - applied intuition*, https://www.appliedintuition.com/products/hil-sim, 2024.

[42] K. Technologies, *Hardware-in-the-loop (hil) solutions for adas*, https://www.konrad-technologies.com/solutions/adas/hardware-in-the-loop-hil, 2024.

[43] W. by Toyota, *Ad/adas technology - woven by toyota*, https://woven.toyota/en/ad-adas-technology/, 2024.

[44] arXiv, *Systematic review: Anomaly detection in connected and autonomous vehicles*, https://ar5iv.labs.arxiv.org/html/2405.02731, 2024.

[45] A. C. Car, *Gm's connected car os ultifi for ota updates and more*, https://www.autoconnectedcar.com/2021/09/gms-connected-car-os-ultifi-for-ota-updates-and-more/, 2021.

[46] Carscoops, *Gm unveils ultifi: A new software platform that can update nearly every part of a vehicle over the air*, https://www.carscoops.com/2021/09/gm-unveils-ultifi-a-new-software-platform-that-can-update-nearly-every-part-of-a-vehicle-over-the-air/, 2021.

[47] B. Insights, *Bmw diagnostic software*, https://bmwinsights.com/bmw-diagnostic-software/.

[48] B. USA, *Software updates*, https://www.bmwusa.com/owners/software-updates.html.

[49] S. Bhatta and W. Shi, "Ota: A new approach for open transfer learning," in *Proceedings of the Conference on Knowledge Graphs and Semantic Computing (KGSC)*, 2023. URL: https://weisongshi.org/papers/Bhatta23-OTA.pdf.